How fast is GPUPDAL? A separate comparative speed test against PDAL, LAStools and QGIS

Report date 2026-08-21. Author-produced by an automated run on this repository (branch `p3-skewness-resident`, commit `cc12b04eb`). It is separate from the maintained reference suite, not independent third-party validation.

> **What this report is.** We took the program in this repository, now called **GPUPDAL**, and timed it doing everyday lidar point-cloud jobs. The captured raw results use its historical internal executable identifier, `pdg`; this regenerated report uses the current GPUPDAL product label without changing any benchmark value. We timed the same jobs with three other well-known tools: **PDAL**, **LAStools**, and **QGIS**. We used four point clouds of different sizes (1, 4, 16 and 47 million points) and three different computers (a fast desktop with a big graphics card, a cloud server with a data-centre GPU, and an older budget machine with a small GPU). This report shows the timings as tables and charts, shows what the inputs and outputs look like, and explains where the comparison is fair and where it is not.
>
> **Evidence boundary.** This is one capture per machine with warm-cache medians of 3 timed repeats at 1M/4M, 2 at 16M, and 1 at 47M. The three-machine bundle does not establish the separate ten-GPU claim, a 3DEP population claim, production readiness, or third-party validation.

Contents

1. The short version

- **GPUPDAL was the fastest tool, or tied for fastest, on almost every job at every size we tried.** On the workstation with 1 million points, PDAL 2.10.0 (the program GPUPDAL is based on) took on average **2.3 times longer** than GPUPDAL over 24 jobs (from 1.0× to 11.7× depending on the job); GPUPDAL was faster on 24 of the 24. The gains are biggest on jobs that look at each point's neighbours — noise removal, surface features, classification clean-up — where PDAL took 7 to 12 times longer, and smallest on jobs that are mostly reading and writing files, where PDAL took 1.1 to 3.3 times longer. On the 47-million-point tile PDAL took on average 2.6 times longer.
- **Against LAStools** (the fast, partly commercial tools), GPUPDAL was faster on 19 of 20 comparable jobs at 1 million points; averaged over those jobs LAStools took 2.0 times longer (from 0.8× to 8.6×). Its best case against GPUPDAL was the point-density map. LAStools often uses a different method for a job of the

same name, and its paid tools ran unlicensed, so read those rows as "same task, different program", not as a like-for-like race.

- **Against QGIS**, GPUPDAL was much faster: the QGIS engine (pdal_wrench) took on average 2.6 times longer at 1 million points, and QGIS run through `qgis_process` took 7.8 times longer, partly because QGIS itself needs about 1.2 seconds to start. On big files the QGIS engine's tiling job (which writes uncompressed tiles from many threads) was as fast as GPUPDAL, and its COPC maker (untwine) was the fastest COPC writer of all.
- **The graphics card matters for only a few jobs.** On the workstation, GPUPDAL with the GPU hidden took on average 1.04 times as long as with the GPU on — nearly the same. Most of GPUPDAL's speed comes from doing CPU work better (many threads, faster reading and writing, less copying). At 1 million points the GPU made a big difference for surface-normal and shape-feature computation (the CPU-only run took 3.9 times longer) and nothing for jobs such as compressing or reprojecting.
- **GPUPDAL's automatic mode is cautious — sometimes too cautious.** On files of 4 million points and more, GPUPDAL's automatic mode did not use the GPU at all on this workstation, and its times matched the CPU-only run. When we forced the GPU on, the surface-features job ran 5.5× / 6.1× / 5.9× faster at 4 million / 16 million / 47 million points, and the height-above-ground jobs also sped up. So there is real GPU speed that the shipped decision table leaves unused on big files. Forcing the GPU everywhere is not free either: on some jobs (tiling, DSM) it made things slower.
- **Bigger files, same story.** The speed advantage held or grew as the files grew from 1 to 47 million points. On the 47-million-point tile the largest differences were on the neighbourhood jobs, where PDAL took 2.4 to 14 times longer (for example surface features: PDAL 6.6 minutes, GPUPDAL 2.8 minutes automatic, 28 seconds with the GPU forced on).
- **Other computers.** The same pattern appeared on the two rented machines (section 5): on the Cloud server (A100) PDAL took on average 2.2× longer than GPUPDAL at 1 million points and 2.6× at 47 million; on the Budget PC (RTX 3060) PDAL took on average 1.8× longer than GPUPDAL at 1 million points and 2.0× at 47 million. The advantage over PDAL is smaller on the budget machine, which has fewer CPU threads for GPUPDAL to use, and the "features job only uses the GPU automatically at 1 million points" behaviour was the same on all three machines.
- **Outputs.** Deterministic file outputs were checked byte-for-byte against PDAL 2.10.0. COPC bytes differed in this historical harness and are reported as non-identical, not waived as a conformance pass; the versioned canonical comparator now supplies separate semantic evidence. LAStools and QGIS produce different results for several jobs because they use different methods.

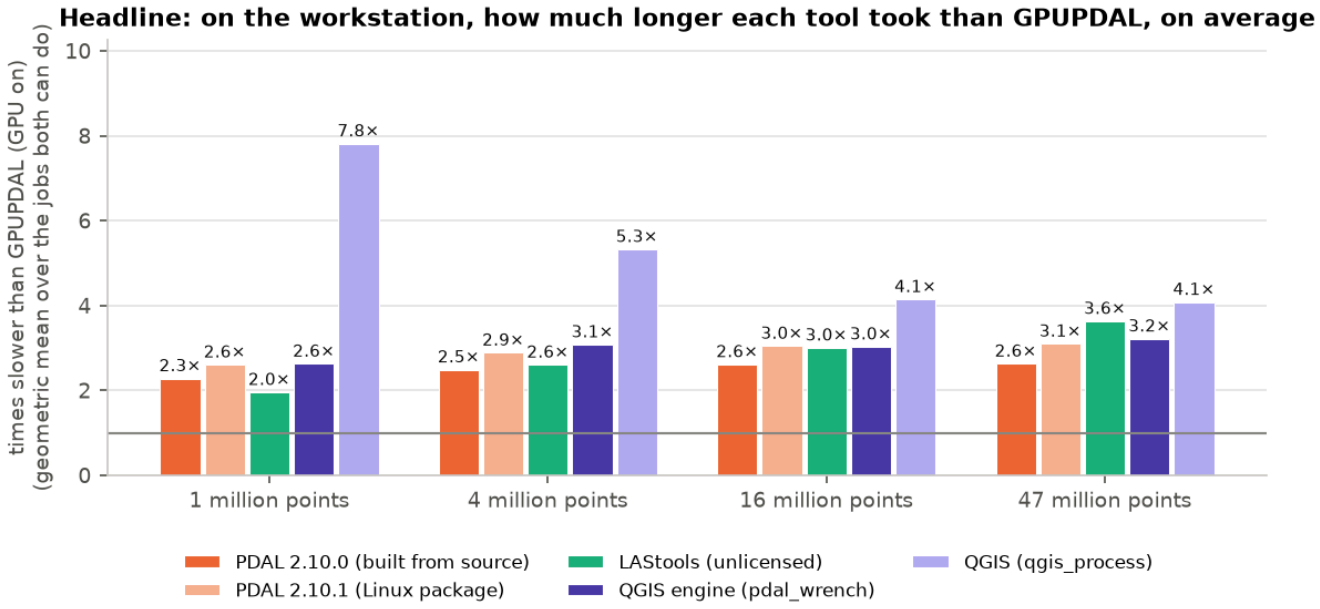


Figure 1. For each file size, how many times longer each tool took than GPUPDAL with the GPU on, averaged (geometric mean) over the jobs the two tools can both do (18 to 24 jobs, see Table 1). 1× would mean the same speed. Workstation.

File size	PDAL 2.10.0 (built)	PDAL 2.10.1 (package)	LAStools	QGIS engine (pdal_wrench)	QGIS (qgis_process)
1 million points	2.3× (from 1.0× to 11.7×, 24 jobs)	2.6× (from 1.0× to 15.6×, 24 jobs)	2.0× (from 0.8× to 8.6×, 20 jobs)	2.6× (from 0.8× to 10.4×, 19 jobs)	7.8× (from 2.0× to 35.2×, 18 jobs)
4 million points	2.5× (from 1.0× to 14.1×, 22 jobs)	2.9× (from 1.0× to 19.0×, 22 jobs)	2.6× (from 1.0× to 34.2×, 20 jobs)	3.1× (from 0.6× to 16.7×, 19 jobs)	5.3× (from 0.8× to 36.2×, 18 jobs)
16 million points	2.6× (from 1.0× to 15.3×, 22 jobs)	3.0× (from 0.9× to 20.5×, 22 jobs)	3.0× (from 0.9× to 133.4×, 20 jobs)	3.0× (from 0.3× to 20.8×, 19 jobs)	4.1× (from 0.4× to 33.8×, 18 jobs)
47 million points	2.6× (from 0.9× to 13.8×, 22 jobs)	3.1× (from 0.9× to 18.3×, 22 jobs)	3.6× (from 1.1× to 390.2×, 20 jobs)	3.2× (from 0.3× to 18.3×, 19 jobs)	4.1× (from 0.3× to 33.8×, 18 jobs)

Table 1. How many times longer each tool took than GPUPDAL (GPU on, automatic), averaged over the jobs both tools can do, with the best and worst single job in brackets. Workstation.

2. What was tested

2.1 The tools

All eight columns in the charts are listed here. Three of them are the same GPUPDAL program in three modes, and two are PDAL in two builds, so there are really four products: GPUPDAL, PDAL, LAStools and QGIS.

Column in charts	What it is	How it was run
GPUPDAL (this project) — GPU on, automatic	The program built from this repository. It is a copy of PDAL with a faster engine that can use an NVIDIA graphics card (GPU). In this mode it decides by itself, per job, whether to use the GPU or the CPU. This is the default mode a user gets.	`gpupdal pipeline job.json`
GPUPDAL — CPU only	The same program, run as if the computer had no NVIDIA graphics card (`CUDA_VISIBLE_DEVICES=""`). This shows what a user without a GPU would get.	same, with the GPU hidden
GPUPDAL — GPU forced on	The same program told to use the GPU for every job that has a GPU version, even where it would normally decide not to (`PDG_EXPERIMENTAL_CUDA_HYBRID=1`).	same, with the GPU forced
PDAL 2.10.0, built from source	The standard open-source PDAL library and command line, built from the exact upstream commit this project is based on. Its results are the reference: GPUPDAL preserves its semantics, with byte equality for deterministic outputs and canonical comparison for inherently nondeterministic containers.	`pdal pipeline job.json`
PDAL 2.10.1, Linux package	PDAL as installed from the Arch Linux package repository — what a typical Linux user would install.	`pdal pipeline job.json`
LAStools 250207 (Linux, unlicensed)	The well-known LAStools programs from rapidlasso, native 64-bit Linux versions. Some tools (ground, height, DTM, DSM, noise, thin, tile, clip) are paid software; they were run without a licence, which is allowed for testing. The maker says the unlicensed versions may add small noise to results and may refuse or slow down on files with more than 3 million points.	`lasground_new64 -i in.laz -o out.laz` and similar
QGIS engine (pdal_wrench 1.5.1)	The command-line program that QGIS uses behind the scenes for point-cloud processing. It is built on the PDAL library and can split work over many CPU threads. Run directly, without QGIS.	`pdal_wrench classify_ground --input … --output … --threads 24`
QGIS 4.2 (qgis_process)	The QGIS desktop GIS, run from the command line with `qgis_process`. This includes QGIS starting up (about 1.2 s) and then calling pdal_wrench. It is what a user gets from the QGIS Processing toolbox.	`qgis_process run pdal:classifyground --INPUT=… --OUTPUT=…`

Versions in the run: pdal 2.10.1 (git-version: Release); pdal_wrench version: 1.5.1 (PDAL version: 2.10.1, GDAL version: 3.13.1); QGIS 4.2.0-Belém do Pará 'Belém do Pará' (exported); LAStools laszip (by info@rapidlasso.de) version 250207.

2.2 The computers

Computer	Plain description	CPU	GPU	Memory	CPU limit (container quota)
Workstation (Ryzen 9 7900 + RTX 4090)	A desktop PC: AMD Ryzen 9 7900 (12 cores, 24 threads), 62 GB of memory, NVIDIA GeForce RTX 4090 graphics card (24 GB), Arch Linux.	AMD Ryzen 9 7900 12-Core Processor	NVIDIA GeForce RTX 4090, 24564 MiB, 610.43.03	62.0 GB	no limit
Cloud server (EPYC 7532 + A100 40 GB)	A rented data-centre machine: AMD EPYC 7532 (a container allowed to use about 31 of its 64 threads), 503 GB of memory, NVIDIA A100 SXM4 data-centre GPU (40 GB), Ubuntu 24.04.	AMD EPYC 7532 32-Core Processor	NVIDIA A100-SXM4-40GB, 40960 MiB, 580.159.03	503.7 GB	3072000 100000
Budget cloud PC (Xeon E5-2697A v4 + RTX 3060 12 GB)	A rented older machine: Intel Xeon E5-2697A v4 from 2016 (a container allowed about 15 threads), 125 GB of memory, NVIDIA GeForce RTX 3060 (12 GB), Ubuntu 24.04.	Intel(R) Xeon(R) CPU E5-2697A v4 @ 2.60GHz	NVIDIA GeForce RTX 3060, 12288 MiB, 595.84	125.7 GB	1536000 100000

The two cloud machines were rented by the hour from Vast.ai for this test and destroyed afterwards. On rented machines the container is only allowed a share of the CPU (the "quota" column: 3072000/100000 means about 30.7 CPU threads), while the programs see all of the machine's threads, so multi-threaded tools may over-subscribe there. QGIS itself was only installed on the workstation; on the cloud machines only its engine (pdal_wrench, built from source against the same PDAL 2.10.0) was measured. PDAL from a Linux package was measured only on the workstation.

2.3 The point clouds

All four inputs are real airborne laser scans (lidar). The three smaller ones are the first 1, 4 and 16 million points of one tile of a 2018 survey of hilly, forested land in New York State (USA); this repository's own benchmark fixtures are cut from the same tile, and, as in those fixtures, the coordinates are treated as if they were in the Dutch national grid (EPSG:28992) — the numbers happen to fall inside that grid's range. This only matters for the reprojection jobs, where the arithmetic is the same whatever the ground truth. Because the points are stored in the order the scanner produced them, the 1- and 4-million-point files are thin east–west strips of the tile, and the 16-million-point file covers the whole 1.5 km × 1.5 km tile. The largest input is a complete public Dutch AHN4 tile (25GN1_01, a built-up area) with 47 million points, colours and near-infrared, covering about 1 km × 1.3 km. Every input was written by PDAL 2.10.0 (compressed LAZ, LAS 1.4, point format 7 or 8, coordinate system stamped in the header) so that all tools read exactly the same bytes. For jobs that start from a "ground-classified" file, PDAL 2.10.0's SMRF ground filter was run once beforehand and its output was the input for every tool.

Input	Points	Area covered	Density	LAZ size	LAS size
1 million points	1,000,000	1,500 m × 77 m	8.7 points/m²	7 MB	36 MB
4 million points	4,000,000	1,500 m × 282 m	9.5 points/m²	27 MB	144 MB
16 million points	16,000,000	1,500 m × 1,075 m	9.9 points/m²	107 MB	576 MB
47 million points	47,478,228	1,040 m × 1,290 m	35.4 points/m²	333 MB	1,804 MB

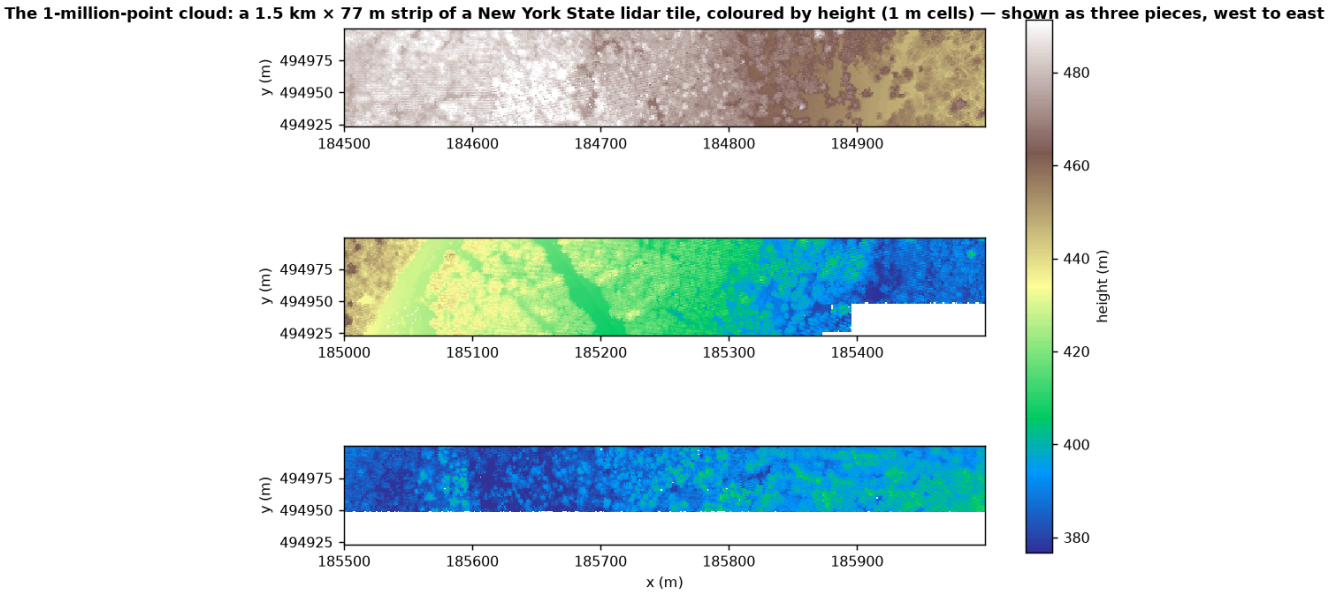


Figure 2. The 1-million-point input, seen from above, coloured by height. It is a thin east–west strip (1.5 km × 77 m), shown here in three pieces.

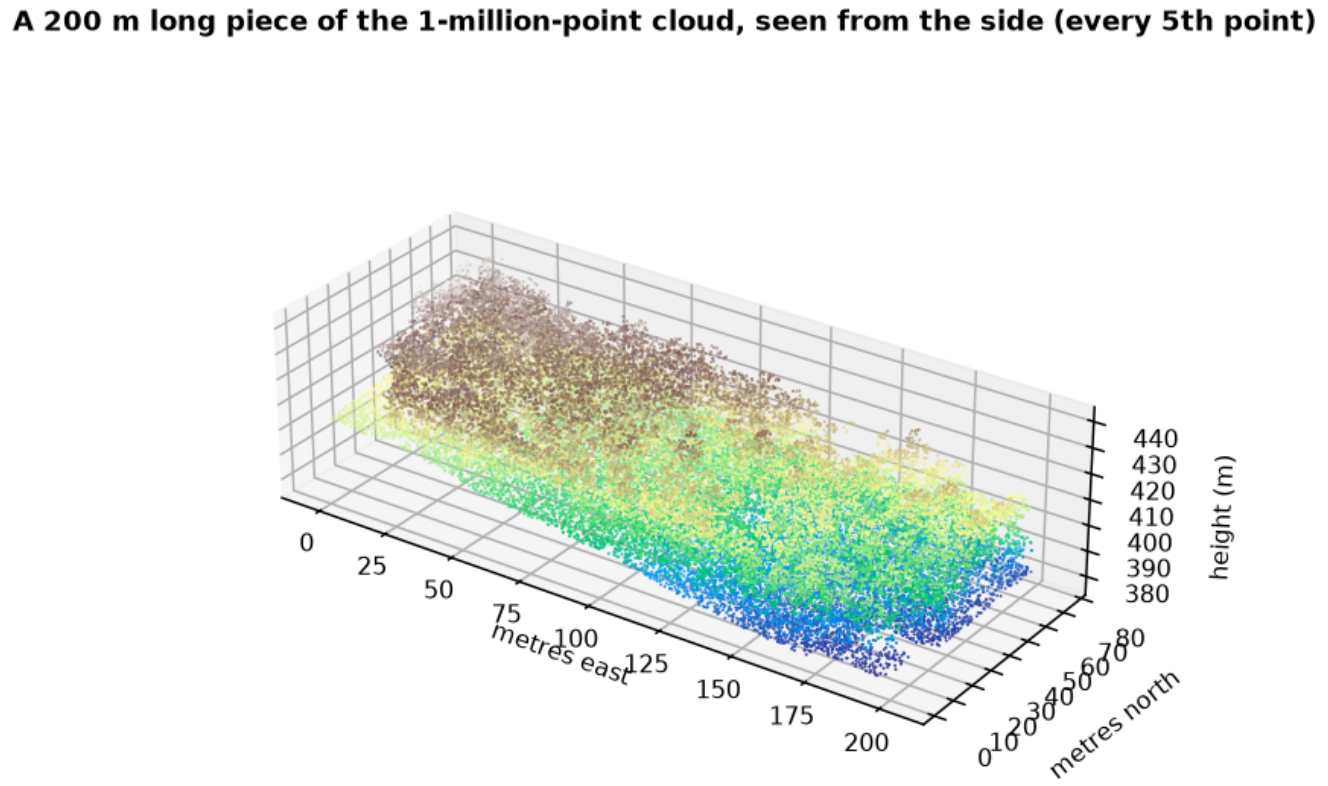


Figure 3. A 300 m long piece of the same cloud seen from the side: buildings, trees and ground are visible.

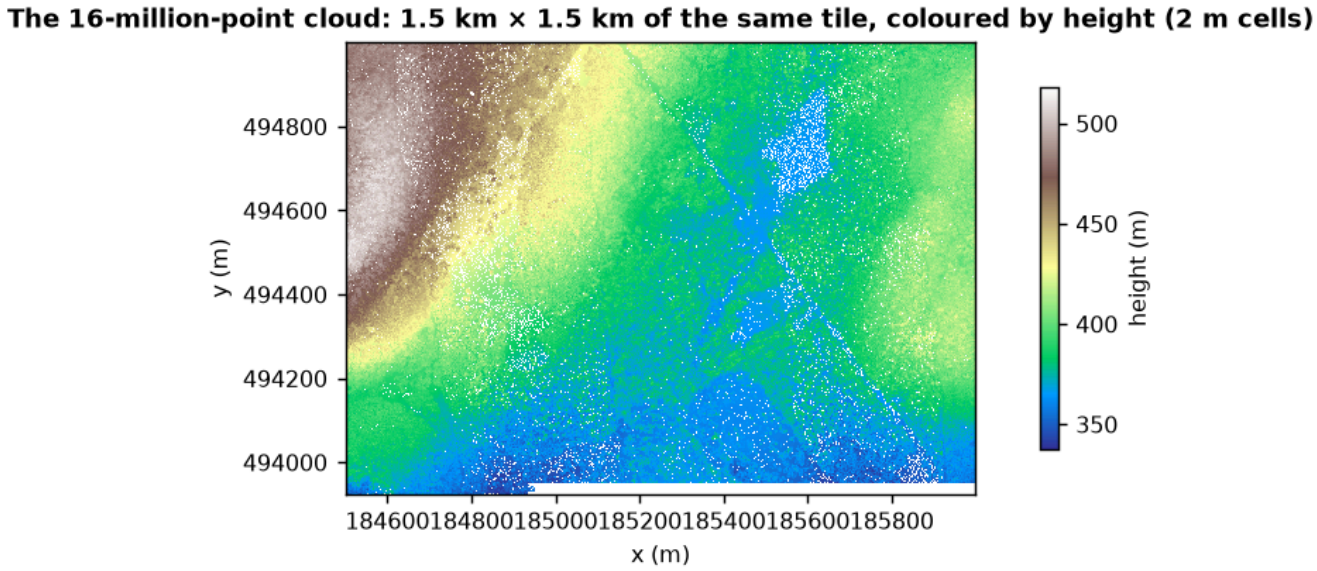


Figure 4. The 16-million-point input: the full 1.5 km × 1.5 km area.

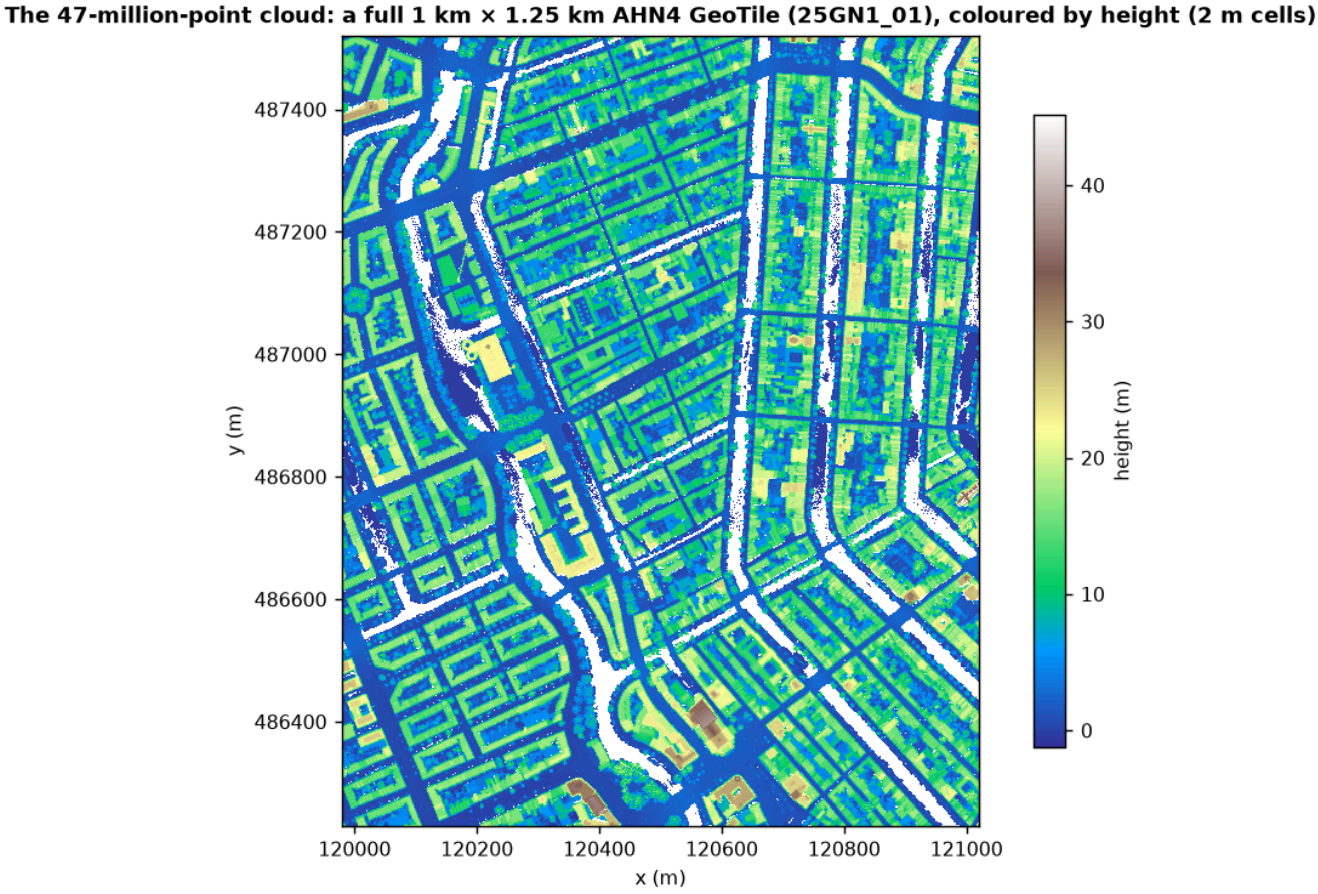


Figure 5. The 47-million-point input: a complete AHN4 tile.

2.4 The jobs

We chose jobs that lidar users do all the time. Each job is described in plain words below, with the exact way each tool was asked to do it. Where a tool cannot do a job it is left out for that job. Where a tool does the job with a different method (so the result is not the same), that is noted — the timing is still shown because a user would still use that tool for that job, but the comparison is then "same task, different method".

Family	Job	What it means	Tools that can do it	Notes on differences
Files and formats	**Compress LAS to LAZ**	Take an uncompressed LAS file and write it as a compressed LAZ file.	GPUPDAL (GPU), PDAL 2.10.0, LAStools, QGIS engine, QGIS	
Files and formats	**Decompress LAZ to LAS**	Take a compressed LAZ file and write it as an uncompressed LAS file.	GPUPDAL (GPU), PDAL 2.10.0, LAStools, QGIS engine, QGIS	
Files and formats	**Make a COPC file**	Turn a LAZ file into a cloud-optimized COPC file (a LAZ file with a built-in spatial index).	GPUPDAL (GPU), PDAL 2.10.0, LAStools, QGIS engine, QGIS	QGIS engine: untwine, the indexer QGIS ships
Files and formats	**Read the file summary**	Read the file and print its summary: point count, extent, and other header facts.	GPUPDAL (GPU), PDAL 2.10.0, LAStools, QGIS engine, QGIS	
Files and formats	**Merge two files**	Read two LAZ files (the west half and the east half of the area) and write one merged LAZ file.	GPUPDAL (GPU), PDAL 2.10.0, LAStools, QGIS engine, QGIS	
Files and formats	**Split into 256 m tiles**	Cut the point cloud into square 256 m tiles and write one file per tile.	GPUPDAL (GPU), PDAL 2.10.0, LAStools, QGIS engine, QGIS	QGIS engine: pdal_wrench writes the tiles as uncompressed LAS; QGIS: QGIS writes the tiles as uncompressed LAS
Files and formats	**Read a window from a COPC file**	Open a COPC file, read only the points inside the middle rectangle at 1 m resolution, compute statistics, and write a LAS file.	GPUPDAL (GPU), PDAL 2.10.0	
Coordinates and clipping	**Reproject to Web Mercator**	Change the map coordinates of every point from the Dutch national grid (EPSG:28992) to Web Mercator (EPSG:3857) and write a LAZ file.	GPUPDAL (GPU), PDAL 2.10.0, LAStools, QGIS engine, QGIS	

Family	Job	What it means	Tools that can do it	Notes on differences
Coordinates and clipping	**Crop to a rectangle, then reproject**	Keep only the points inside a rectangle covering the middle of the area, reproject them to Web Mercator, and write a LAZ file (the most common everyday PDAL job).	GPUPDAL (GPU), PDAL 2.10.0, LAStools, QGIS engine	
Coordinates and clipping	**Clip with a polygon**	Keep only the points inside a polygon shape (a rectangle with a hole in it, plus a second rectangle) and write a LAZ file.	GPUPDAL (GPU), PDAL 2.10.0, LAStools, QGIS engine, QGIS	
Reducing points	**Thin to about one point per metre**	Reduce the point cloud so that no two kept points are closer than 1 m (Poisson sampling), and write a LAZ file.	GPUPDAL (GPU), PDAL 2.10.0, LAStools, QGIS engine, QGIS	LAStools: lasthin keeps one point per 1 m grid cell (a different rule with a similar result)
Reducing points	**Keep one point per 2.5 m cell**	Reduce the point cloud to one original point per 2.5 m cell (the point nearest the cell's centre) and write a LAZ file.	GPUPDAL (GPU), PDAL 2.10.0, LAStools	LAStools: lasthin -central: 2-D cells (PDAL/pdg use 3-D voxels)
Ground and heights	**Find the ground**	Decide which points are bare ground (ground classification, SMRF method) and write a LAZ file with the ground points labelled.	GPUPDAL (GPU), PDAL 2.10.0, LAStools, QGIS engine, QGIS	LAStools: lasground_new uses its own algorithm, not SMRF
Ground and heights	**Height above ground**	Starting from a file whose ground points are already labelled, work out how high every point is above the ground (nearest ground point) and store that height with each point.	GPUPDAL (GPU), PDAL 2.10.0, LAStools, QGIS engine, QGIS	LAStools: lasheight uses a TIN of the ground points
Ground and heights	**Find the ground, then heights (one job)**	Do the two steps above as one job: find the ground, then give every point its height above ground, and write one LAZ file. Tools without a pipeline run two commands in a row.	GPUPDAL (GPU), PDAL 2.10.0, LAStools, QGIS engine, QGIS	
Rasters (maps)	**Bare-earth elevation map (DTM)**	From a file whose ground points are labelled, make a 1 m raster (GeoTIFF) of the ground height.	GPUPDAL (GPU), PDAL 2.10.0, LAStools, QGIS engine, QGIS	PDAL 2.10.0: inverse-distance weighting of the ground points in each cell; LAStools: las2dem builds a triangle mesh (TIN) and rasterizes it; QGIS engine: average ground height in each cell; QGIS: average ground height in each cell

Family	Job	What it means	Tools that can do it	Notes on differences
Rasters (maps)	**Surface elevation map (DSM)**	Make a 1 m raster (GeoTIFF) of the surface height using the first return of every laser pulse (tree tops, roofs, and ground where nothing is above it).	GPUPDAL (GPU), PDAL 2.10.0, LAStools, QGIS engine, QGIS	PDAL 2.10.0: highest first-return point per cell; LAStools: highest first-return point per cell; QGIS engine: pdal_wrench can only average the first returns in each cell, not pick the highest; QGIS: QGIS can only average the first returns in each cell, not pick the highest
Rasters (maps)	**Point density map**	Make a 1 m raster where each cell holds the number of points that fell inside it.	GPUPDAL (GPU), PDAL 2.10.0, LAStools, QGIS engine, QGIS	
Rasters (maps)	**Outline of the covered area**	Work out a polygon that outlines where the points are and write it as a vector file.	GPUPDAL (GPU), PDAL 2.10.0, LAStools, QGIS engine, QGIS	PDAL 2.10.0: hexagon-bin outline printed as text; LAStools: concave hull; QGIS engine: hexagon-bin outline; QGIS: hexagon-bin outline
Cleaning	**Find noise points**	Find isolated stray points (statistical outliers, 8 neighbours, 2 standard deviations) and label them as noise; write a LAZ file.	GPUPDAL (GPU), PDAL 2.10.0, LAStools, QGIS engine, QGIS	LAStools: lasnoise looks for isolated points in a 3-D grid (a different rule)
Cleaning	**Remove noise, then thin (one job)**	Find and drop the noise points, then thin to about one point per metre, and write a LAZ file. Tools without a pipeline run two commands in a row.	GPUPDAL (GPU), PDAL 2.10.0, LAStools, QGIS engine, QGIS	
Cleaning	**Ground, noise, and neighbour vote (one job)**	Find the ground, label noise points, then let each unclassified point take the majority label of its 7 nearest neighbours; write a LAZ file.	GPUPDAL (GPU), PDAL 2.10.0	
Neighbourhood analysis	**Surface normals and shape features**	For every point, look at its 8 nearest neighbours and compute the surface normal and shape features (how line-like, flat, or scattered the neighbourhood is); write everything to a LAZ file.	GPUPDAL (GPU), PDAL 2.10.0	
Neighbourhood analysis	**Colour the points from an aerial image**	Give every point a colour by looking it up in an aerial image (in a different map projection), then write a LAZ file.	GPUPDAL (GPU), PDAL 2.10.0	

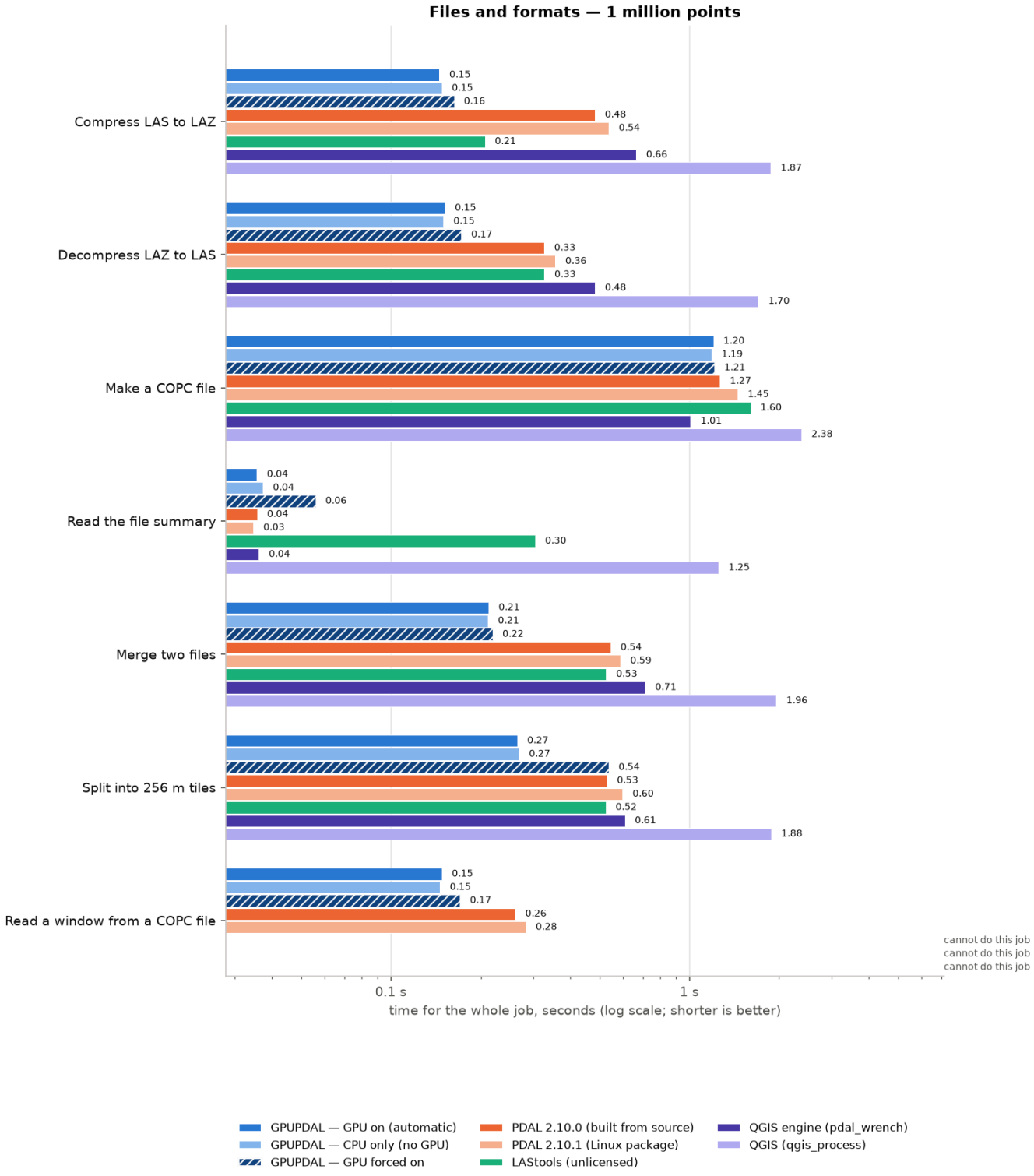
2.5 How we timed things

- Every job was run as a complete program from the command line, and we measured **wall-clock time** from start to finish, including program start-up, reading the input, doing the work, and writing the output. This is what a user experiences.
- Each job was first run once as a warm-up (not counted), then **3 times** for the 1- and 4-million-point files, **2 times** for 16 million, and **once** for 47 million (those runs take minutes). We report the **median** of the timed runs. The input files were already in the computer's memory cache (a "warm" run), which is the common case when you work on a file repeatedly.
- For jobs that need two commands with a tool that has no pipeline (for example, "find the ground, then heights" with LAStools or QGIS), the two commands were run one after the other and the total time was measured.
- We also recorded the **peak memory** (RSS) of the whole job and checked every output: file size, number of points, and a checksum. For GPUPDAL versus PDAL the checksums show whether the outputs are byte-for-byte identical.
- Nothing else was running on the workstation during the timed runs. On the rented machines the runs happened inside a container with a CPU quota (see 2.2).
- All numbers are from one benchmark run per computer. Repeats varied by a few per cent for the short jobs; the raw times of every repeat are in the appendix data file.

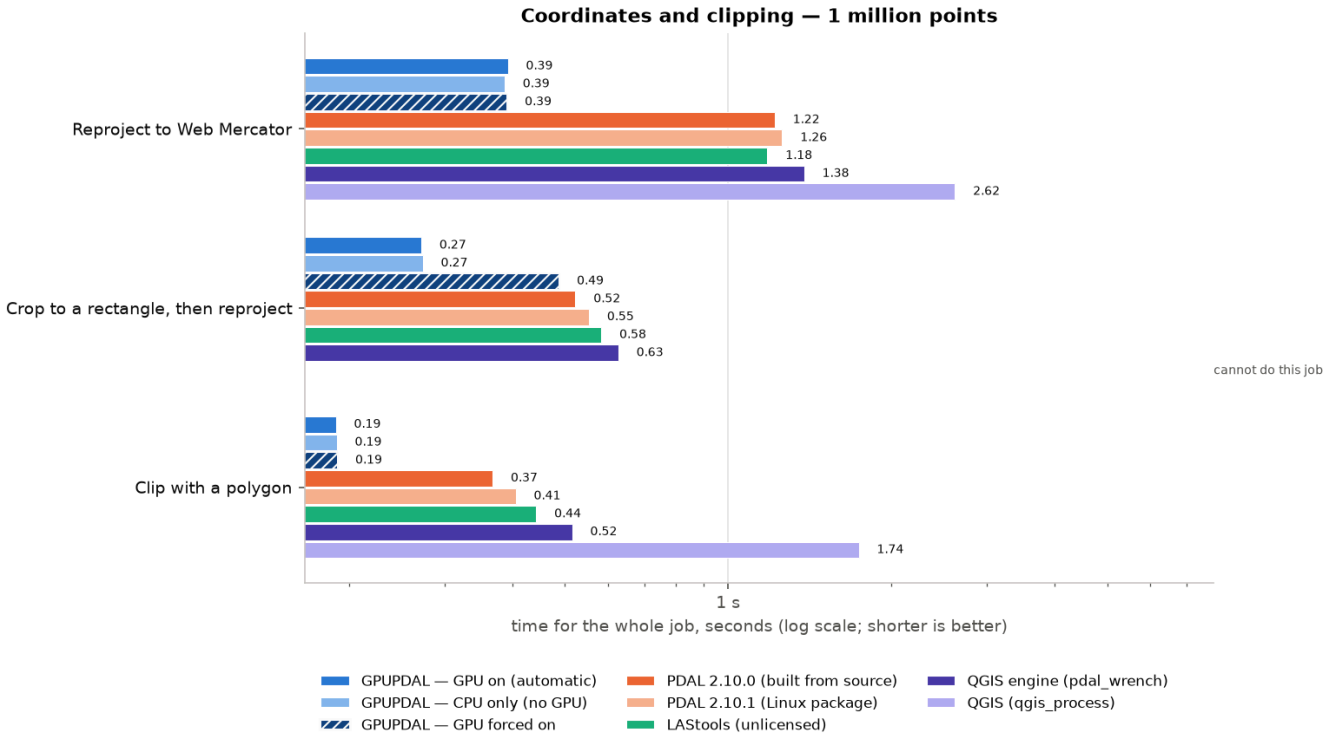
3. Results on the workstation

3.1 One million points: every job, every tool

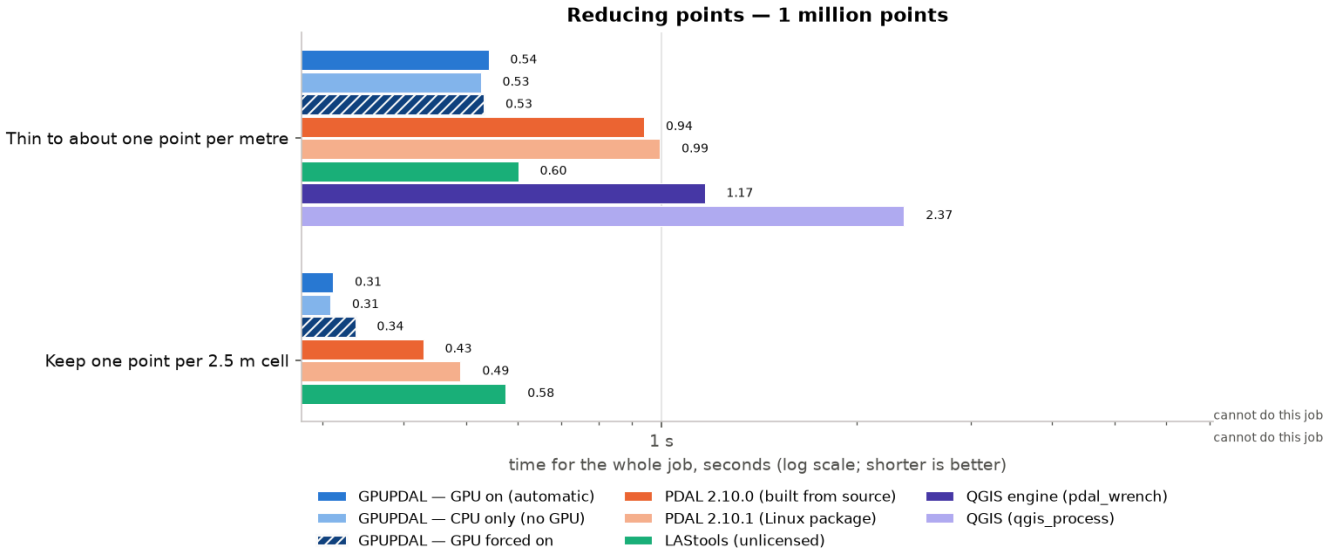
The charts below show the time for each job on a logarithmic scale (each grid line is 10× the previous one), grouped by job family. Shorter bars are better. The number at the end of each bar is the median time in seconds. "Cannot do this job" means the tool has no feature for it.



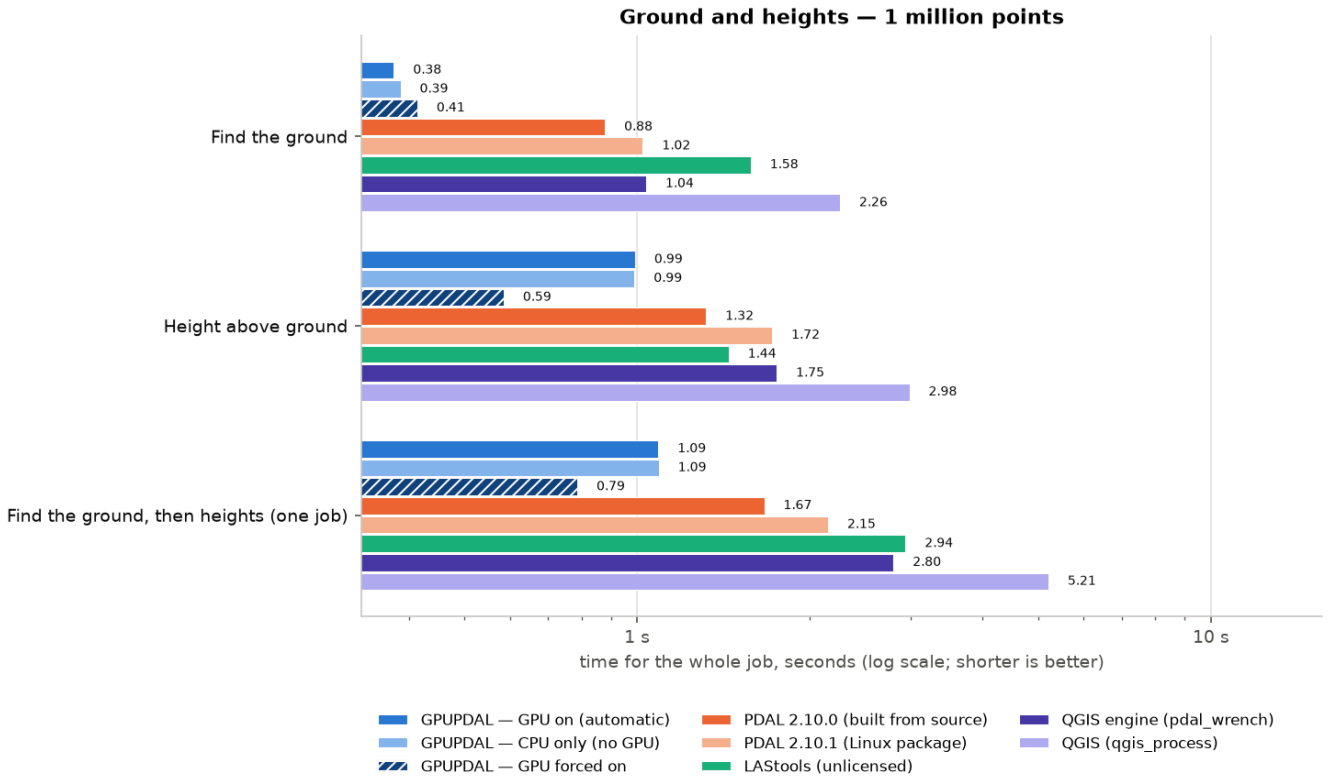
Files and formats: median time in seconds, 1 million points, workstation.



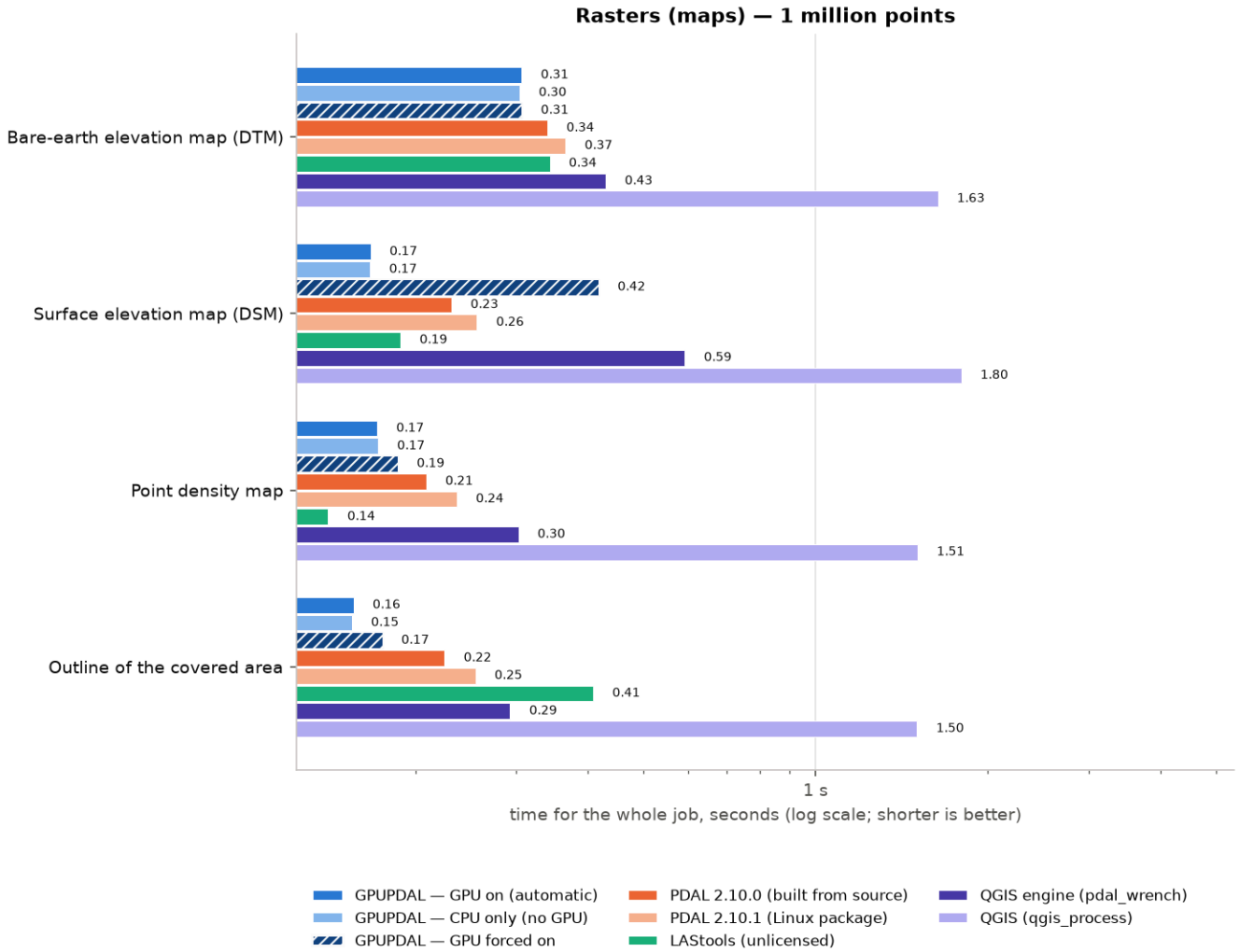
Coordinates and clipping: median time in seconds, 1 million points, workstation.



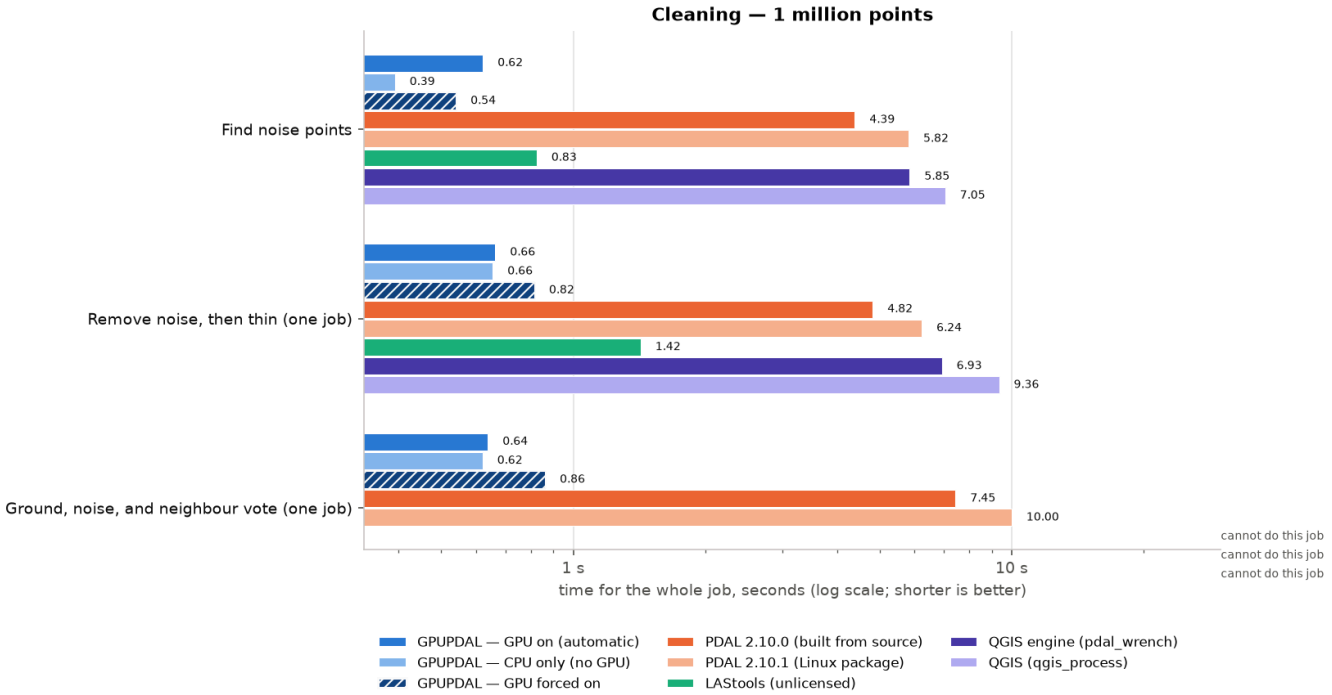
Reducing points: median time in seconds, 1 million points, workstation.



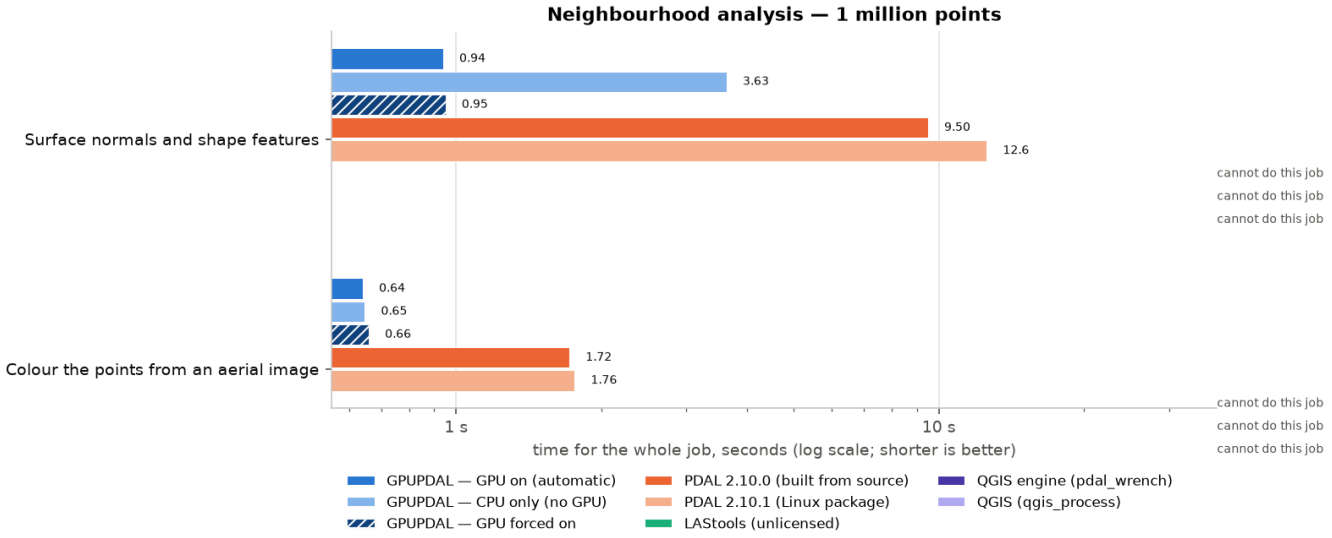
Ground and heights: median time in seconds, 1 million points, workstation.



Rasters (maps): median time in seconds, 1 million points, workstation.



Cleaning: median time in seconds, 1 million points, workstation.



Neighbourhood analysis: median time in seconds, 1 million points, workstation.

3.2 The same numbers as a table (seconds; the fastest in each row is in bold blue)

Job	GPUPDAL (GPU)	GPUPDAL (CPU only)	GPUPDAL (GPU forced)	PDAL 2.10.0	PDAL 2.10.1 pkg	LAStools	QGIS engine	QGIS
Compress LAS to LAZ	**0.15**	0.15	0.16	0.48	0.54	0.21	0.66	1.87
Decompress LAZ to LAS	0.15	**0.15**	0.17	0.33	0.36	0.33	0.48	1.70
Make a COPC file	1.20	1.19	1.21	1.27	1.45	1.60	**1.01**	2.38
Read the file summary	0.04	0.04	0.06	0.04	**0.03**	0.30	0.04	1.25
Reproject to Web Mercator	0.39	**0.39**	0.39	1.22	1.26	1.18	1.38	2.62
Crop to a rectangle, then reproject	**0.27**	0.27	0.49	0.52	0.55	0.58	0.63	—
Clip with a polygon	**0.19**	0.19	0.19	0.37	0.41	0.44	0.52	1.74
Merge two files	0.21	**0.21**	0.22	0.54	0.59	0.53	0.71	1.96

Job	GPUPDAL (GPU)	GPUPDAL (CPU only)	GPUPDAL (GPU forced)	PDAL 2.10.0	PDAL 2.10.1 pkg	LAStools	QGIS engine	QGIS
Split into 256 m tiles	**0.27**	0.27	0.54	0.53	0.60	0.52	0.61	1.88
Thin to about one point per metre	0.54	**0.53**	0.53	0.94	0.99	0.60	1.17	2.37
Keep one point per 2.5 m cell	0.31	**0.31**	0.34	0.43	0.49	0.58	—	—
Find the ground	**0.38**	0.39	0.41	0.88	1.02	1.58	1.04	2.26
Height above ground	0.99	0.99	**0.59**	1.32	1.72	1.44	1.75	2.98
Find the ground, then heights (one job)	1.09	1.09	**0.79**	1.67	2.15	2.94	2.80	5.21
Bare-earth elevation map (DTM)	0.31	**0.30**	0.31	0.34	0.37	0.34	0.43	1.63
Surface elevation map (DSM)	0.17	**0.17**	0.42	0.23	0.26	0.19	0.59	1.80
Point density map	0.17	0.17	0.19	0.21	0.24	**0.14**	0.30	1.51
Outline of the covered area	0.16	**0.15**	0.17	0.22	0.25	0.41	0.29	1.50
Find noise points	0.62	**0.39**	0.54	4.39	5.82	0.83	5.85	7.05
Remove noise, then thin (one job)	0.66	**0.66**	0.82	4.82	6.24	1.42	6.93	9.36
Ground, noise, and neighbour vote (one job)	0.64	**0.62**	0.86	7.45	10.00	—	—	—
Surface normals and shape features	**0.94**	3.63	0.95	9.50	12.6	—	—	—
Colour the points from an aerial image	**0.64**	0.65	0.66	1.72	1.76	—	—	—
Read a window from a COPC file	0.15	**0.15**	0.17	0.26	0.28	—	—	—

Seconds, median of the timed runs; the fastest tool in each row is in bold blue; "—" = the tool cannot do this job; * = LAStools ran unlicensed and says its output is deliberately distorted.

3.3 How many times faster is GPUPDAL?

The next chart divides each other tool's time by GPUPDAL's time (GPU on, automatic). Bars to the right of the 1× line mean the other tool was slower; bars to the left mean it was faster.

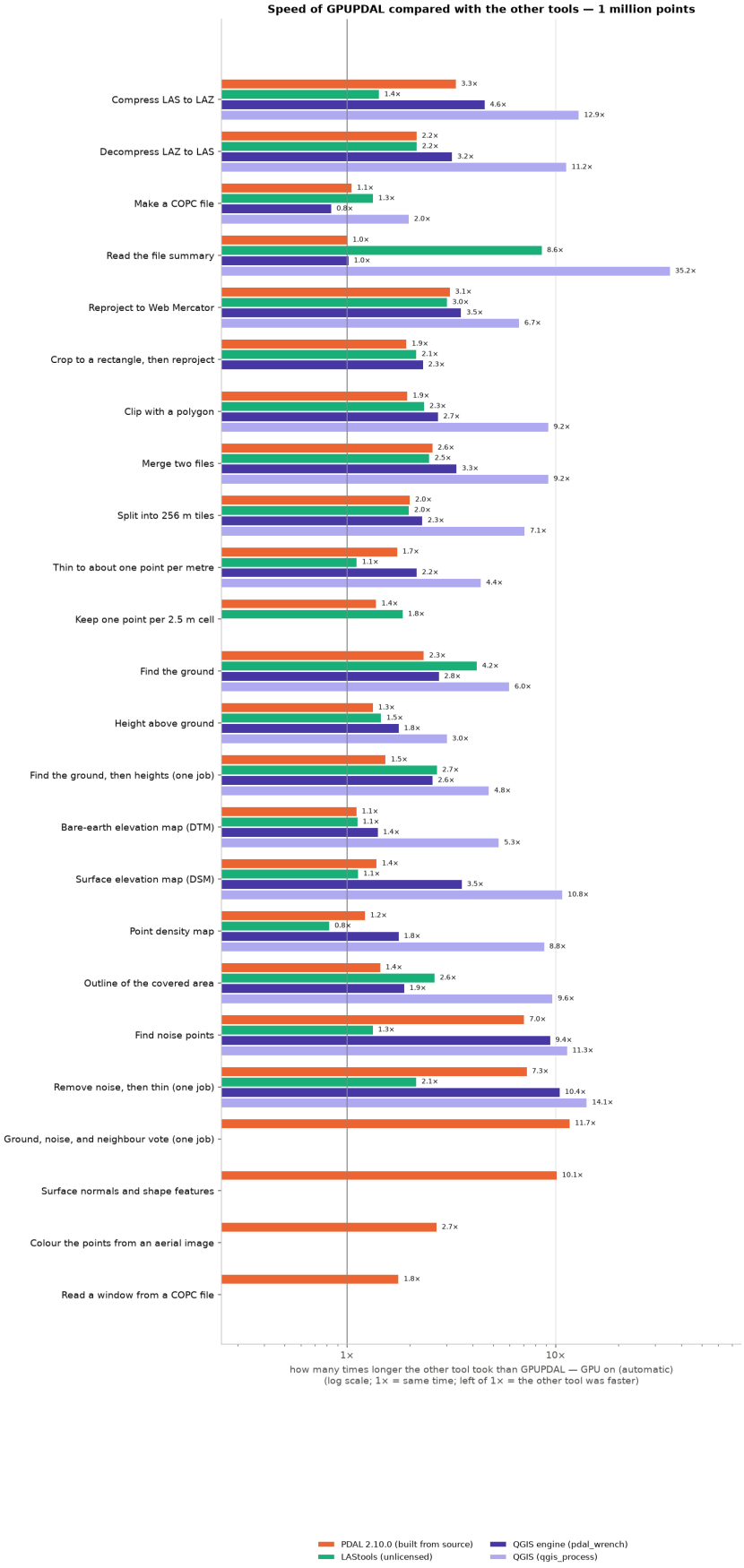


Figure. Other tool's time ÷ GPUPDAL's time, 1 million points, workstation.

3.4 Bigger files: 4, 16 and 47 million points

The full charts for the larger files are in the appendix; here are the summary tables and the scaling chart. As files grow, the jobs that look at neighbours (noise, features, classification clean-up) grow fastest for PDAL and QGIS.

4 million points (seconds)

Job	GPUPDAL (GPU)	GPUPDAL (CPU only)	GPUPDAL (GPU forced)	PDAL 2.10.0	PDAL 2.10.1 pkg	LAStools	QGIS engine	QGIS
Compress LAS to LAZ	0.41	**0.40**	0.43	1.82	2.01	0.80	2.48	3.67
Decompress LAZ to LAS	0.42	**0.41**	0.43	1.10	1.23	1.29	1.70	2.91
Make a COPC file	5.42	5.43	5.44	5.70	6.49	7.09	**3.25**	4.57
Read the file summary	0.03	0.04	0.05	0.03	**0.03**	1.19	0.04	1.26
Reproject to Web Mercator	1.20	1.19	**1.18**	4.53	4.77	4.59	5.24	6.43
Crop to a rectangle, then reproject	0.69	**0.69**	0.83	1.82	1.95	2.24	2.25	—
Clip with a polygon	**0.47**	0.48	0.47	1.24	1.39	1.72*	1.79	3.02
Merge two files	0.52	0.52	**0.51**	1.89	2.09	2.07	2.54	3.83
Split into 256 m tiles	**0.78**	0.80	1.09	1.87	2.17	2.06	2.13	3.40
Thin to about one point per metre	**2.22**	2.23	2.25	3.63	3.82	2.39	4.53	5.73
Keep one point per 2.5 m cell	1.09	**1.08**	1.11	1.53	1.76	2.29	—	—
Find the ground	1.22	**1.22**	1.34	3.19	3.77	6.58*	3.82	5.03
Height above ground	3.70	3.73	**1.36**	5.07	6.68	5.71*	6.79	7.93
Find the ground, then heights (one job)	4.16	4.13	**2.05**	6.43	8.30	11.8*	10.5	12.8
Bare-earth elevation map (DTM)	1.08	**1.07**	1.09	1.30	1.38	1.31*	1.64	2.81
Surface elevation map (DSM)	0.48	**0.48**	0.69	0.71	0.82	0.71	2.25	3.41
Point density map	**0.50**	0.51	0.52	0.68	0.76	0.52	1.03	2.18
Outline of the covered area	0.41	**0.41**	0.44	0.71	0.79	1.34	0.91	2.06
Find noise points	1.41	**1.17**	1.38	17.5	23.6	3.23	23.4	24.6
Remove noise, then thin (one job)	2.62	**2.58**	2.84	19.5	25.3	5.59	27.8	30.0
Ground, noise, and neighbour vote (one job)	2.13	**2.12**	2.32	30.0	40.6	—	—	—
Surface normals and shape features	14.4	14.1	**2.61**	38.0	50.3	—	—	—

Seconds, median of the timed runs; the fastest tool in each row is in bold blue; "—" = the tool cannot do this job; * = LAStools ran unlicensed and says its output is deliberately distorted.

16 million points (seconds)

Job	GPUPDAL (GPU)	GPUPDAL (CPU only)	GPUPDAL (GPU forced)	PDAL 2.10.0	PDAL 2.10.1 pkg	LAStools	QGIS engine	QGIS
Compress LAS to LAZ	1.42	**1.42**	1.44	7.14	7.87	3.13	9.65	10.8
Decompress LAZ to LAS	1.41	**1.40**	1.45	4.13	4.66	5.10	6.54	7.68
Make a COPC file	21.3	21.3	21.3	22.2	25.4	28.4	**6.72**	8.17
Read the file summary	0.03	0.03	0.05	0.03	**0.03**	4.67	0.03	1.18

Job	GPUPDAL (GPU)	GPUPDAL (CPU only)	GPUPDAL (GPU forced)	PDAL 2.10.0	PDAL 2.10.1 pkg	LAStools	QGIS engine	QGIS
Reproject to Web Mercator	4.29	**4.28**	4.29	18.8	18.6	18.1	20.3	21.6
Crop to a rectangle, then reproject	2.24	2.25	**2.10**	6.33	6.65	8.12	7.77	—
Clip with a polygon	1.50	1.50	**1.49**	4.52	5.14	6.54*	6.57	7.73
Merge two files	1.67	1.67	**1.66**	7.24	8.03	8.18	9.78	11.2
Split into 256 m tiles	**2.71**	2.74	3.16	6.97	8.20	8.17	3.37	4.64
Thin to about one point per metre	**8.70**	9.21	8.97	15.0	15.7	9.55*	18.4	19.8
Keep one point per 2.5 m cell	4.17	4.14	**4.11**	5.87	6.71	9.30*	—	—
Find the ground	**4.34**	4.34	4.77	12.0	14.5	30.7*	14.3	15.5
Height above ground	14.2	14.7	**4.43**	20.1	26.4	23.1*	27.0	29.2
Find the ground, then heights (one job)	16.2	16.3	**6.94**	25.0	32.6	51.8*	41.1	43.4
Bare-earth elevation map (DTM)	5.54	5.45	5.51	6.28	6.89	**5.31***	7.76	8.87
Surface elevation map (DSM)	**1.73**	1.73	1.90	2.59	3.04	2.75*	10.4	11.5
Point density map	2.17	2.20	2.24	2.74	3.22	**2.02***	4.14	5.30
Outline of the covered area	**1.45**	1.47	1.47	2.56	2.95	5.08*	3.39	4.56
Find noise points	4.54	**4.32**	4.49	69.5	93.2	13.0*	94.3	94.5
Remove noise, then thin (one job)	**10.6**	10.6	10.9	77.8	101	21.1*	111	114
Ground, noise, and neighbour vote (one job)	**7.88**	7.90	7.93	118	159	—	—	—
Surface normals and shape features	56.3	56.1	**9.19**	149	200	—	—	—

Seconds, median of the timed runs; the fastest tool in each row is in bold blue; "—" = the tool cannot do this job; * = LAStools ran unlicensed and says its output is deliberately distorted.

47 million points (seconds)

Job	GPUPDAL (GPU)	GPUPDAL (CPU only)	GPUPDAL (GPU forced)	PDAL 2.10.0	PDAL 2.10.1 pkg	LAStools	QGIS engine	QGIS
Compress LAS to LAZ	4.12	**4.11**	4.19	19.7	22.2	9.24	31.8	33.1
Decompress LAZ to LAS	**4.19**	4.35	4.37	12.2	14.0	15.0	21.3	22.5
Make a COPC file	51.6	51.4	51.5	54.2	62.5	101	**14.3**	15.9
Read the file summary	0.04	0.03	0.05	**0.03**	0.03	13.7	0.03	1.18
Reproject to Web Mercator	12.5	**12.4**	12.5	51.6	53.3	54.1	63.5	64.6
Crop to a rectangle, then reproject	5.64	5.68	**4.88**	15.8	17.3	22.1	21.4	—
Clip with a polygon	4.52	**4.48**	4.51	13.2	15.0	20.1*	21.5	22.6
Merge two files	4.82	**4.79**	4.80	20.0	22.6	24.0	32.2	33.7
Split into 256 m tiles	**7.19**	7.28	8.16	20.0	23.4	24.1	8.09	9.34
Thin to about one point per metre	9.57	9.60	**9.54**	18.8	20.3	27.3*	26.8	26.1
Keep one point per 2.5 m cell	**9.61**	9.62	9.67	12.9	15.3	27.2*	—	—
Find the ground	**9.47**	9.58	11.3	26.2	31.4	90.7*	34.4	35.6
Height above ground	54.0	53.9	**41.4**	72.3	96.7	68.8*	101	102

Job	GPUPDAL (GPU)	GPUPDAL (CPU only)	GPUPDAL (GPU forced)	PDAL 2.10.0	PDAL 2.10.1 pkg	LAStools	QGIS engine	QGIS
Find the ground, then heights (one job)	57.0	57.2	**45.9**	79.5	106	171*	136	145
Bare-earth elevation map (DTM)	**12.0**	12.1	12.1	15.0	16.2	17.1*	18.6	19.9
Surface elevation map (DSM)	**5.24**	5.27	5.37	7.96	9.59	7.01*	31.4	32.5
Point density map	**4.46**	4.46	4.46	6.81	8.02	4.91*	10.4	11.5
Outline of the covered area	4.71	**4.71**	4.77	8.21	9.36	14.6*	10.7	11.8
Find noise points	13.1	12.5	**10.5**	181	238	38.1*	239	240
Remove noise, then thin (one job)	17.1	17.1	**14.9**	181	236	63.2*	263	266
Ground, noise, and neighbour vote (one job)	20.9	20.3	**19.9**	281	372	—	—	—
Surface normals and shape features	166	165	**28.1**	397	515	—	—	—

Seconds, median of the timed runs; the fastest tool in each row is in bold blue; "—" = the tool cannot do this job; * = LAStools ran unlicensed and says its output is deliberately distorted.

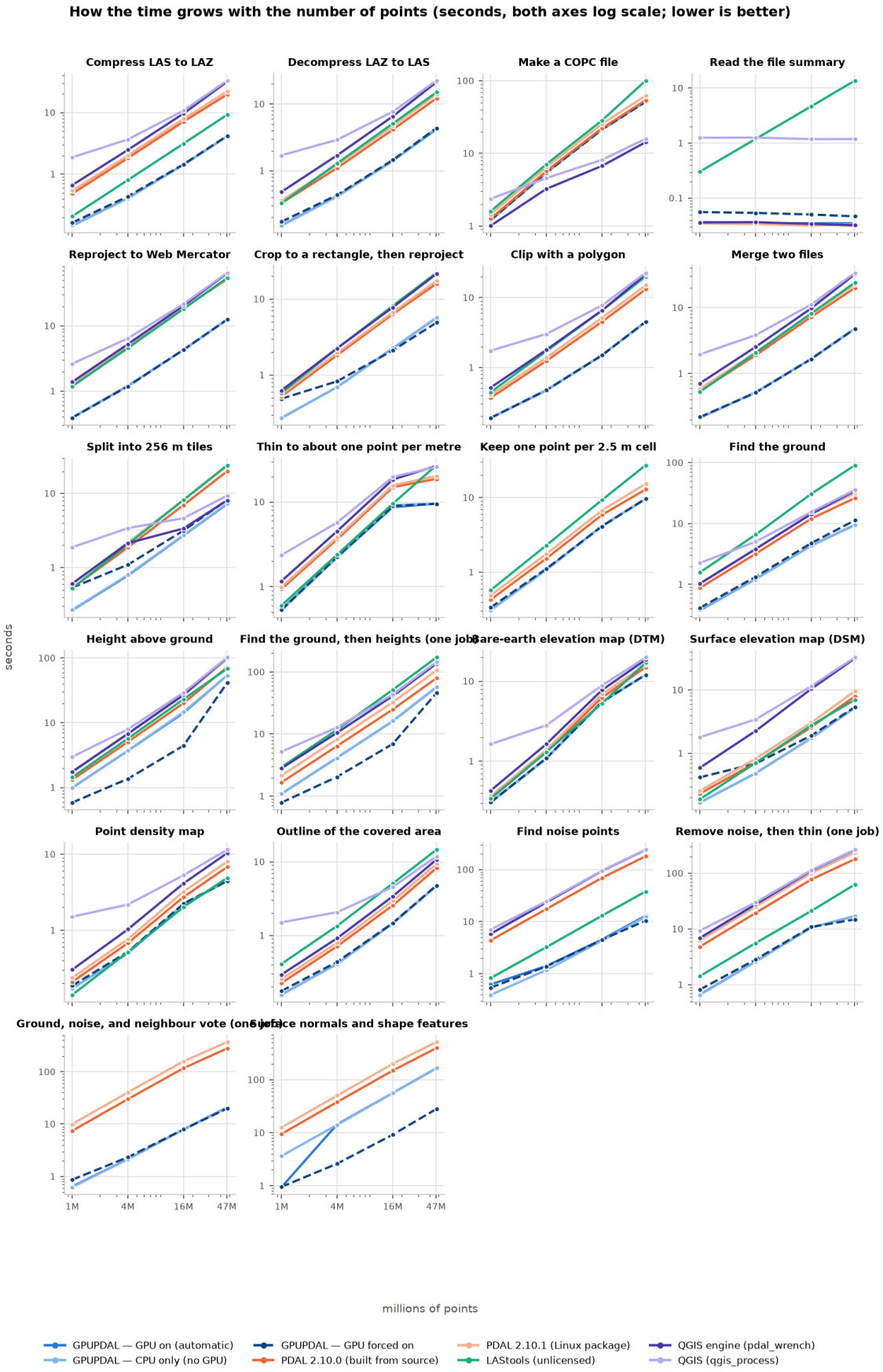


Figure. Time versus number of points for every job, all tools (both axes logarithmic). A straight line means the time grows in proportion to the number of points; a steeper line means it grows faster than that.

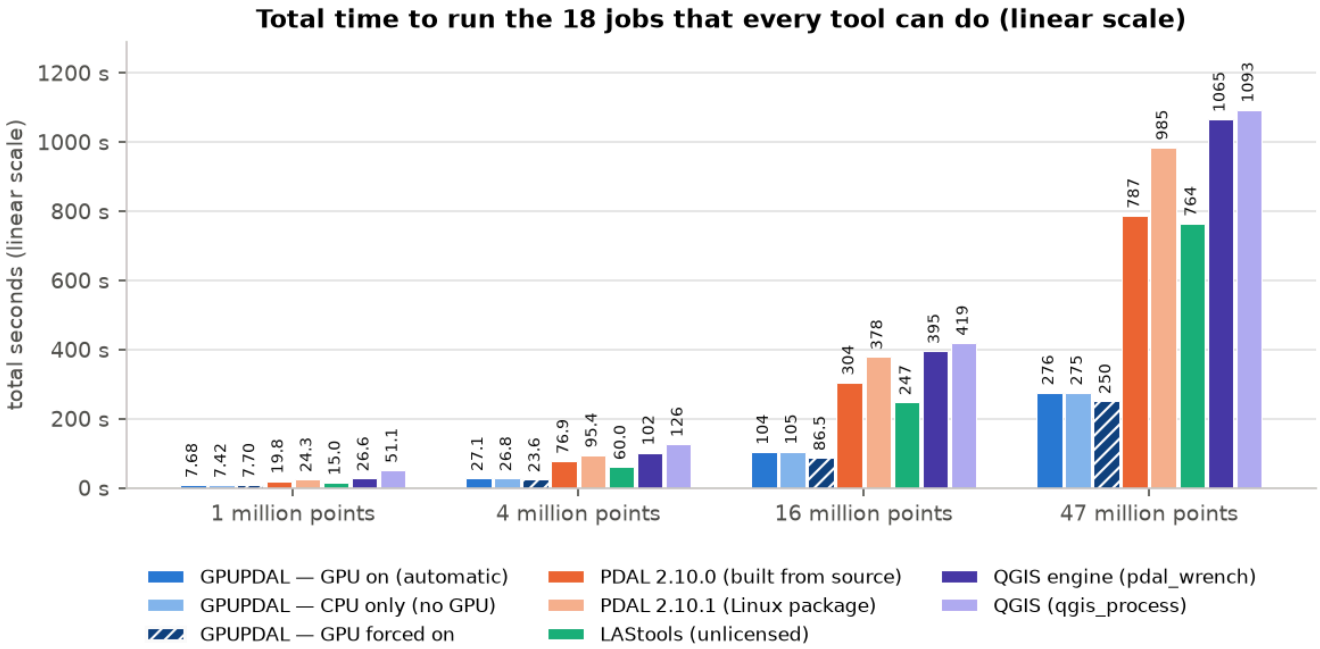


Figure. Aggregate wall-clock time to run, one after another, the 18 jobs that every tool completed at every size. The y-axis is linear; each bar is the sum of those jobs' median times. The GPUPDAL series retain the historical raw-result identifier pdg; the benchmark values are unchanged.

3.5 Memory

Peak memory of the whole job. GPUPDAL uses about the same memory as PDAL for most jobs, sometimes about a gigabyte more (it keeps extra copies of the data for its threads), and clearly more when the GPU is forced on for the features job (the GPU path stages the whole cloud in pinned memory). LAStools uses far less memory than everyone else because it streams points through instead of loading them all. QGIS needs about 380 MB just to start.

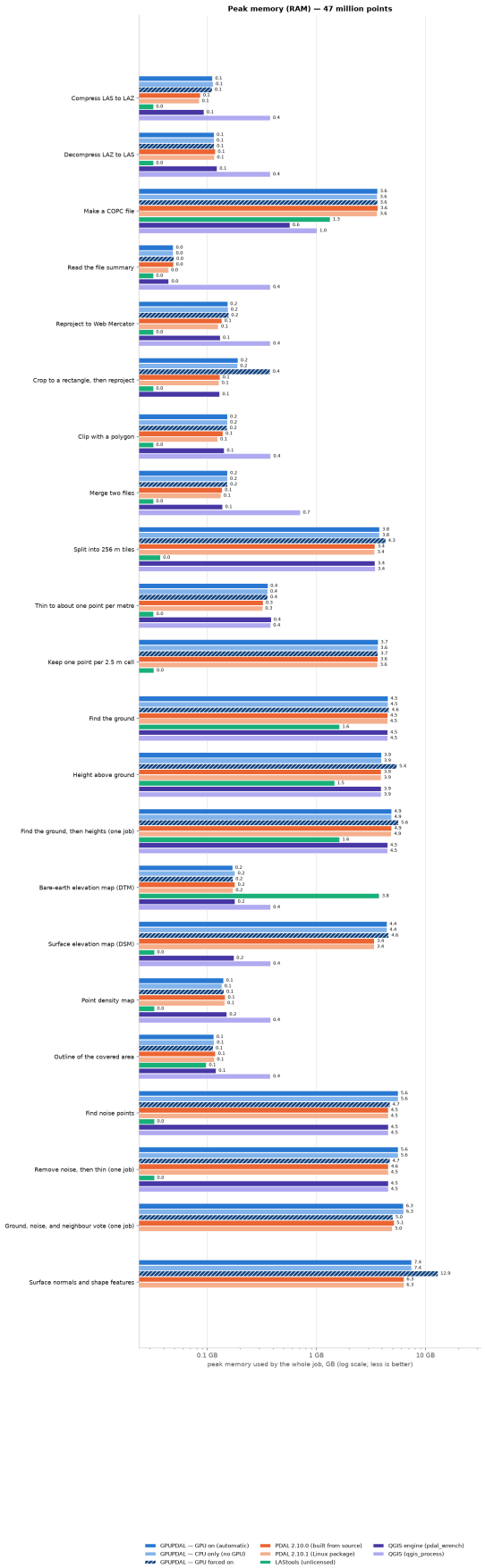


Figure. Peak memory (RSS) per job, 47 million points, workstation.

4. With and without the graphics card

GPUPDAL can use an NVIDIA graphics card (GPU), but it does not always do so. In its normal, automatic mode it uses a built-in table of measurements to decide, per job and per file size, whether the GPU would help. To see what the GPU is really worth, we ran GPUPDAL three ways on the same computer: automatic (the default), with the GPU hidden (as if the machine had none), and with the GPU forced on for every job that has a GPU path.

The chart below shows the time of the "CPU only" and "GPU forced" runs relative to the automatic run. A bar at 1× means no difference. Bars to the right mean that mode was slower than automatic; bars to the left mean it was faster (in other words, the automatic choice was not the best one for that job).

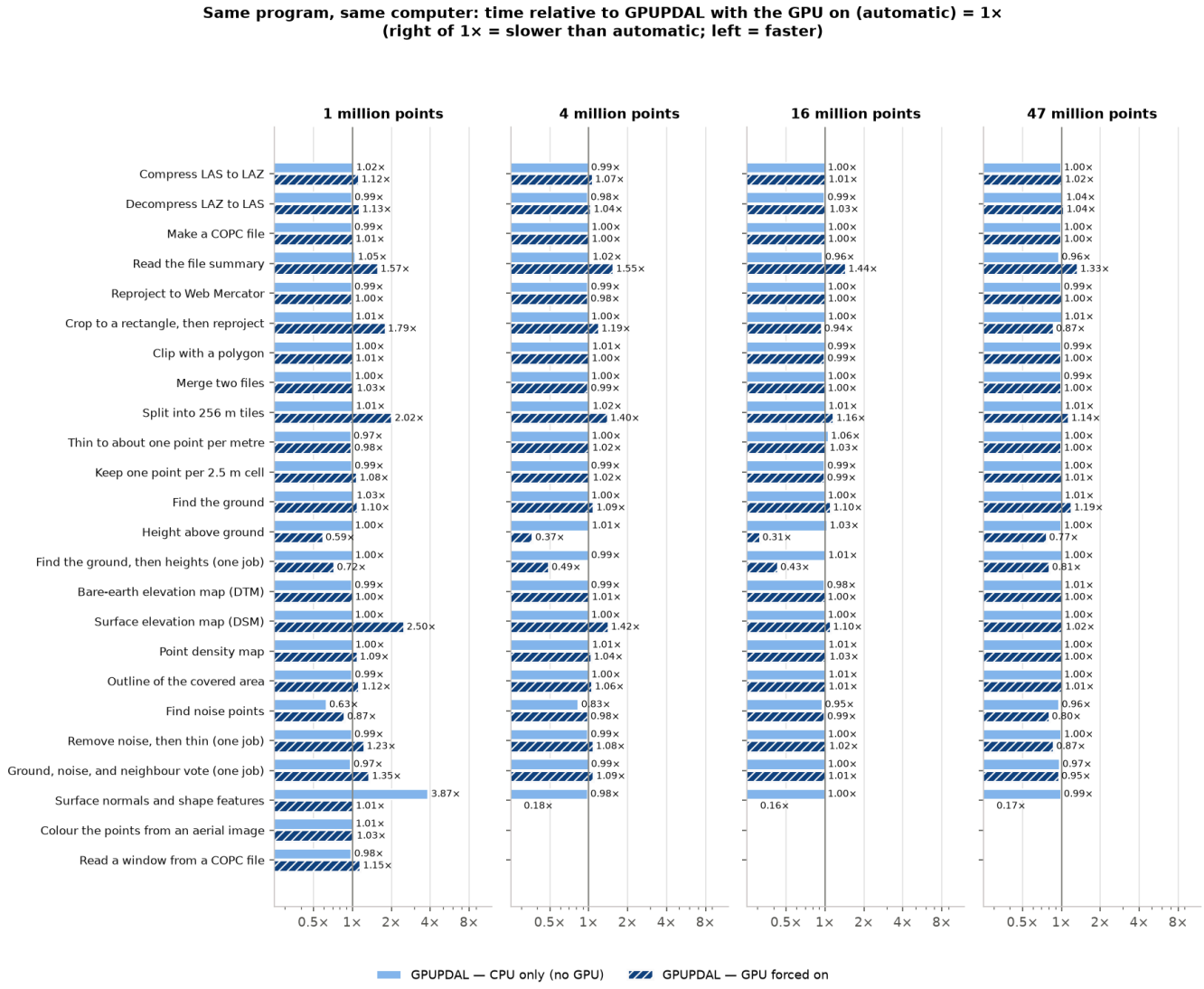


Figure. GPUPDAL CPU-only and GPU-forced time divided by GPUPDAL automatic time, per job and file size, workstation (RTX 4090).

How to read this. Most jobs sit at 1×: GPUPDAL does not use the GPU for them at all, so hiding it changes nothing. The jobs where the GPU makes a real difference are the neighbourhood jobs, above all "Surface normals and shape features". Where the "GPU forced" bar is far to the right (for example the DSM and tiling jobs), forcing the GPU on is a bad idea — moving the data to the card costs more than it saves — and the automatic mode was right not to use it. Where a bar is to the left of 1×, the automatic mode left some speed on the table for that job on this machine.

Two things we checked more closely.

1. Why did the automatic mode stop using the GPU for the features job above 1 million points? We asked the program for its own decision record (`pdg-engine resident … --stats`). At 1 million points it reports "device faster" and runs on the GPU; at 4 million it reports `mixed_calibration_models` and runs on the CPU. The reason is in the source: for this job with the "write all extra values" output, the automatic GPU route is only allowed for a short list of exact input layouts that were measured beforehand — a 1,000,000-point, point-format-7 compressed file; a point-format-6 compressed file; and a point-format-8 compressed file with 44-byte records (the original AHN4 layout). Our 4- and 16-million-point files are point-format-7 with other counts, and our 47-million-point file was re-written by PDAL with 38-byte records, so none of them qualifies, and GPUPDAL deliberately stays on the CPU rather than trust a speed model it did not measure for that layout. This is a cautious design choice, but for a user it means the GPU speed for this job (5–6× at these sizes) is only reached automatically for particular files, or by forcing the GPU on.

2. Why is "Find noise points" a little slower with the GPU on? Re-timing it eight times gave 0.55–0.61 s with the GPU visible and 0.35–0.37 s with it hidden at 1 million points; the difference (about 0.2 s, and 0.2–0.5 s at every size) is a fixed start-up cost of the GPU path that this job does not win back. It is a small absolute loss.

5. Results on the other computers

The same harness, the same inputs and the same commands were run on two rented cloud machines. Only the tools that could be built there were run: GPUPDAL (three modes), PDAL 2.10.0 built from source, LAStools, and the QGIS engine pdal_wrench built from source. Times are not directly comparable across machines when the CPUs differ (and the containers have CPU quotas), but the ratios between tools on the same machine are.

Cloud server (EPYC 7532 + A100 40 GB)

A rented data-centre machine: AMD EPYC 7532 (a container allowed to use about 31 of its 64 threads), 503 GB of memory, NVIDIA A100 SXM4 data-centre GPU (40 GB), Ubuntu 24.04.

1 million points (seconds)

Job	GPUPDAL (GPU)	GPUPDAL (CPU only)	GPUPDAL (GPU forced)	PDAL 2.10.0	LAStools	QGIS engine
Compress LAS to LAZ	**0.55**	0.56	0.59	1.62	0.57	1.90
Decompress LAZ to LAS	**0.52**	0.56	0.58	1.21	0.88	1.45
Make a COPC file	3.03	2.99	**2.97**	3.15	3.58	—
Read the file summary	0.08	0.08	0.14	**0.08**	0.81	0.09
Reproject to Web Mercator	1.21	**1.20**	1.20	3.87	3.07	4.13
Crop to a rectangle, then reproject	**1.04**	1.06	1.32	1.72	1.48	1.90
Clip with a polygon	0.74	**0.74**	0.74	1.24	1.12	1.51
Merge two files	0.79	0.77	**0.74**	1.78	1.32	2.04
Split into 256 m tiles	0.90	**0.88**	1.49	1.56	1.33	1.82
Thin to about one point per metre	1.67	1.62	1.63	2.91	**1.56**	3.40
Keep one point per 2.5 m cell	**0.75**	0.77	0.82	1.00	1.50	—
Find the ground	1.29	1.29	**1.26**	2.42	4.21	2.48
Height above ground	3.12	3.12	**1.54**	3.86	3.57	3.98
Find the ground, then heights (one job)	3.63	3.67	**2.27**	4.79	7.56	6.44
Bare-earth elevation map (DTM)	0.75	**0.71**	0.72	0.91	0.90	1.05
Surface elevation map (DSM)	**0.47**	0.48	1.08	0.66	0.50	1.29
Point density map	0.66	0.60	0.68	0.76	**0.40**	0.87
Outline of the covered area	0.61	**0.55**	0.62	0.80	1.11	0.87
Find noise points	1.53	**0.95**	1.42	12.4	2.11	12.4
Remove noise, then thin (one job)	1.97	**1.95**	2.33	13.7	3.70	15.7
Ground, noise, and neighbour vote (one job)	**1.68**	1.77	2.34	21.0	—	—
Surface normals and shape features	**3.07**	10.1	3.15	26.7	—	—
Colour the points from an aerial image	**1.97**	1.99	2.05	5.47	—	—
Read a window from a COPC file	**0.51**	0.51	0.57	0.72	—	—

Seconds, median of the timed runs; the fastest tool in each row is in bold blue; "—" = the tool cannot do this job; * = LAStools ran unlicensed and says its output is deliberately distorted.

47 million points (seconds)

Job	GPUPDAL (GPU)	GPUPDAL (CPU only)	GPUPDAL (GPU forced)	PDAL 2.10.0	LAStools	QGIS engine
Compress LAS to LAZ	20.8	20.9	**19.8**	72.6	26.6	90.8
Decompress LAZ to LAS	16.6	16.3	**16.3**	51.1	41.0	64.2
Make a COPC file	**134**	141	141	148	229	—
Read the file summary	0.09	0.09	0.14	**0.09**	38.2	0.09
Reproject to Web Mercator	42.7	43.4	**41.8**	172	144	190
Crop to a rectangle, then reproject	22.9	23.4	**15.1**	59.3	59.5	65.9
Clip with a polygon	20.6	19.8	**19.4**	51.1	52.9*	62.7
Merge two files	20.2	20.3	**19.6**	73.4	63.1	92.8
Split into 256 m tiles	25.5	26.3	27.8	66.1	62.9	**16.6**
Thin to about one point per metre	29.3	**28.8**	29.0	68.0	74.7*	78.1
Keep one point per 2.5 m cell	**22.5**	24.5	22.6	36.3	74.6*	—
Find the ground	**32.3**	32.4	36.4	84.4	263*	94.4
Height above ground	173	174	**128**	222	180*	234
Find the ground, then heights (one job)	187	186	**145**	245	458*	326
Bare-earth elevation map (DTM)	23.8	24.2	**23.2**	41.4	47.0*	45.1
Surface elevation map (DSM)	17.1	**16.9**	17.4	27.8	19.6*	64.4
Point density map	20.1	20.4	20.1	30.0	**15.0***	33.8
Outline of the covered area	19.1	18.8	**18.8**	35.9	40.7*	36.6
Find noise points	37.7	35.4	**29.8**	522	101*	528
Remove noise, then thin (one job)	48.4	46.5	**40.7**	522	169*	602
Ground, noise, and neighbour vote (one job)	57.5	55.9	**55.2**	816	—	—
Surface normals and shape features	451	449	**73.6**	1127	—	—

Seconds, median of the timed runs; the fastest tool in each row is in bold blue; "—" = the tool cannot do this job; * = LAStools ran unlicensed and says its output is deliberately distorted.

16 million points (seconds)

Job	GPUPDAL (GPU)	GPUPDAL (CPU only)	GPUPDAL (GPU forced)	PDAL 2.10.0	LAStools	QGIS engine
Compress LAS to LAZ	6.70	**6.44**	6.70	24.5	8.85	27.9
Decompress LAZ to LAS	**5.28**	5.42	5.42	16.4	13.7	19.9
Make a COPC file	53.0	**53.0**	53.2	56.1	63.3	—
Read the file summary	0.09	0.09	0.14	**0.08**	12.7	0.09
Reproject to Web Mercator	14.0	14.3	**13.9**	57.7	48.1	62.1
Crop to a rectangle, then reproject	9.79	9.72	**6.98**	21.8	21.3	24.2
Clip with a polygon	**6.62**	6.69	6.69	16.5	17.0*	19.6
Merge two files	7.50	**7.06**	7.19	24.9	20.9	28.5
Split into 256 m tiles	8.56	8.59	10.0	22.1	20.8	**8.49**
Thin to about one point per metre	25.7	26.1	26.1	47.0	**25.4***	54.3
Keep one point per 2.5 m cell	9.21	**9.14**	9.31	14.4	24.8*	—
Find the ground	**13.6**	13.6	14.9	35.2	77.7*	35.5
Height above ground	45.4	45.2	**12.5**	61.0	58.5*	62.8
Find the ground, then heights (one job)	52.0	52.4	**20.6**	75.2	131*	98.2

Job	GPUPDAL (GPU)	GPUPDAL (CPU only)	GPUPDAL (GPU forced)	PDAL 2.10.0	LAStools	QGIS engine
Bare-earth elevation map (DTM)	18.8	18.7	18.8	24.1	**14.6***	25.6
Surface elevation map (DSM)	**5.08**	5.52	5.76	8.43	7.63*	30.3
Point density map	11.0	10.9	11.0	14.2	**6.06***	15.8
Outline of the covered area	**5.39**	5.39	5.50	10.3	14.1*	10.5
Find noise points	13.2	12.9	**12.8**	201	34.1*	202
Remove noise, then thin (one job)	31.0	31.2	**30.9**	227	55.9*	254
Ground, noise, and neighbour vote (one job)	23.4	**22.2**	22.3	344	—	—
Surface normals and shape features	158	156	**25.4**	436	—	—

Seconds, median of the timed runs; the fastest tool in each row is in bold blue; "—" = the tool cannot do this job; * = LAStools ran unlicensed and says its output is deliberately distorted.

4 million points (seconds)

Job	GPUPDAL (GPU)	GPUPDAL (CPU only)	GPUPDAL (GPU forced)	PDAL 2.10.0	LAStools	QGIS engine
Compress LAS to LAZ	1.72	**1.70**	1.78	6.17	2.22	7.03
Decompress LAZ to LAS	1.50	**1.49**	1.53	4.26	3.45	5.22
Make a COPC file	13.2	**13.1**	13.2	13.9	15.1	—
Read the file summary	0.09	0.09	0.13	**0.08**	3.19	0.09
Reproject to Web Mercator	**3.68**	3.69	3.93	14.5	12.0	15.7
Crop to a rectangle, then reproject	3.00	2.91	**2.47**	6.21	5.78	6.96
Clip with a polygon	2.04	**2.01**	2.10	4.41	4.44*	5.18
Merge two files	2.00	2.11	**1.97**	6.43	5.22	7.30
Split into 256 m tiles	**2.47**	2.52	3.31	5.68	5.21	6.30
Thin to about one point per metre	6.21	6.17	6.30	11.3	**6.12**	13.1
Keep one point per 2.5 m cell	2.61	2.69	**2.59**	3.68	5.93	—
Find the ground	3.85	**3.82**	4.17	9.08	17.7*	9.25
Height above ground	11.6	11.6	**3.76**	15.2	14.6*	15.7
Find the ground, then heights (one job)	13.6	13.5	**6.09**	19.0	30.9*	24.9
Bare-earth elevation map (DTM)	2.60	**2.50**	2.52	3.65	3.63*	4.21
Surface elevation map (DSM)	1.39	**1.36**	2.10	2.20	1.93	5.11
Point density map	2.05	2.08	2.08	2.65	**1.54**	3.21
Outline of the covered area	1.69	**1.64**	1.75	2.72	3.68	2.91
Find noise points	4.10	**3.11**	3.94	50.6	8.43	50.3
Remove noise, then thin (one job)	**7.53**	7.55	8.16	56.2	14.6	63.3
Ground, noise, and neighbour vote (one job)	5.87	**5.75**	6.54	86.5	—	—
Surface normals and shape features	39.9	39.3	**7.42**	109	—	—

Seconds, median of the timed runs; the fastest tool in each row is in bold blue; "—" = the tool cannot do this job; * = LAStools ran unlicensed and says its output is deliberately distorted.

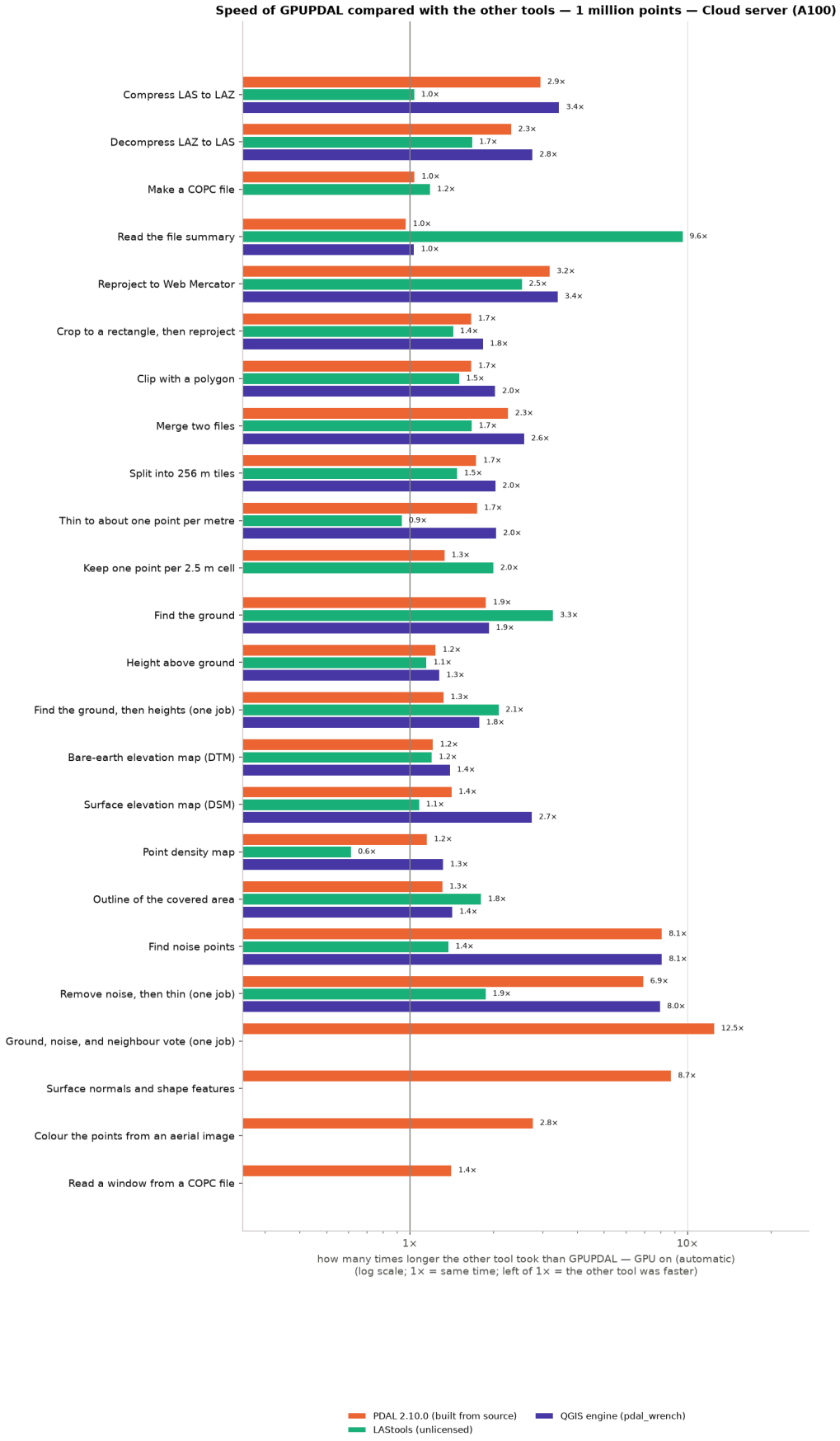


Figure. Other tool's time ÷ GPUPDAL's time, 1 million points, Cloud server (A100).

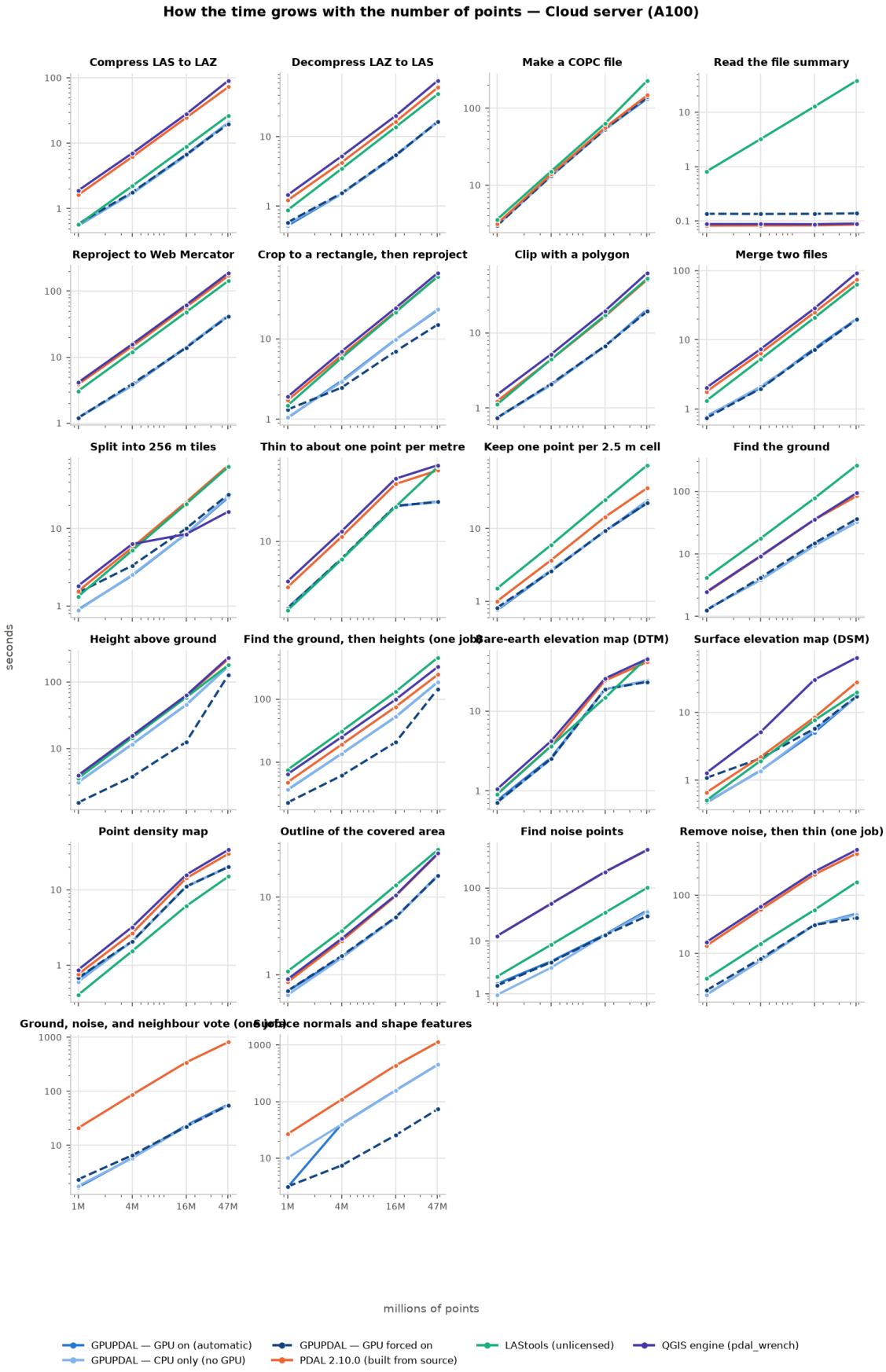


Figure. Time versus number of points, Cloud server (A100).

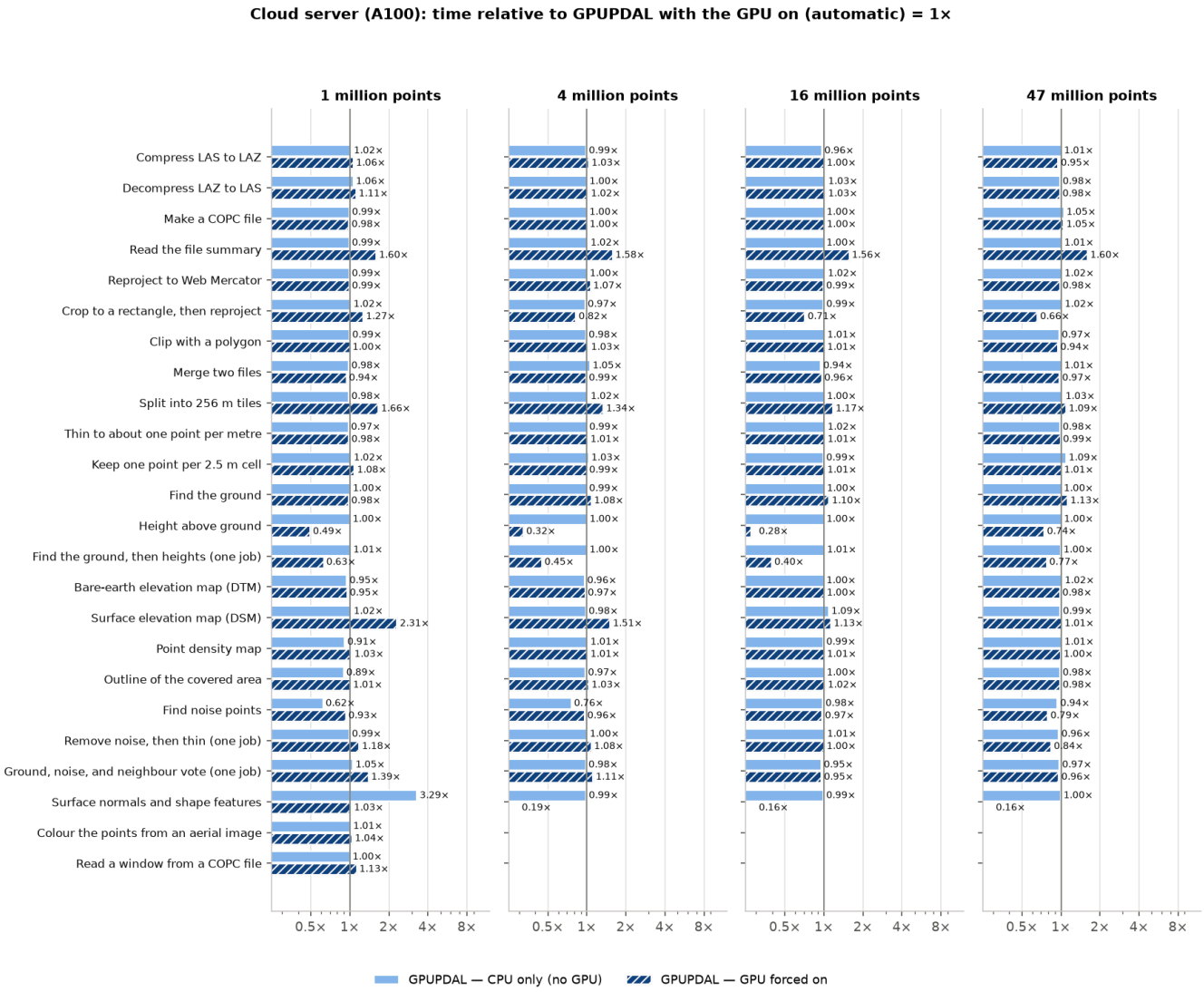


Figure. GPUPDAL CPU-only and GPU-forced time relative to automatic, Cloud server (A100).

Budget cloud PC (Xeon E5-2697A v4 + RTX 3060 12 GB)

A rented older machine: Intel Xeon E5-2697A v4 from 2016 (a container allowed about 15 threads), 125 GB of memory, NVIDIA GeForce RTX 3060 (12 GB), Ubuntu 24.04.

1 million points (seconds)

Job	GPUPDAL (GPU)	GPUPDAL (CPU only)	GPUPDAL (GPU forced)	PDAL 2.10.0	LAStools	QGIS engine
Compress LAS to LAZ	0.46	0.46	0.56	1.07	**0.44**	1.26
Decompress LAZ to LAS	**0.42**	0.45	0.47	0.82	0.67	0.92
Make a COPC file	**2.38**	2.50	2.47	2.46	2.62	—
Read the file summary	0.06	0.06	0.09	**0.06**	0.65	0.06
Reproject to Web Mercator	**1.22**	1.24	1.23	2.55	2.41	2.95
Crop to a rectangle, then reproject	1.01	0.99	**0.96**	1.19	1.17	1.20
Clip with a polygon	0.68	0.70	**0.67**	0.83	0.85	1.05
Merge two files	**0.62**	0.65	0.63	1.22	1.00	1.39
Split into 256 m tiles	**0.68**	0.69	1.10	1.09	1.06	1.27
Thin to about one point per metre	1.31	1.28	1.37	2.09	**1.26**	2.32

Job	GPUPDAL (GPU)	GPUPDAL (CPU only)	GPUPDAL (GPU forced)	PDAL 2.10.0	LAStools	QGIS engine
Keep one point per 2.5 m cell	**0.61**	0.62	0.65	0.73	1.13	—
Find the ground	0.93	**0.91**	1.03	1.66	4.42	1.70
Height above ground	2.25	2.33	**1.10**	2.68	2.67	2.89
Find the ground, then heights (one job)	2.77	2.76	**1.60**	3.31	7.10	4.43
Bare-earth elevation map (DTM)	**0.63**	0.64	0.65	0.65	0.68	0.83
Surface elevation map (DSM)	0.42	0.42	0.76	0.49	**0.39**	0.96
Point density map	0.63	0.63	0.80	0.51	**0.30**	0.67
Outline of the covered area	0.53	**0.50**	0.56	0.54	0.97	0.66
Find noise points	1.31	**1.03**	1.08	8.75	1.65	8.68
Remove noise, then thin (one job)	**1.72**	1.75	1.77	10.1	2.81	11.0
Ground, noise, and neighbour vote (one job)	**1.72**	1.76	1.90	14.8	—	—
Surface normals and shape features	**2.17**	7.76	2.19	19.2	—	—
Colour the points from an aerial image	2.33	2.33	**2.21**	3.55	—	—
Read a window from a COPC file	0.43	**0.41**	0.47	0.62	—	—

Seconds, median of the timed runs; the fastest tool in each row is in bold blue; "—" = the tool cannot do this job; * = LAStools ran unlicensed and says its output is deliberately distorted.

47 million points (seconds)

Job	GPUPDAL (GPU)	GPUPDAL (CPU only)	GPUPDAL (GPU forced)	PDAL 2.10.0	LAStools	QGIS engine
Compress LAS to LAZ	**16.9**	18.3	17.0	48.8	18.7	61.0
Decompress LAZ to LAS	**15.1**	16.1	16.1	32.2	32.1	37.7
Make a COPC file	102	102	**102**	110	162	—
Read the file summary	0.06	**0.05**	0.10	0.06	27.9	0.06
Reproject to Web Mercator	46.0	**45.2**	47.8	111	106	129
Crop to a rectangle, then reproject	28.6	29.2	**16.5**	39.8	44.8	42.4
Clip with a polygon	21.3	**20.8**	21.8	35.4	41.6*	42.5
Merge two files	19.2	**19.0**	19.9	51.8	47.3	66.4
Split into 256 m tiles	17.6	18.1	20.7	44.5	49.5	**17.5**
Thin to about one point per metre	29.1	28.5	**28.4**	47.8	57.0*	53.5
Keep one point per 2.5 m cell	17.2	**17.2**	17.4	25.4	57.5*	—
Find the ground	**25.5**	25.5	27.9	57.2	264*	60.6
Height above ground	129	130	**103**	154	140*	166
Find the ground, then heights (one job)	137	140	**115**	190	428*	226
Bare-earth elevation map (DTM)	**24.6**	24.7	25.7	29.2	35.4*	31.4
Surface elevation map (DSM)	13.5	**13.2**	14.0	16.7	15.4*	46.4
Point density map	27.6	27.0	27.2	19.4	**10.8***	22.0
Outline of the covered area	22.3	**21.0**	23.2	24.7	35.1*	24.3
Find noise points	39.9	43.0	**26.8**	372	80.4*	370
Remove noise, then thin (one job)	47.8	47.2	**34.2**	375	126*	427
Ground, noise, and neighbour vote (one job)	72.9	72.7	**55.8**	591	—	—

Job	GPUPDAL (GPU)	GPUPDAL (CPU only)	GPUPDAL (GPU forced)	PDAL 2.10.0	LAStools	QGIS engine
Surface normals and shape features	337	328	**58.0**	815	—	—

Seconds, median of the timed runs; the fastest tool in each row is in bold blue; "—" = the tool cannot do this job; * = LAStools ran unlicensed and says its output is deliberately distorted.

16 million points (seconds)

Job	GPUPDAL (GPU)	GPUPDAL (CPU only)	GPUPDAL (GPU forced)	PDAL 2.10.0	LAStools	QGIS engine
Compress LAS to LAZ	5.78	5.72	**5.64**	17.0	6.48	19.0
Decompress LAZ to LAS	5.77	5.65	**5.50**	10.7	10.3	12.3
Make a COPC file	43.4	43.4	**41.9**	44.2	46.8	—
Read the file summary	0.06	**0.06**	0.09	0.06	9.68	0.06
Reproject to Web Mercator	15.6	15.6	**15.2**	37.9	35.9	41.6
Crop to a rectangle, then reproject	11.8	11.3	**7.18**	15.2	16.2	16.7
Clip with a polygon	7.20	7.09	**7.03**	11.4	13.1*	13.4
Merge two files	**6.63**	6.79	6.74	17.6	15.9	19.0
Split into 256 m tiles	6.75	**6.70**	8.00	15.2	16.8	6.98
Thin to about one point per metre	22.6	22.5	21.9	35.1	**19.7***	40.0
Keep one point per 2.5 m cell	7.19	**7.13**	7.13	10.3	18.9*	—
Find the ground	11.5	**10.9**	11.7	23.8	75.2*	24.3
Height above ground	33.8	35.3	**10.3**	42.0	45.5*	44.8
Find the ground, then heights (one job)	38.7	38.4	**15.7**	51.9	117*	68.3
Bare-earth elevation map (DTM)	13.1	13.4	13.1	15.0	**10.3***	15.3
Surface elevation map (DSM)	**4.13**	4.46	4.60	5.25	5.49*	18.9
Point density map	11.3	11.0	11.7	8.47	**4.54***	10.0
Outline of the covered area	7.09	7.26	**6.63**	7.23	11.8*	7.17
Find noise points	14.7	14.0	**11.1**	144	26.1*	142
Remove noise, then thin (one job)	29.5	27.7	**26.0**	164	43.1*	184
Ground, noise, and neighbour vote (one job)	27.8	28.7	**21.2**	251	—	—
Surface normals and shape features	114	112	**20.4**	316	—	—

Seconds, median of the timed runs; the fastest tool in each row is in bold blue; "—" = the tool cannot do this job; * = LAStools ran unlicensed and says its output is deliberately distorted.

4 million points (seconds)

Job	GPUPDAL (GPU)	GPUPDAL (CPU only)	GPUPDAL (GPU forced)	PDAL 2.10.0	LAStools	QGIS engine
Compress LAS to LAZ	1.53	**1.46**	1.52	4.10	1.63	4.58
Decompress LAZ to LAS	1.37	**1.36**	1.48	2.79	2.57	3.22
Make a COPC file	10.5	10.5	11.2	**10.5**	11.0	—
Read the file summary	0.06	0.06	0.10	**0.06**	2.35	0.06
Reproject to Web Mercator	4.08	**4.03**	4.27	9.65	9.39	10.4
Crop to a rectangle, then reproject	3.20	3.18	**2.29**	4.23	4.64	4.75
Clip with a polygon	2.03	**1.96**	2.02	3.00	3.64*	3.58

Job	GPUPDAL (GPU)	GPUPDAL (CPU only)	GPUPDAL (GPU forced)	PDAL 2.10.0	LAStools	QGIS engine
Merge two files	1.87	**1.81**	1.88	4.39	3.97	4.77
Split into 256 m tiles	**2.05**	2.07	2.64	3.73	4.01	4.45
Thin to about one point per metre	5.31	5.16	4.98	7.94	**4.90**	9.13
Keep one point per 2.5 m cell	2.03	**1.99**	2.03	2.62	4.77	—
Find the ground	**3.11**	3.25	3.32	6.49	17.9*	6.54
Height above ground	8.75	8.71	**2.87**	11.3	11.0*	10.7
Find the ground, then heights (one job)	9.89	10.0	**4.62**	12.8	27.2*	16.8
Bare-earth elevation map (DTM)	2.30	2.45	**2.14**	2.51	2.62*	2.75
Surface elevation map (DSM)	1.31	**1.30**	1.61	1.40	1.52	3.56
Point density map	2.42	2.28	2.48	1.82	**1.14**	2.01
Outline of the covered area	**1.71**	1.83	1.91	1.86	3.24	2.00
Find noise points	4.15	3.78	**3.44**	36.0	6.44	37.7
Remove noise, then thin (one job)	7.28	7.20	**6.53**	41.3	11.5	46.3
Ground, noise, and neighbour vote (one job)	7.26	7.30	**6.33**	61.9	—	—
Surface normals and shape features	28.8	28.7	**5.89**	80.4	—	—

Seconds, median of the timed runs; the fastest tool in each row is in bold blue; "—" = the tool cannot do this job; * = LAStools ran unlicensed and says its output is deliberately distorted.

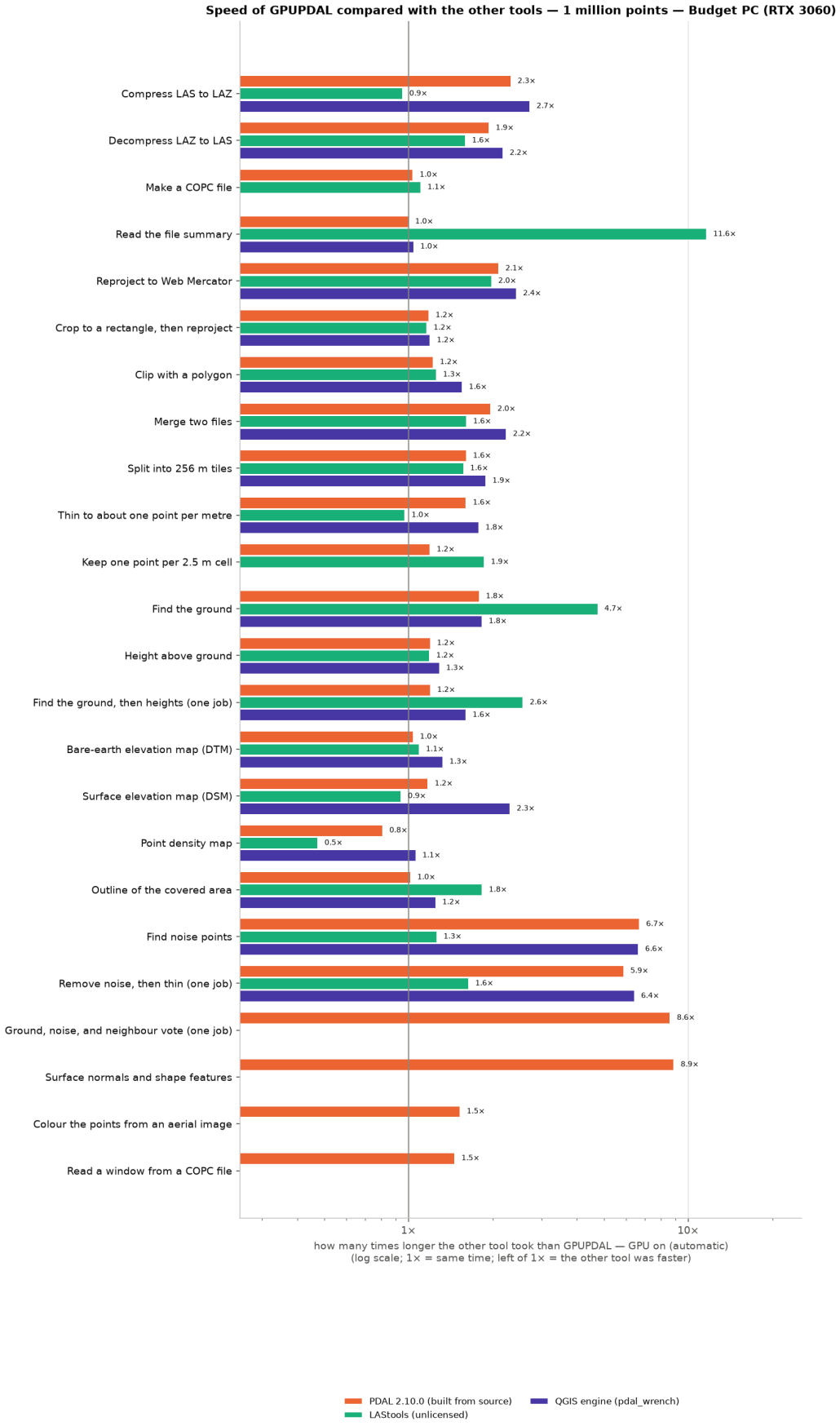


Figure. Other tool's time ÷ GPUPDAL's time, 1 million points, Budget PC (RTX 3060).

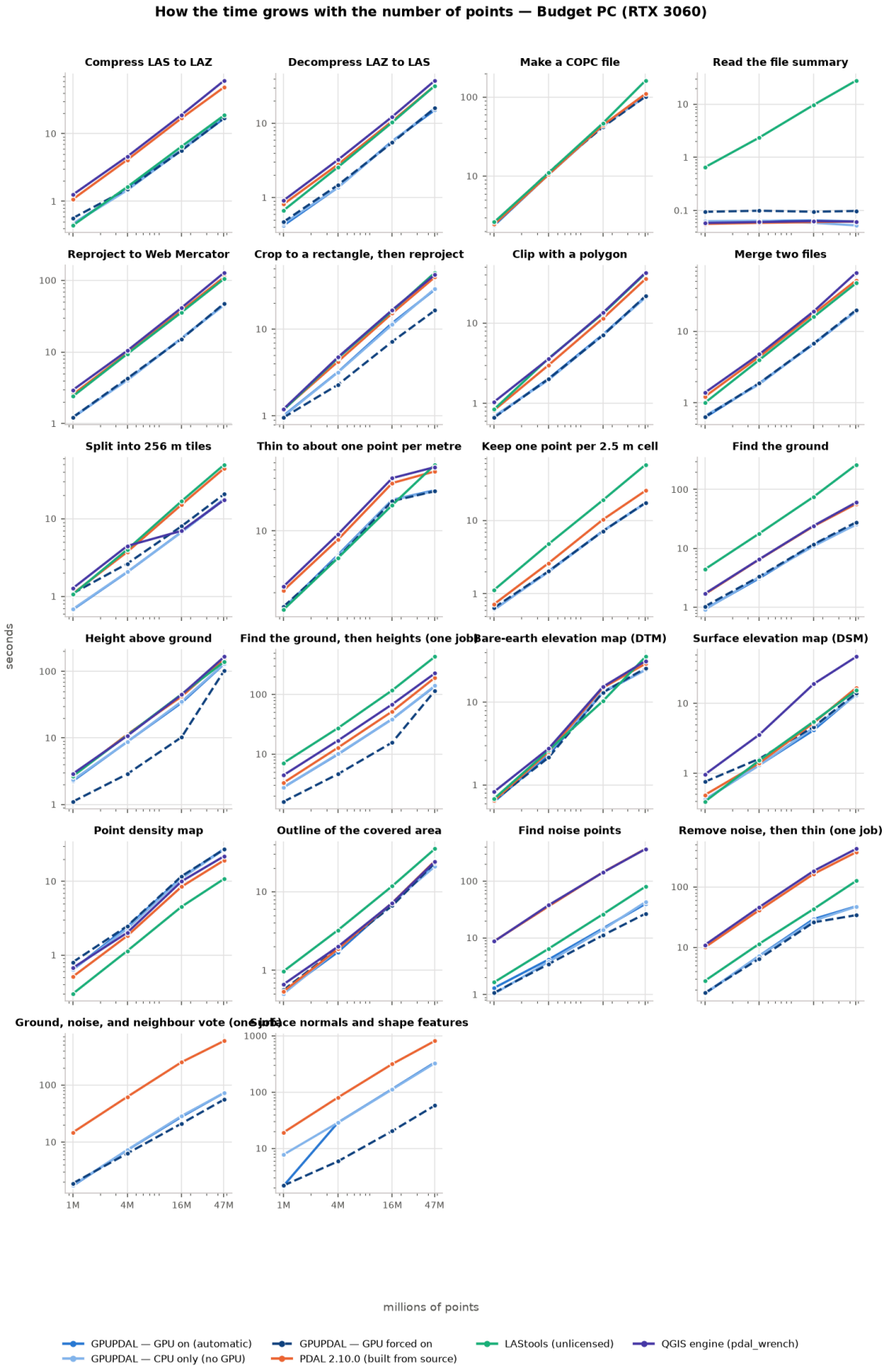


Figure. Time versus number of points, Budget PC (RTX 3060).

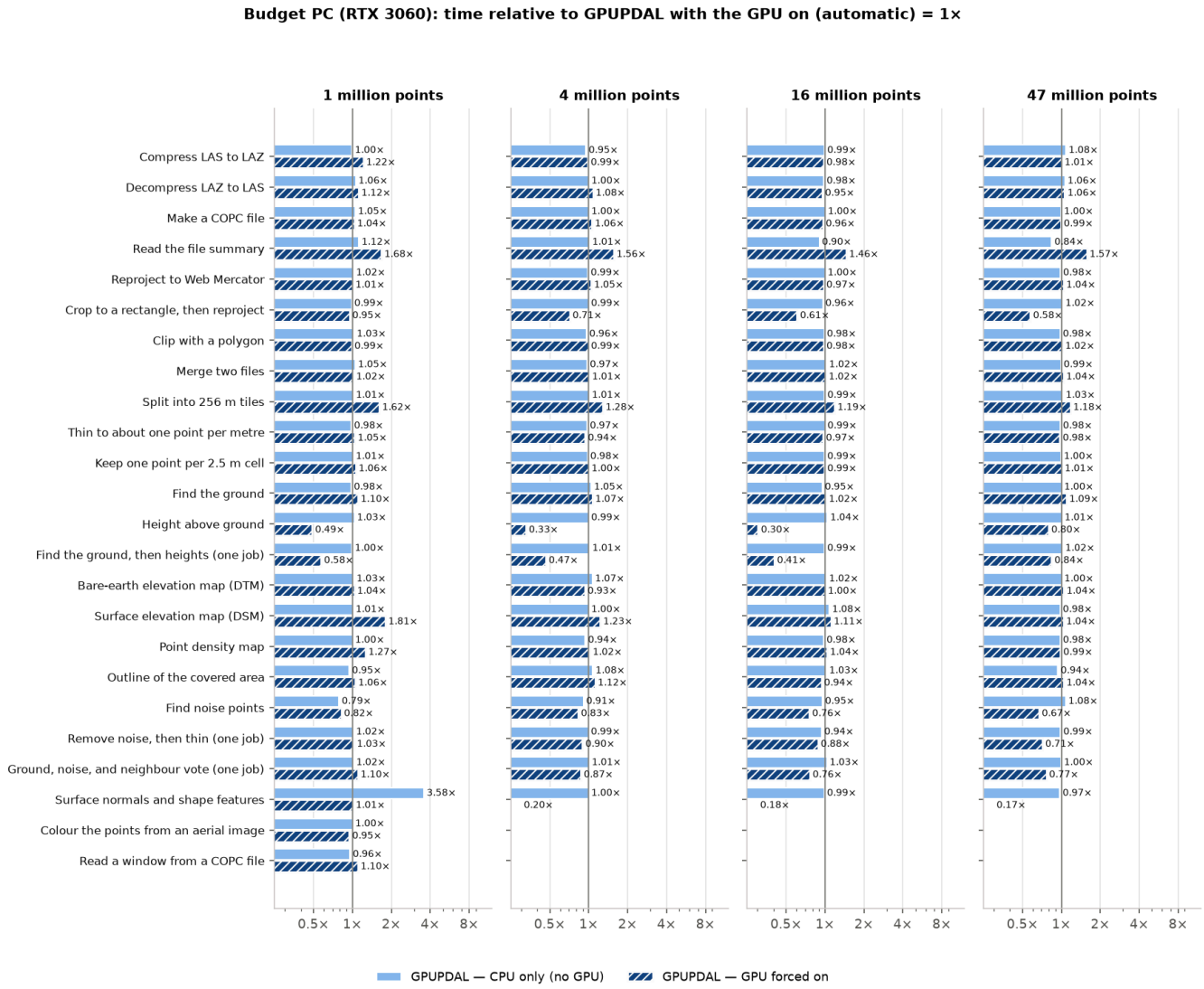


Figure. GPUPDAL CPU-only and GPU-forced time relative to automatic, Budget PC (RTX 3060).

Side by side: the same jobs on all three computers

Figure. Selected jobs on all three computers, 1 million points (seconds, log scale).

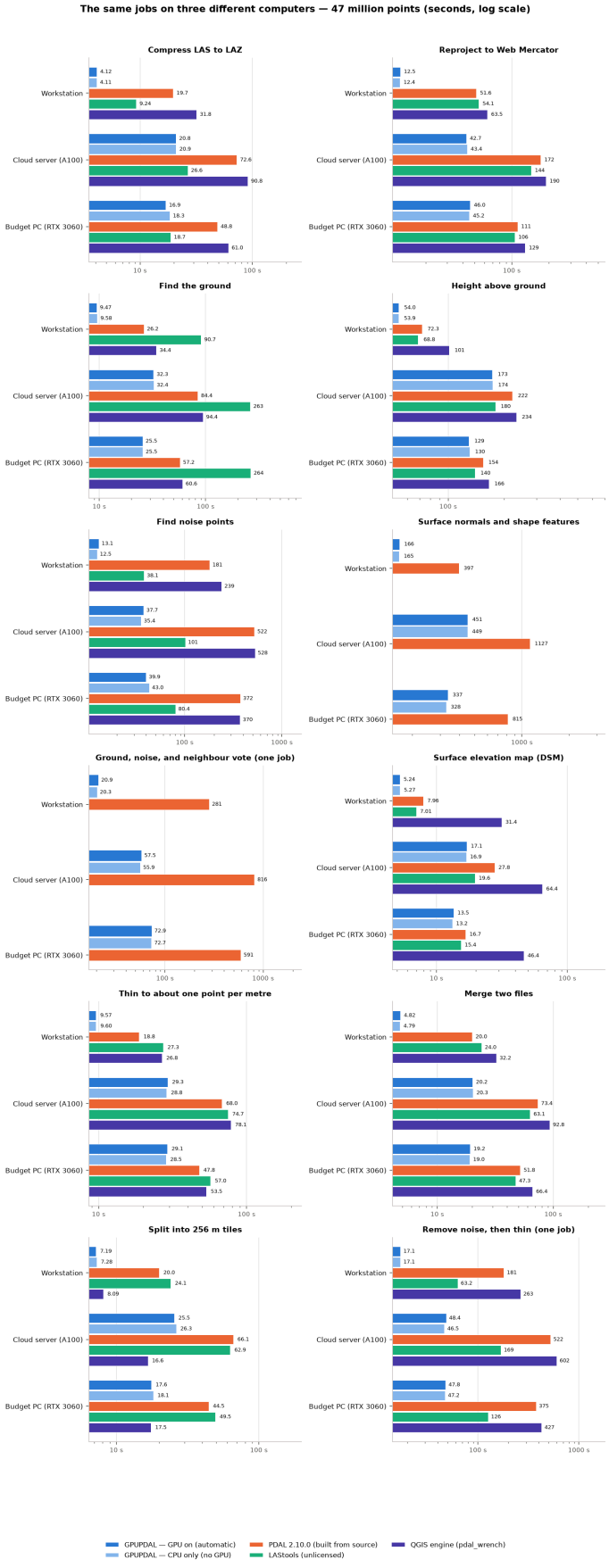


Figure. Selected jobs on all three computers, 47 million points (seconds, log scale).

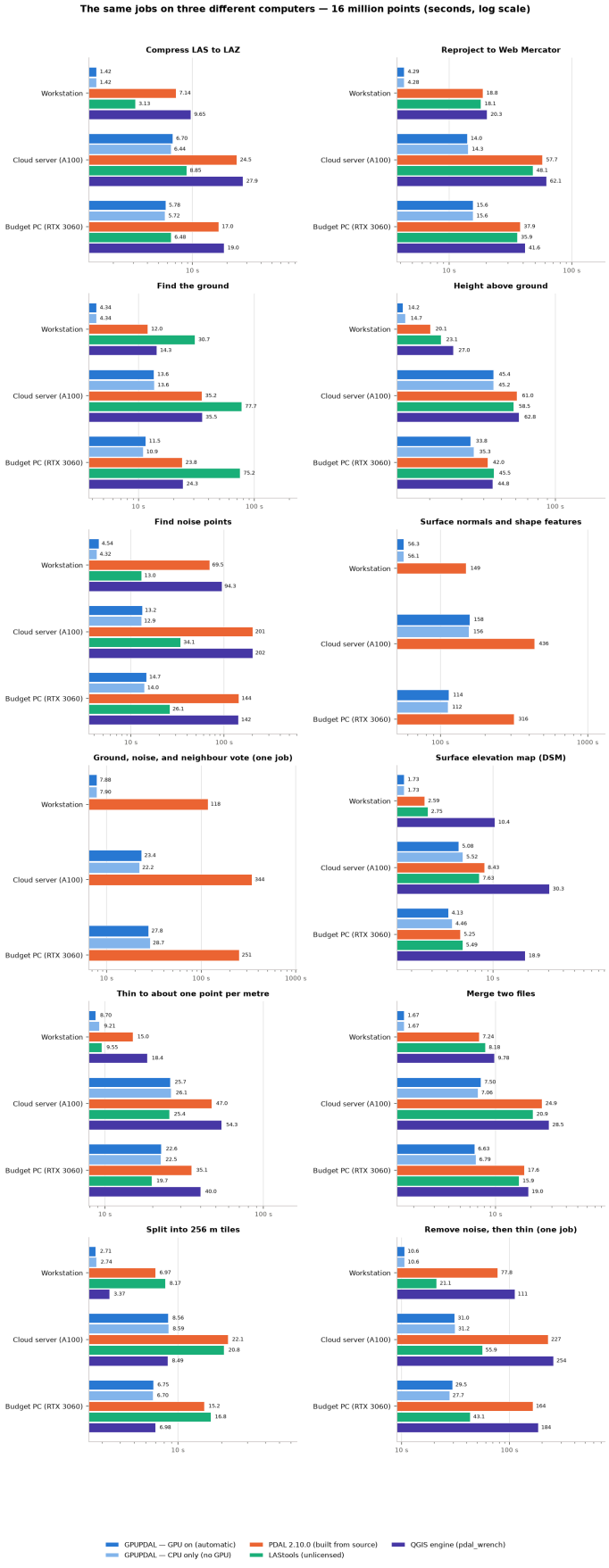


Figure. Selected jobs on all three computers, 16 million points (seconds, log scale).

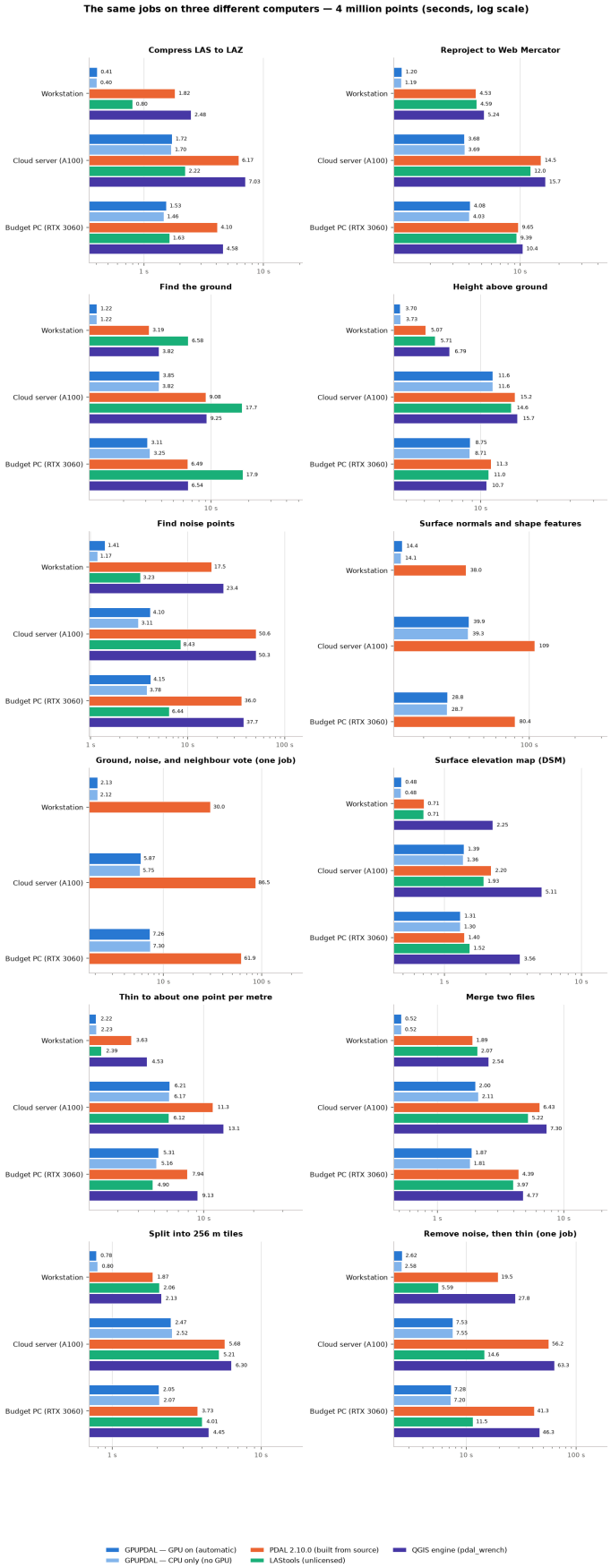


Figure. Selected jobs on all three computers, 4 million points (seconds, log scale).

6. What the outputs look like

These pictures are made from the actual output files of the 1-million-point runs on the workstation. Where tools use different methods, their outputs are shown next to each other.

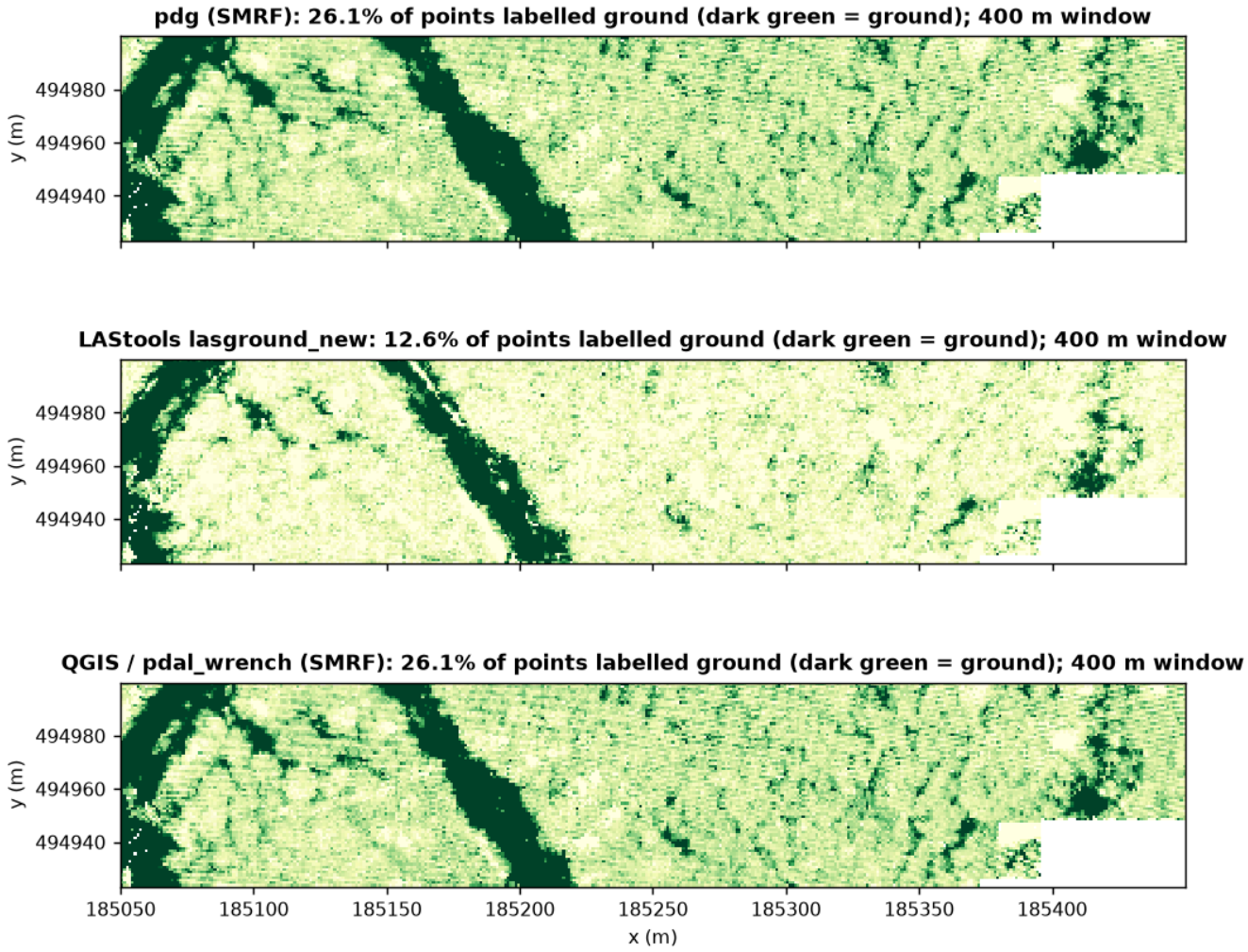


Figure. "Find the ground": which points each tool labelled as ground (green). GPUPDAL and PDAL give exactly the same answer (SMRF); QGIS also uses SMRF; LAStools uses its own method and finds a different set.

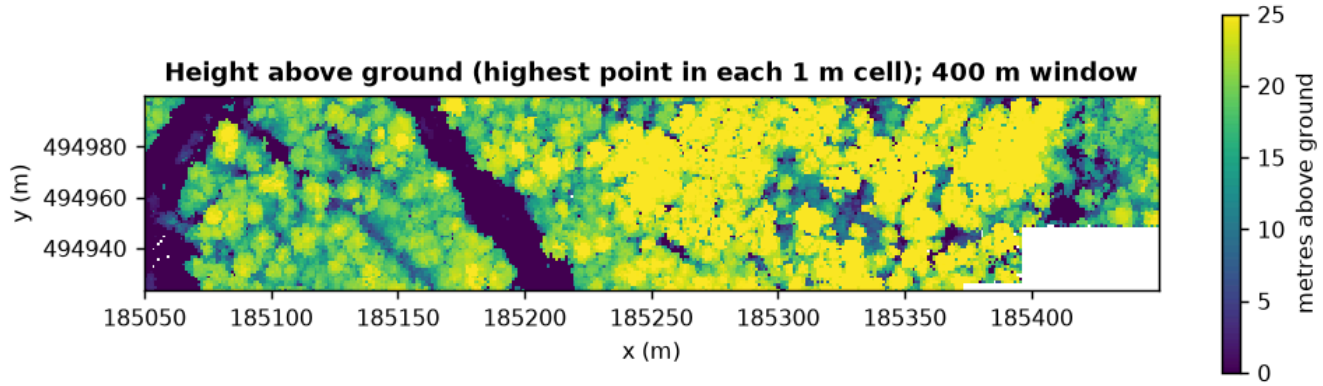


Figure. "Height above ground": the tallest thing in each 1 m cell (GPUPDAL / PDAL output).

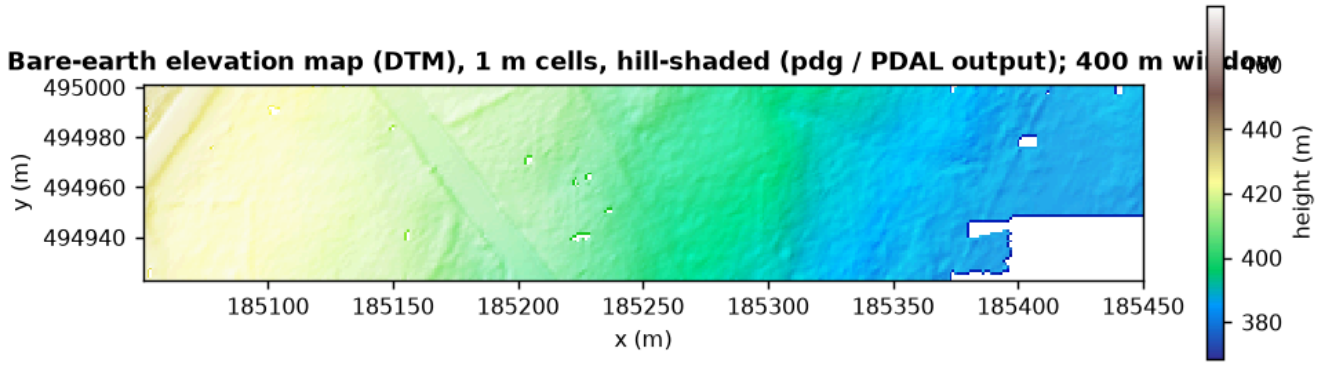


Figure. "Bare-earth elevation map (DTM)" from GPUPDAL / PDAL, hill-shaded.

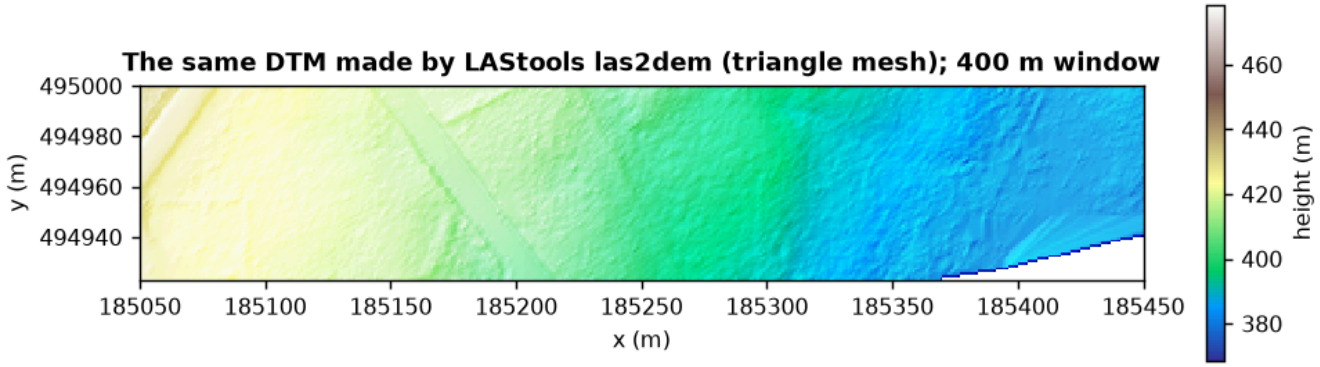


Figure. The same DTM job done by LAStools las2dem (triangle mesh) — smoother because it interpolates across gaps.

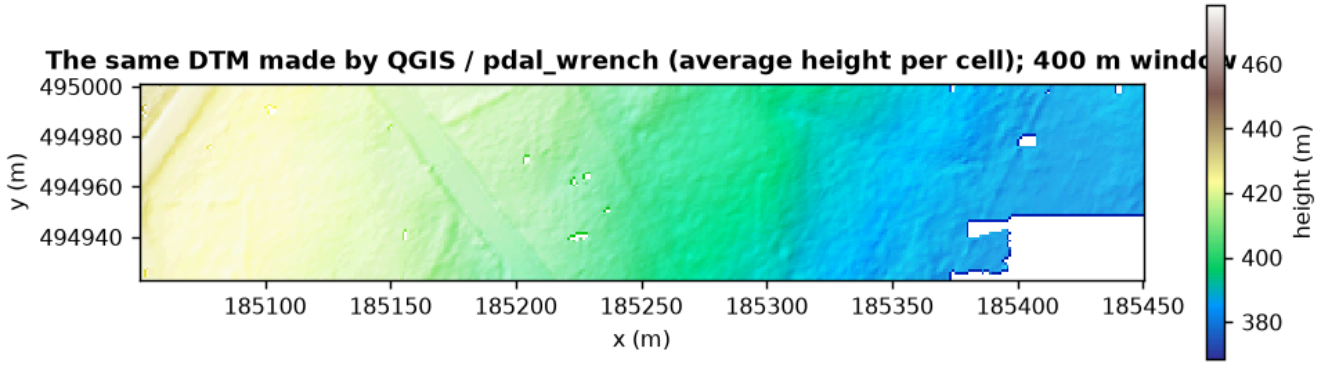


Figure. The same DTM job done by QGIS / pdal_wrench (average ground height per cell).

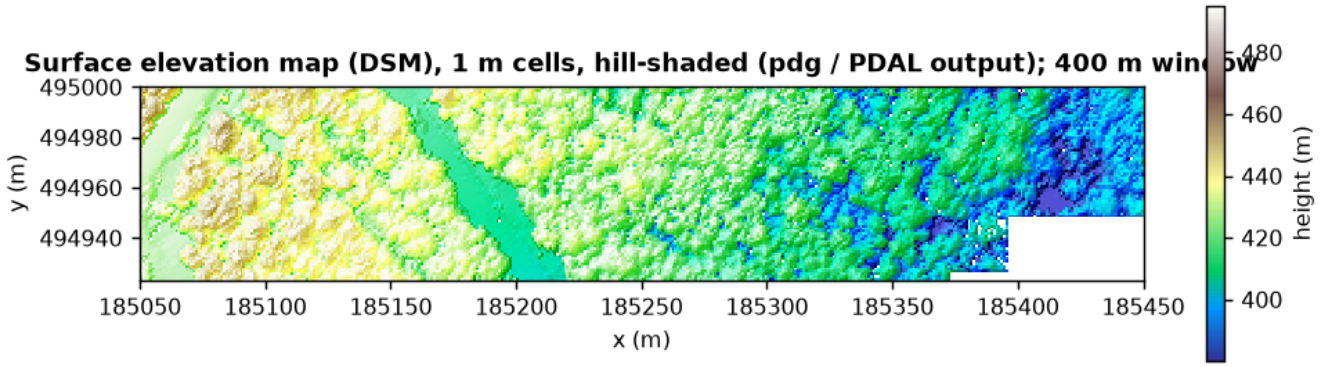


Figure. "Surface elevation map (DSM)": highest first return per 1 m cell (GPUPDAL / PDAL output).

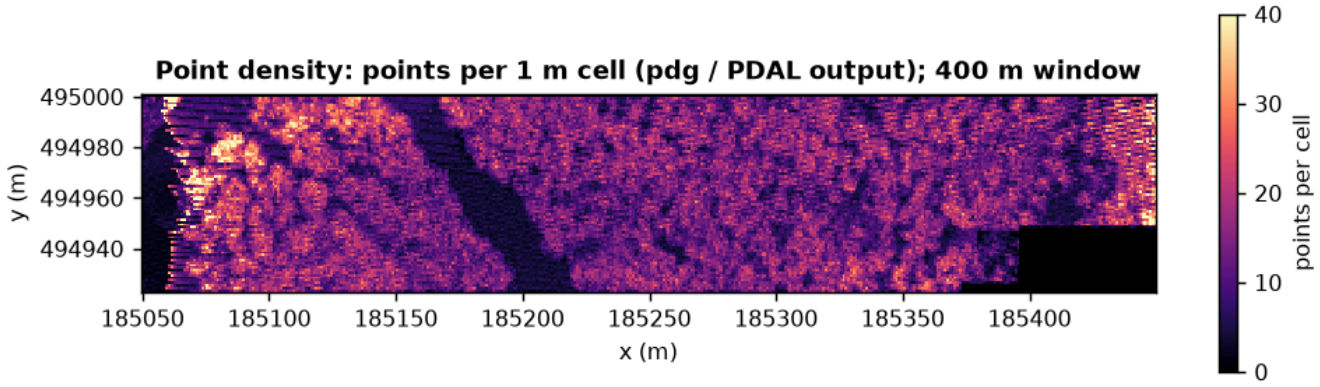


Figure. "Point density map": points per 1 m cell (GPUPDAL / PDAL output).

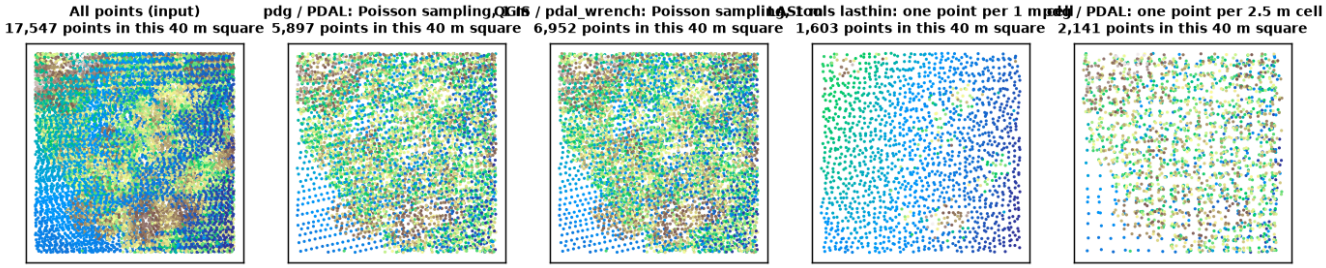


Figure. Thinning: a 40 m square of the input and the outputs of the thinning jobs. Poisson sampling keeps points that are at least 1 m apart in 3-D (so walls and tree crowns keep more points); LAStools' lasthin keeps one point per 1 m ground cell; the 2.5 m job keeps one point per 2.5 m cell.

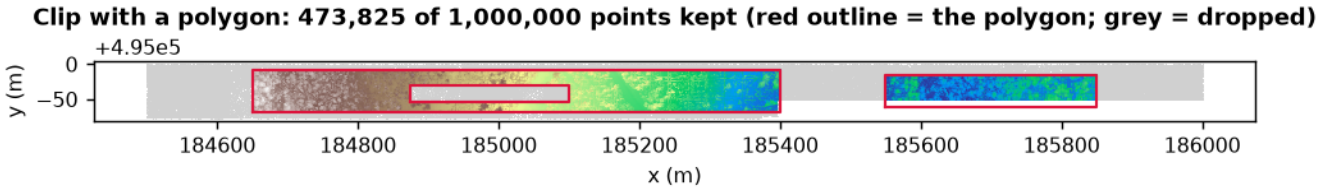


Figure. "Clip with a polygon": the kept points (coloured) and the polygon (red). Note the hole in the left rectangle. LAStools' lasclip kept the points inside the hole too (see the point counts in section 7).

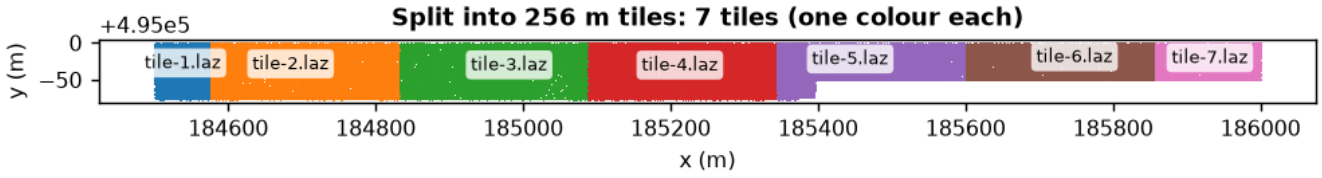


Figure. "Split into 256 m tiles": one colour per output file.

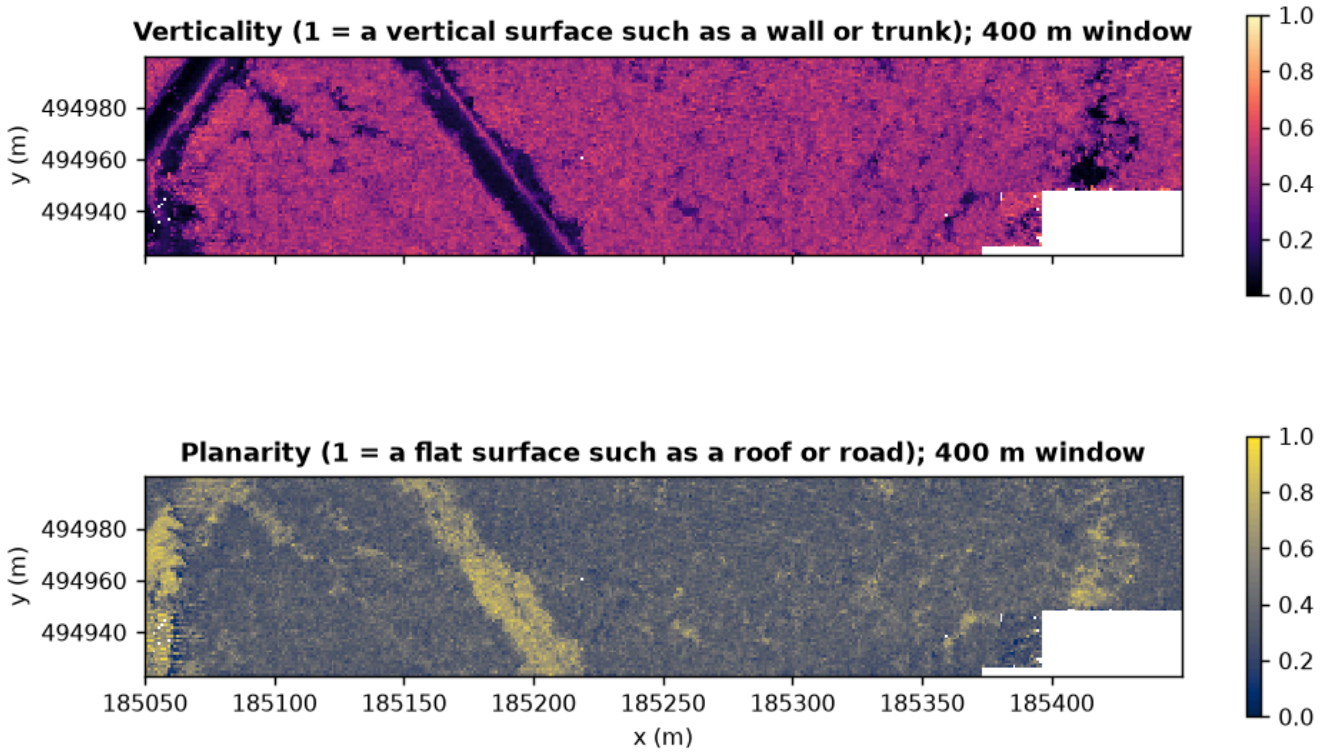


Figure. "Surface normals and shape features": two of the computed features, verticality and planarity, averaged per 1 m cell.

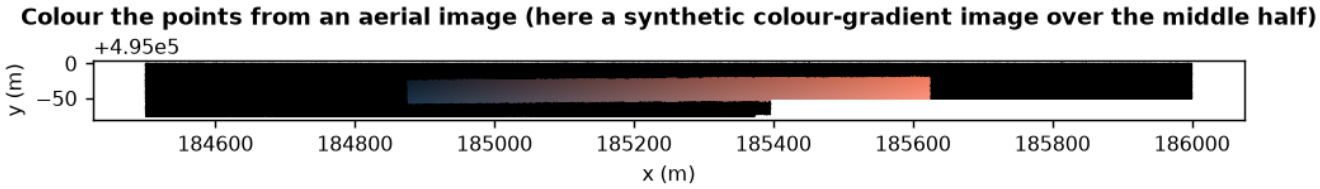


Figure. "Colour the points from an aerial image": the test image is a synthetic colour gradient in Web Mercator covering the middle half of the strip, so the points there get colours and the rest stay black.

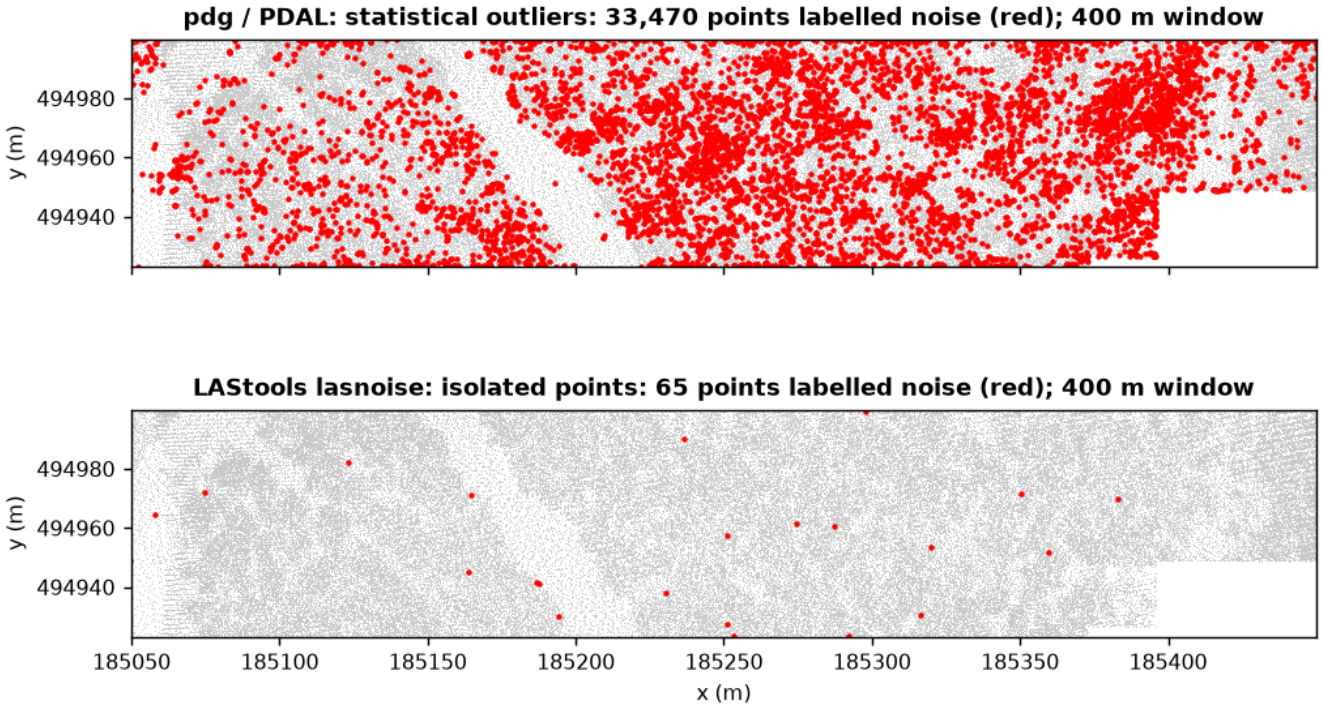


Figure. "Find noise points": the points labelled noise (red) by GPUPDAL / PDAL (statistical outliers) and by LAStools lasnoise (isolated points). Different methods, different answers.

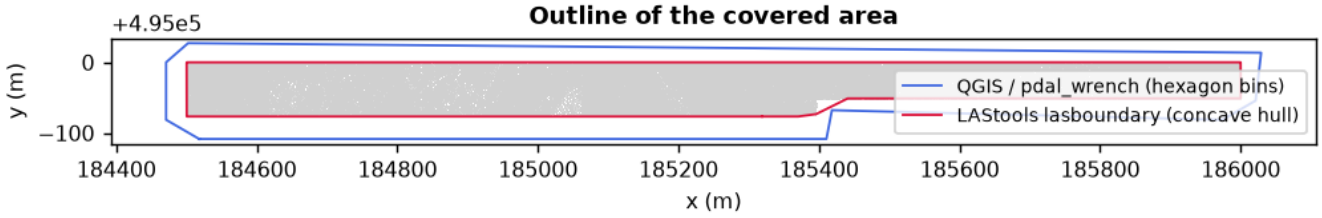


Figure. "Outline of the covered area" as drawn by the QGIS engine (hexagon bins) and by LAStools (concave hull).

7. Are the outputs the same?

GPUPDAL default mode preserves PDAL 2.10.0 semantics. We checksum deterministic outputs byte-for-byte; inherently nondeterministic containers such as COPC require the separately versioned canonical comparison. The table also shows whether GPUPDAL's CPU-only and GPU-forced modes gave the same deterministic file as its automatic mode.

Job	Same output as PDAL 2.10.0?	GPUPDAL's other modes vs its automatic mode
Compress LAS to LAZ	yes — identical bytes	CPU only: same, GPU forced: same
Decompress LAZ to LAS	yes — identical bytes	CPU only: same, GPU forced: same
Make a COPC file	not byte-identical — historical exploratory result; run the explicit canonical COPC comparator for supplemental semantics	CPU only: differs, GPU forced: differs
Read the file summary	no file written (text output)	
Reproject to Web Mercator	yes — identical bytes	CPU only: same, GPU forced: same
Crop to a rectangle, then reproject	yes — identical bytes	CPU only: same, GPU forced: same
Clip with a polygon	yes — identical bytes	CPU only: same, GPU forced: same
Merge two files	yes — identical bytes	CPU only: same, GPU forced: same
Split into 256 m tiles	historical check only — same file count, total size and point count; directory members were not hashed	CPU only: same, GPU forced: same
Thin to about one point per metre	yes — identical bytes	CPU only: same, GPU forced: same
Keep one point per 2.5 m cell	yes — identical bytes	CPU only: same, GPU forced: same
Find the ground	yes — identical bytes	CPU only: same, GPU forced: same
Height above ground	yes — identical bytes	CPU only: same, GPU forced: same
Find the ground, then heights (one job)	yes — identical bytes	CPU only: same, GPU forced: same
Bare-earth elevation map (DTM)	yes — identical bytes	CPU only: same, GPU forced: same
Surface elevation map (DSM)	yes — identical bytes	CPU only: same, GPU forced: same
Point density map	yes — identical bytes	CPU only: same, GPU forced: same
Outline of the covered area	no file written (text output)	
Find noise points	yes — identical bytes	CPU only: same, GPU forced: same
Remove noise, then thin (one job)	yes — identical bytes	CPU only: same, GPU forced: same
Ground, noise, and neighbour vote (one job)	yes — identical bytes	CPU only: same, GPU forced: same
Surface normals and shape features	yes — identical bytes	CPU only: same, GPU forced: same

Job	Same output as PDAL 2.10.0?	GPUPDAL's other modes vs its automatic mode
Colour the points from an aerial image	yes — identical bytes	CPU only: same, GPU forced: same
Read a window from a COPC file	yes — identical bytes	CPU only: same, GPU forced: same

The same historical check on the larger files: 4 million points: 18 of 20 file-writing jobs proved byte-identical (different: Make a COPC file; historical directory runs checked only by file count, total size and point count: Split into 256 m tiles); 16 million points: 18 of 20 file-writing jobs proved byte-identical (different: Make a COPC file; historical directory runs checked only by file count, total size and point count: Split into 256 m tiles); 47 million points: 18 of 20 file-writing jobs proved byte-identical (different: Make a COPC file; historical directory runs checked only by file count, total size and point count: Split into 256 m tiles). A COPC byte mismatch remains a mismatch; semantic equivalence now requires the separately versioned canonical comparator.

The other tools do not promise the same output, and often use a different method for the same job. The number of points in each output shows where they differ:

Job	GPUPDAL (GPU)	PDAL 2.10.0	LAStools	QGIS engine	QGIS
Compress LAS to LAZ	1,000,000	1,000,000	1,000,000	1,000,000	1,000,000
Decompress LAZ to LAS	1,000,000	1,000,000	1,000,000	1,000,000	1,000,000
Make a COPC file	1,000,000	1,000,000	1,000,000	1,000,000	1,000,000
Reproject to Web Mercator	1,000,000	1,000,000	1,000,000	1,000,000	1,000,000
Crop to a rectangle, then reproject	291,617	291,617	291,617	291,617	—
Clip with a polygon	473,825	473,825	522,679	473,825	473,825
Merge two files	1,000,000	1,000,000	1,000,000	1,000,000	1,000,000
Split into 256 m tiles	1,000,000	1,000,000	1,000,000	1,000,000	1,000,000
Thin to about one point per metre	320,298	320,298	100,047	383,993	383,993
Keep one point per 2.5 m cell	122,162	122,162	16,180	—	—
Find the ground	1,000,000	1,000,000	1,000,000	1,000,000	1,000,000
Height above ground	1,000,000	1,000,000	1,000,000	1,000,000	1,000,000
Find the ground, then heights (one job)	1,000,000	1,000,000	1,000,000	1,000,000	1,000,000
Find noise points	1,000,000	1,000,000	1,000,000	1,000,000	1,000,000
Remove noise, then thin (one job)	295,622	295,622	100,047	357,383	357,383
Ground, noise, and neighbour vote (one job)	1,000,000	1,000,000	—	—	—
Surface normals and shape features	1,000,000	1,000,000	—	—	—
Colour the points from an aerial image	1,000,000	1,000,000	—	—	—
Read a window from a COPC file	280,130	280,130	—	—	—

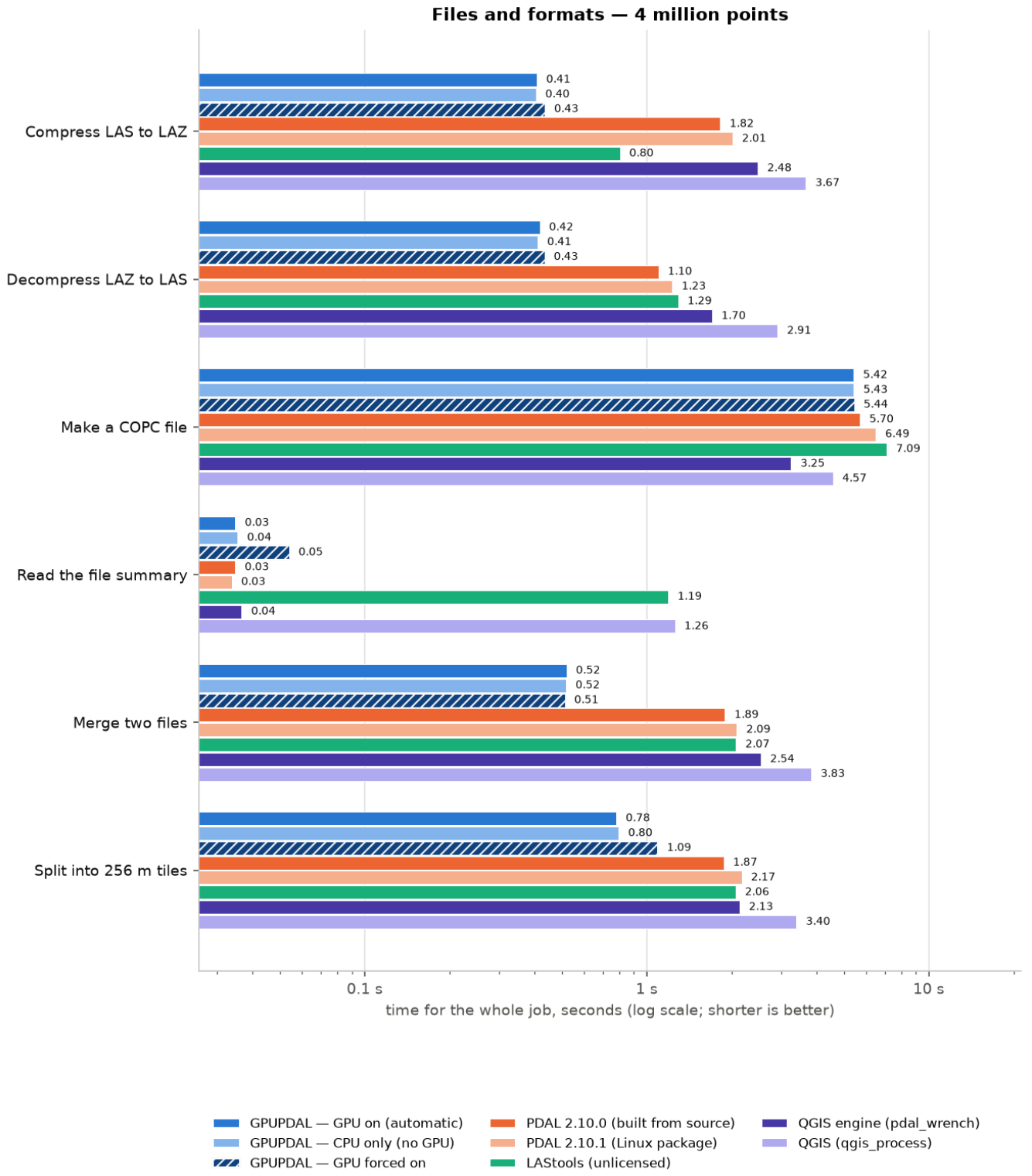
Notes: "Thin" — Poisson sampling in 3-D keeps more points than one-per-cell thinning; QGIS' engine also uses Poisson sampling but its parallel, tiled implementation keeps a different set. "Clip" — LAStools' lasclip ignored the hole in the polygon. "Find the ground" and "noise" — all tools keep every point but label them differently (see the pictures). "Crop, then reproject" — the same 291,617 points in every tool.

8. Things to keep in mind

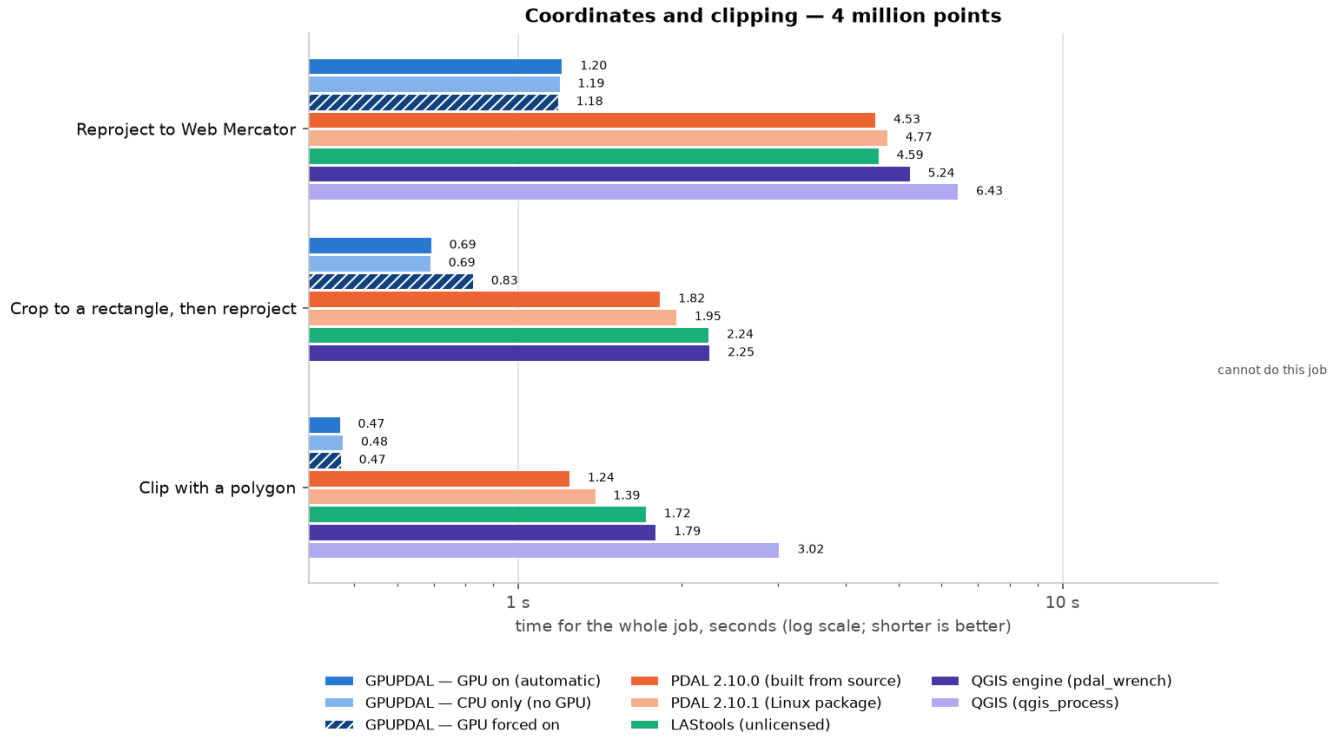
- **Same job, different method.** GPUPDAL and PDAL run the very same algorithms (GPUPDAL is a modified PDAL). QGIS' engine is also built on the PDAL library, so its methods are the same or close (SMRF ground, statistical outliers, Poisson thinning, hexagon boundary), but it splits work into tiles and threads. LAStools uses its own algorithms for ground, height, noise, thinning and rasters, so its times are for a job of the same name, not the same computation.
- **LAStools ran unlicensed.** The paid LAStools programs were used in their free test mode. Above 1.5–10 million points (the limit differs per tool) they print a warning that the output is deliberately distorted ("tiny xyz noise, points permuted, some fields zeroed", or a black diagonal drawn across a raster) and exit with an error code. The historical capture counted those timings and marked them "ran unlicensed"; they are exploratory, nonqualifying observations. The current harness preserves the error status. The free tools (laszip, las2las, lasmerge) are not affected.
- **QGIS start-up.** Every `qgis_process` call pays about 1.2 s to start QGIS and its Python plugins. For a user in the QGIS desktop this cost is paid once, not per job, so for long jobs "QGIS engine (pdal_wrench)" is the fairer number and for short jobs the truth is in between.
- **Threads.** PDAL and LAStools are single-threaded for these jobs. GPUPDAL and pdal_wrench use many CPU threads (up to 24 on the workstation). So part of "GPUPDAL is faster" is "GPUPDAL uses the whole CPU"; that is a real advantage for the user, but it also means the gap shrinks on machines with few cores, and grows on machines with many. On the rented machines the container had a CPU quota smaller than the number of visible threads.
- **Warm cache, one machine per type, few repeats for big files.** Inputs were in memory cache; the 47-million-point runs were timed once. Times on the two rented machines depend on which physical host we got.
- **PDAL versions.** GPUPDAL is based on PDAL 2.10.0. PDAL 2.10.1 from the Linux package was a little slower than the source build in most jobs, most likely because of different compiler settings, not because of the version.
- **Peak memory** is the largest resident memory of the whole job's process tree; for QGIS it includes QGIS itself.
- **This is a separate repository-run harness, not independent validation.** Its task definitions and inputs differ from the maintained `BENCHMARKS.md` suite, but the evidence was still produced by the project author against this repository.

9. Appendix

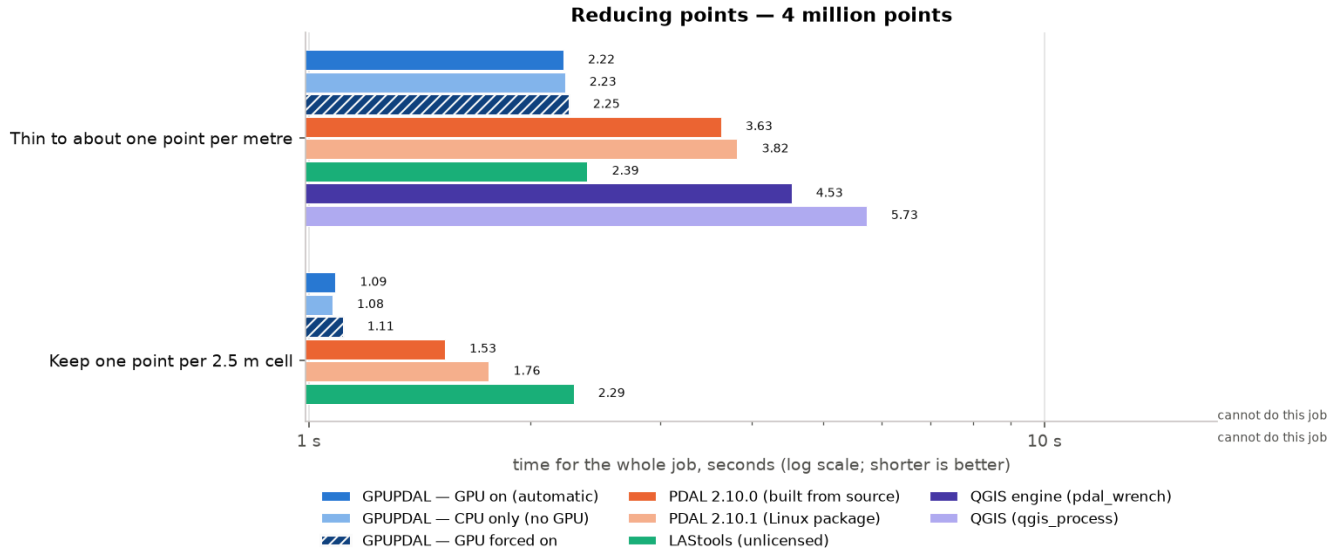
9.1 Every chart for the larger files (workstation)



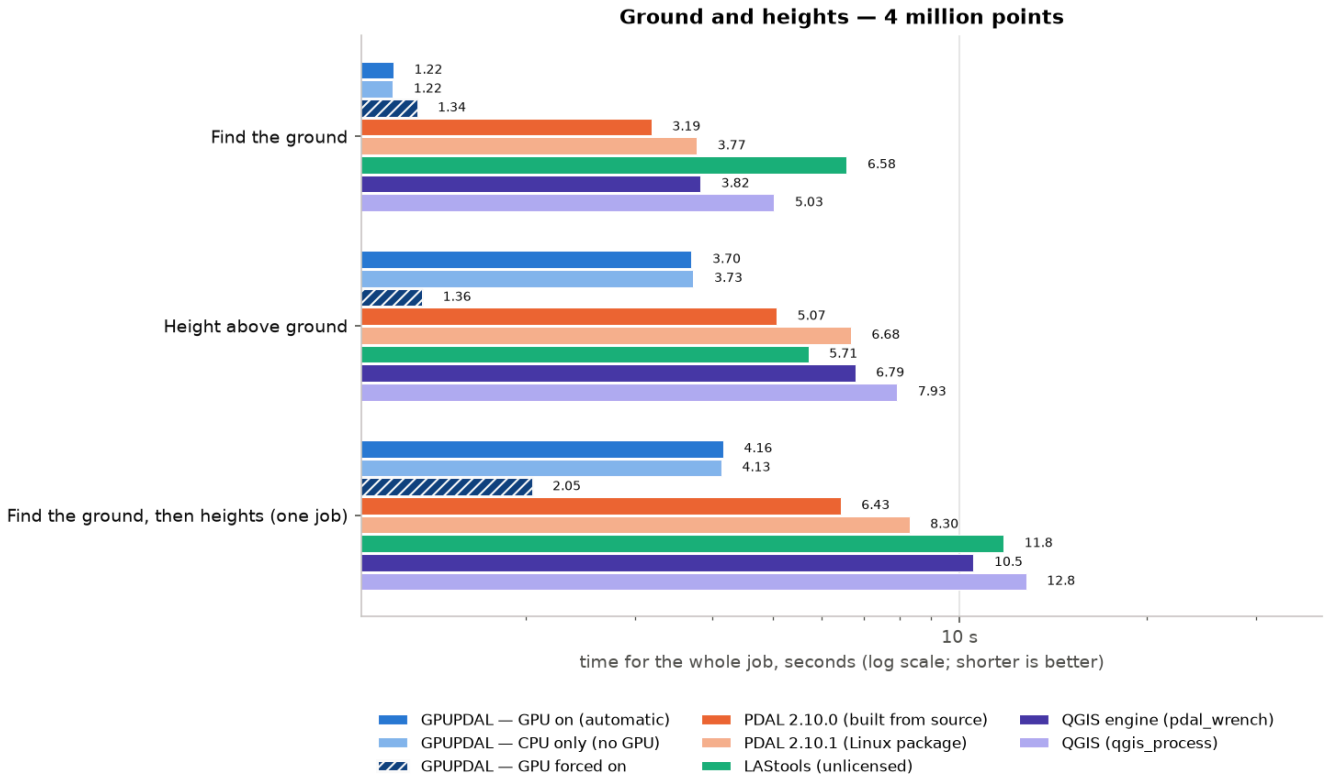
Files and formats: median time in seconds, 4 million points, workstation.



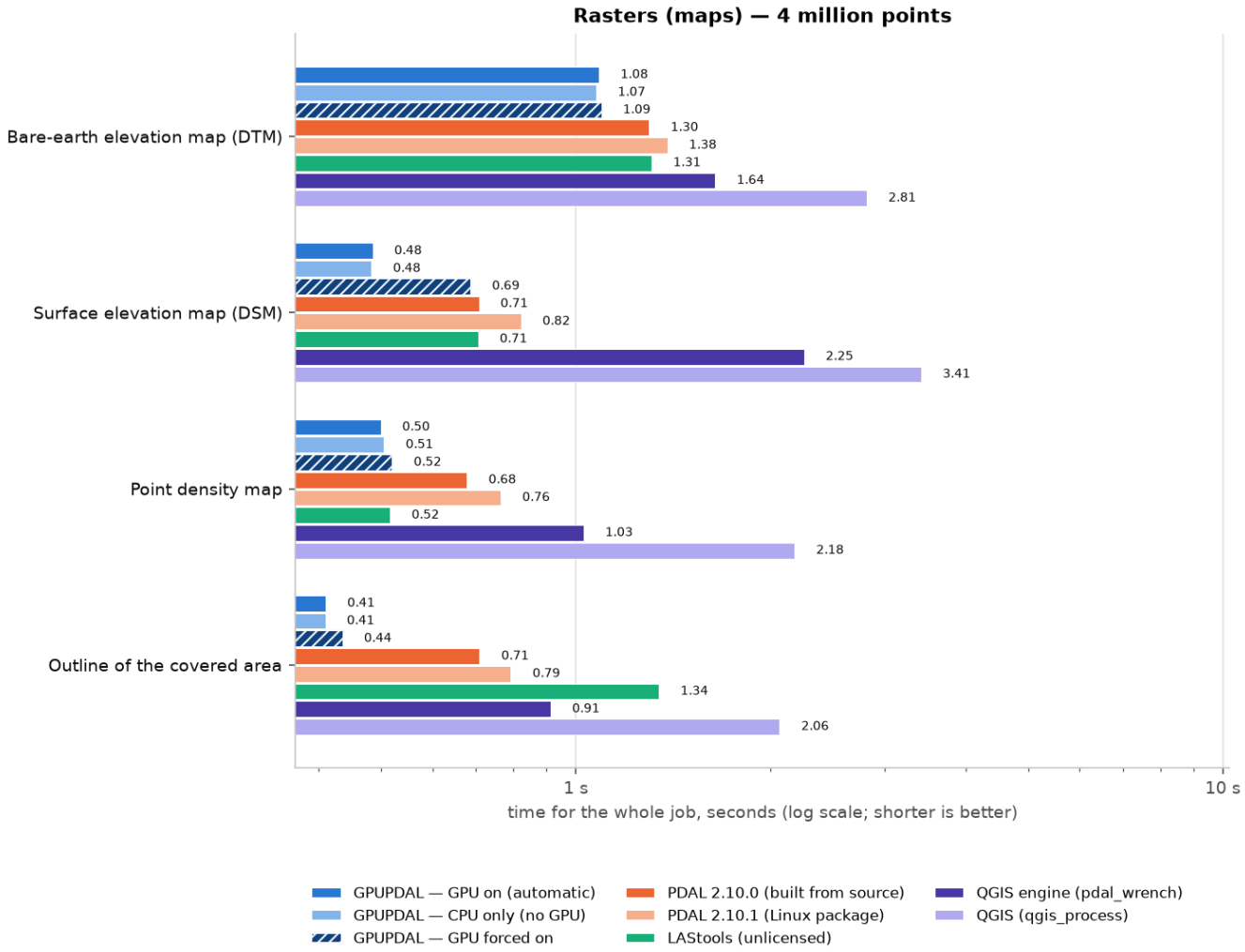
Coordinates and clipping: median time in seconds, 4 million points, workstation.



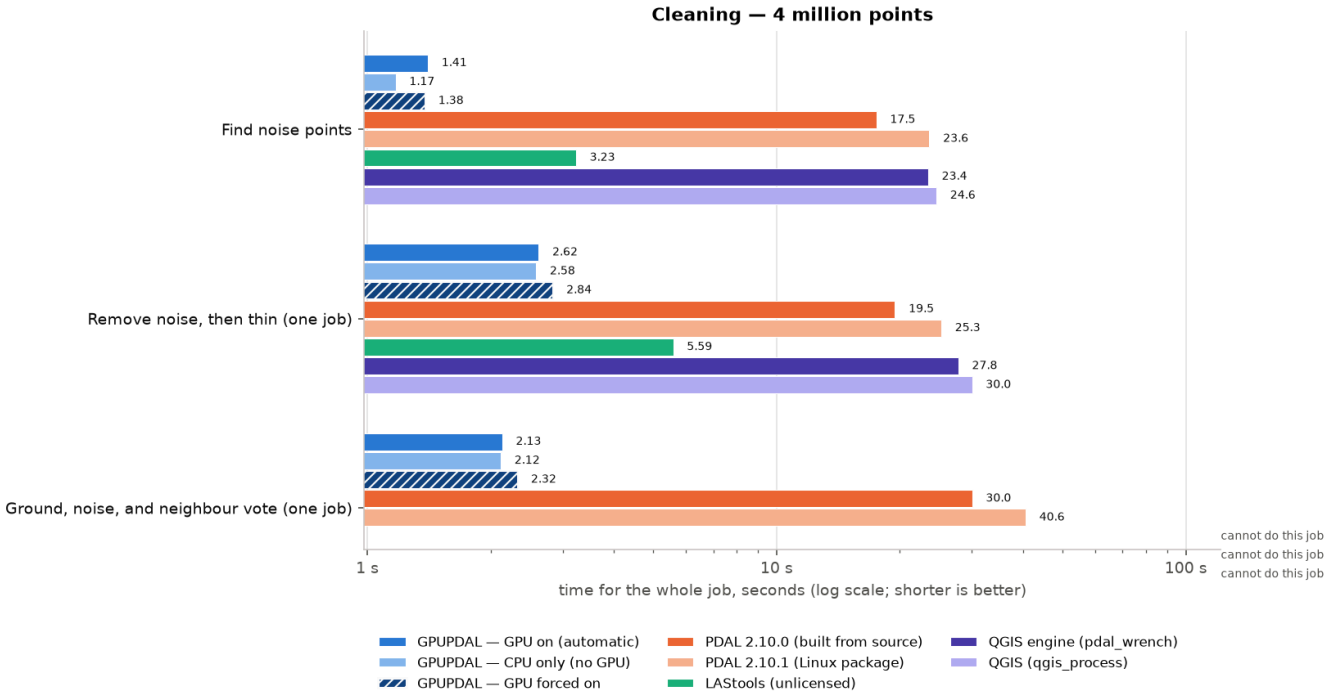
Reducing points: median time in seconds, 4 million points, workstation.



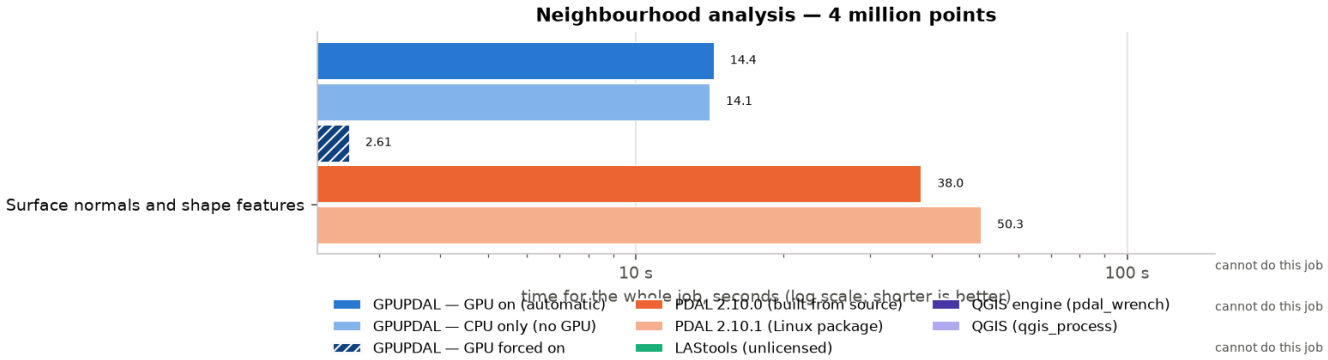
Ground and heights: median time in seconds, 4 million points, workstation.



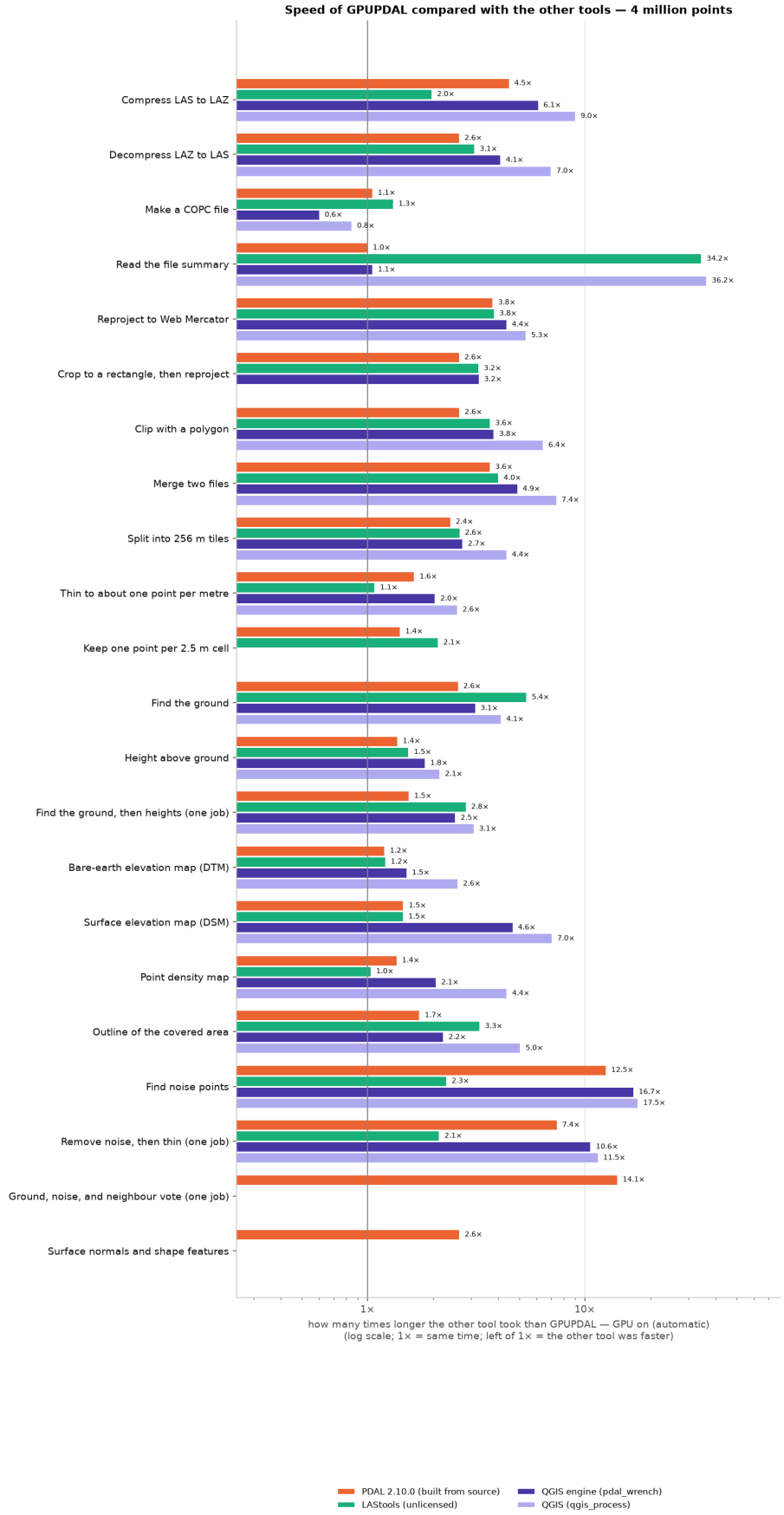
Rasters (maps): median time in seconds, 4 million points, workstation.



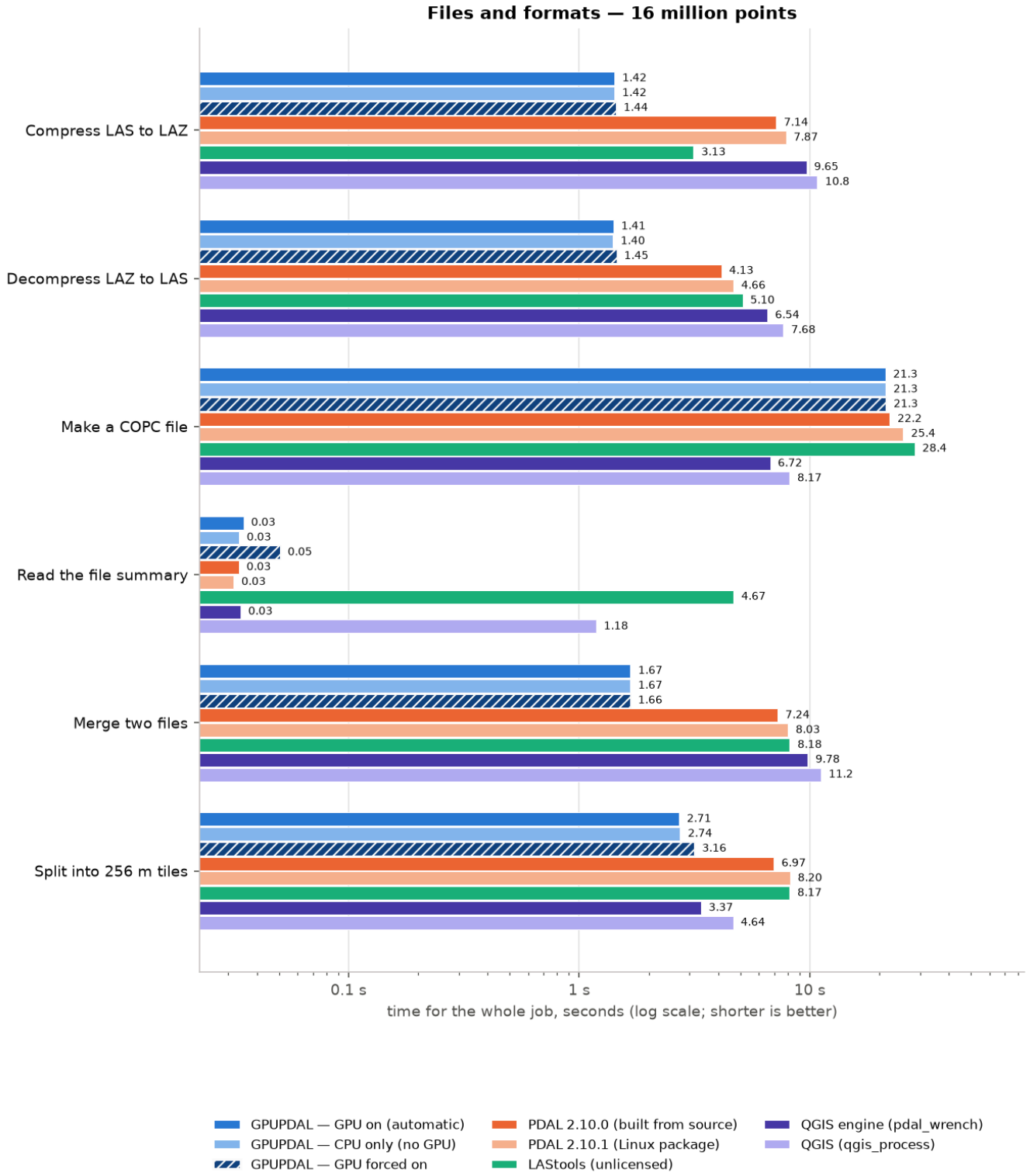
Cleaning: median time in seconds, 4 million points, workstation.



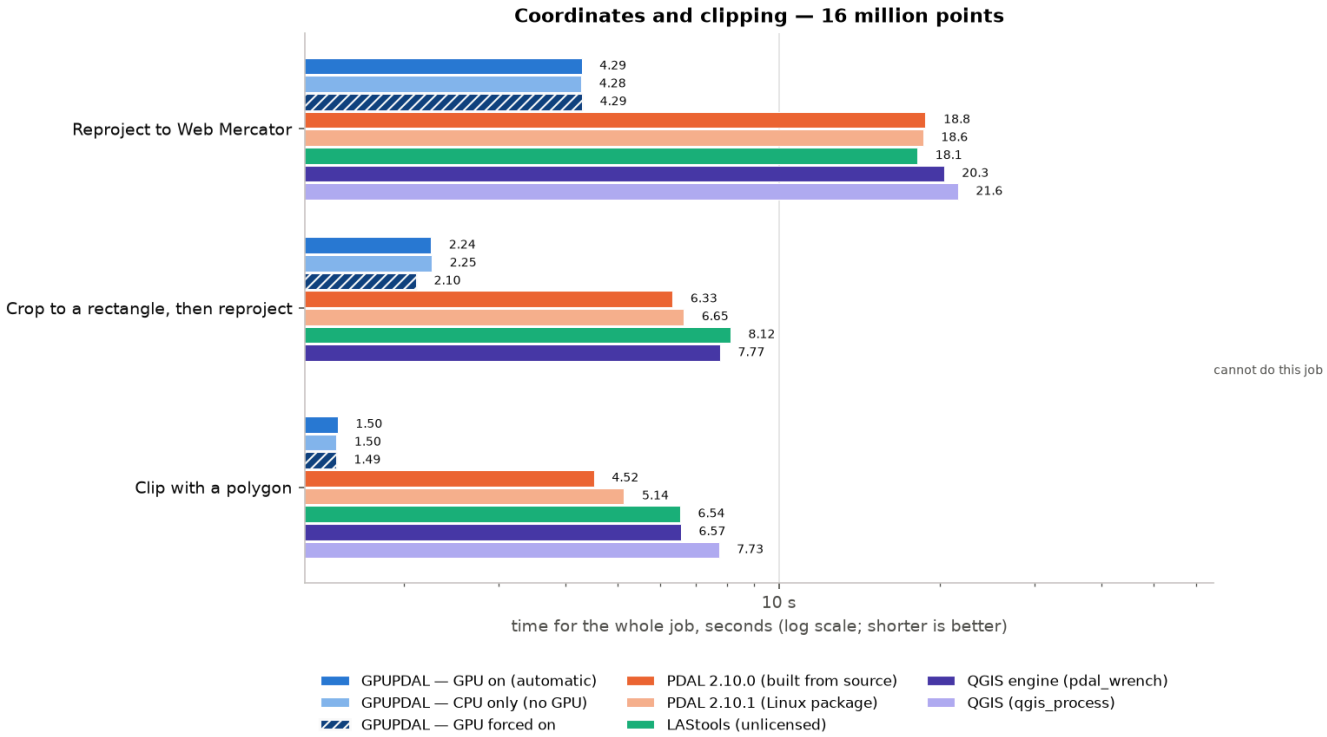
Neighbourhood analysis: median time in seconds, 4 million points, workstation.



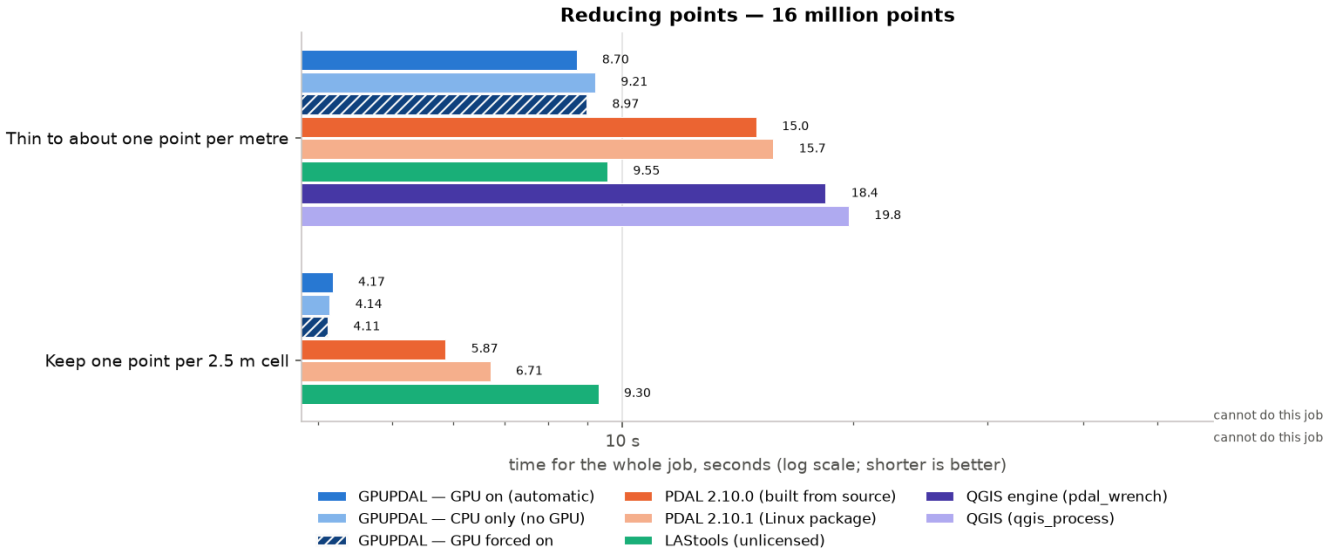
Other tool's time ÷ GPUPDAL's time, 4 million points, workstation.



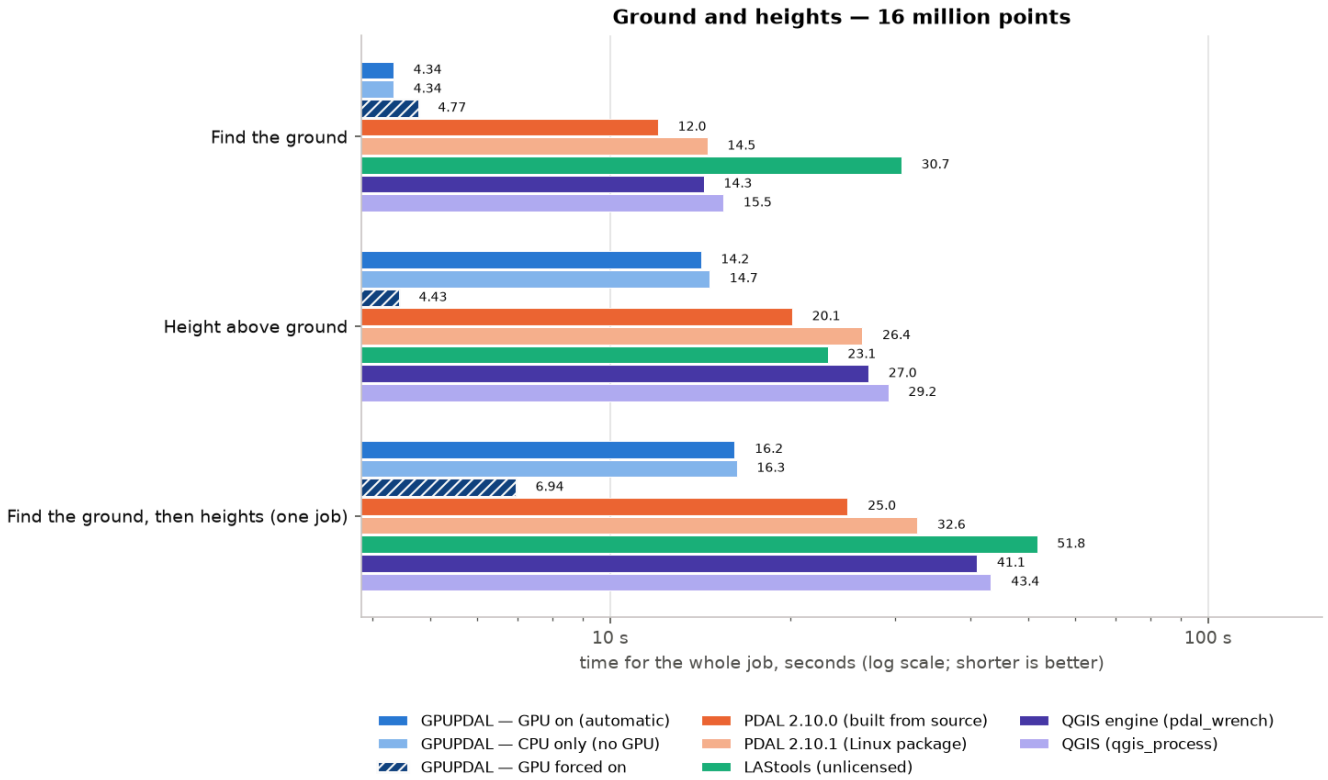
Files and formats: median time in seconds, 16 million points, workstation.



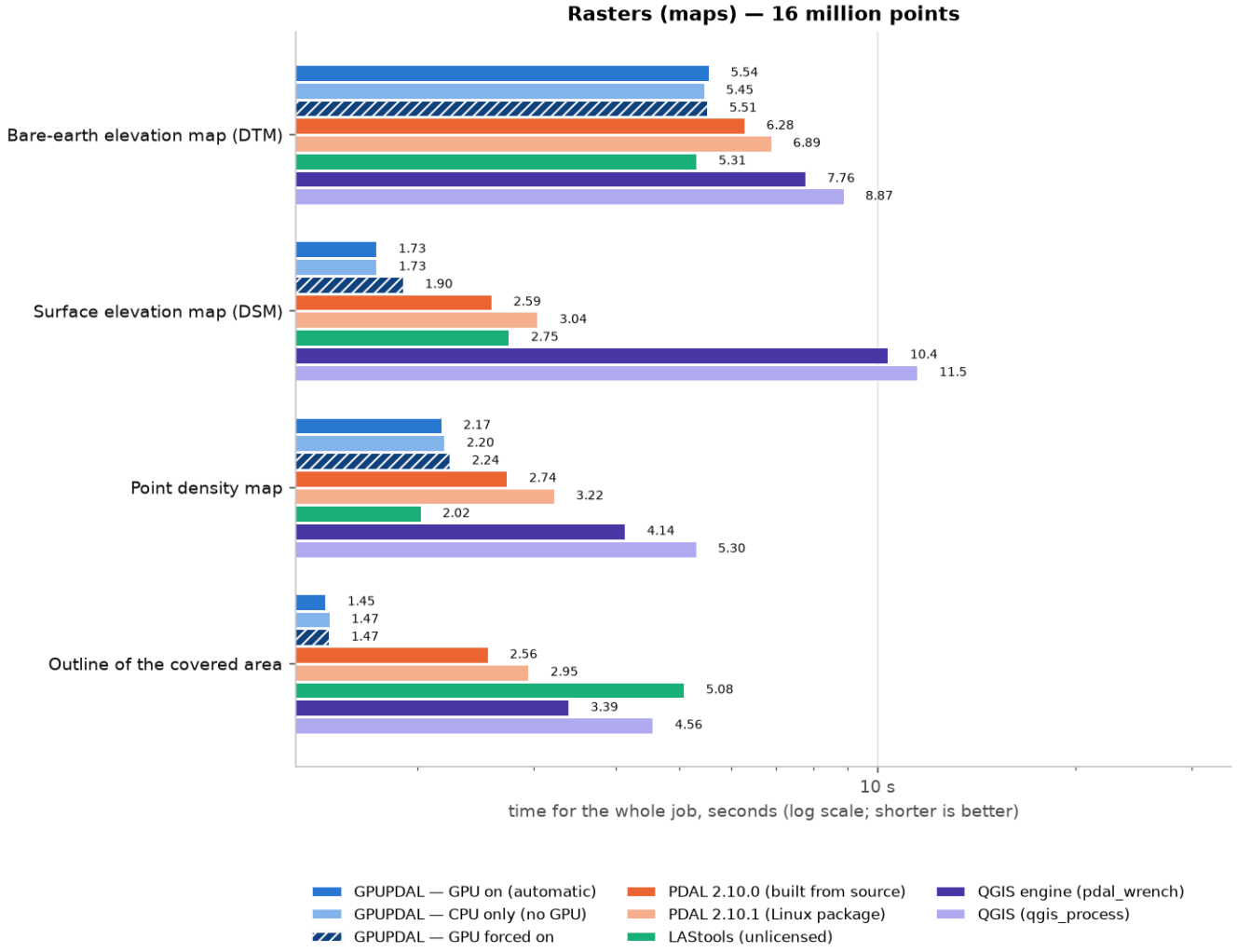
Coordinates and clipping: median time in seconds, 16 million points, workstation.



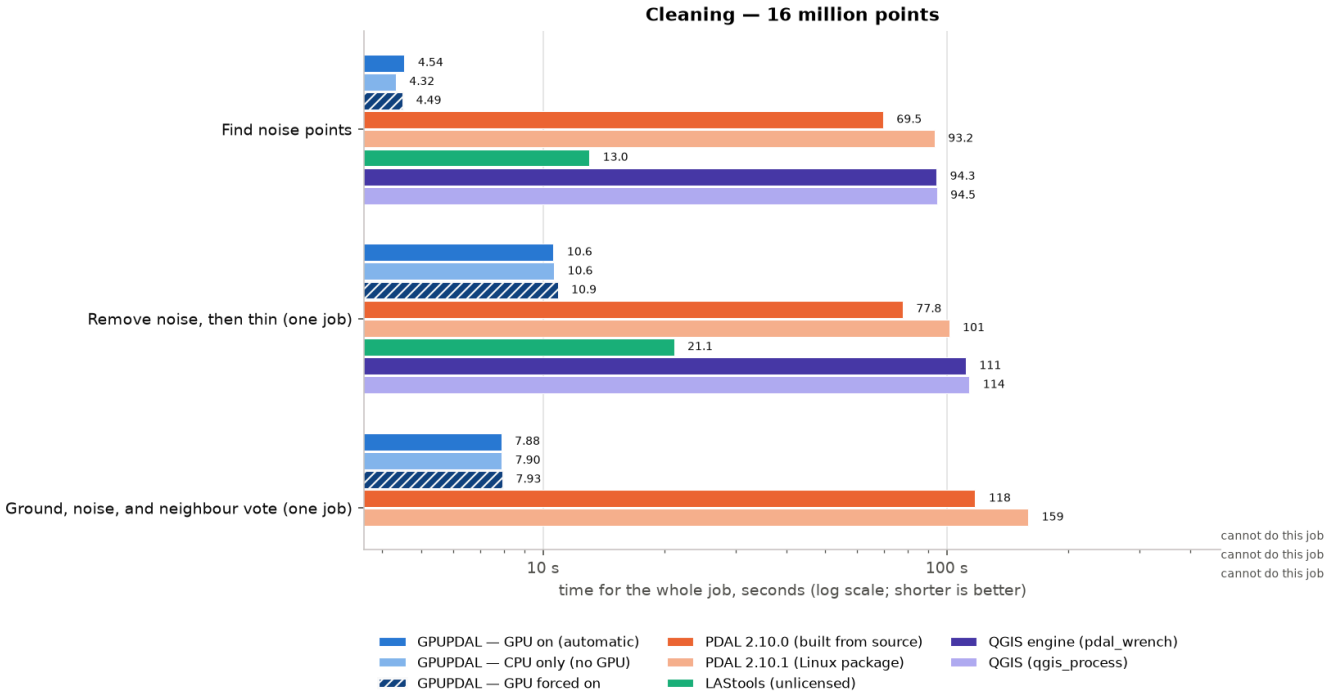
Reducing points: median time in seconds, 16 million points, workstation.



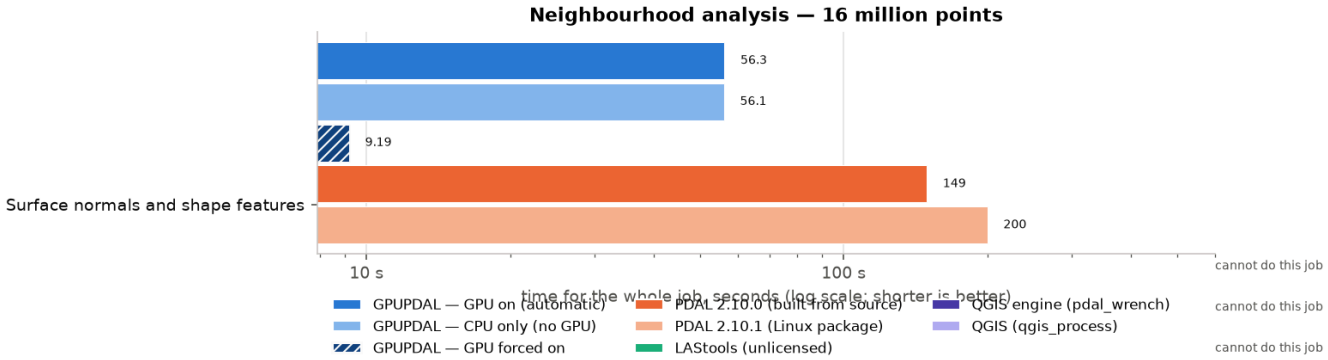
Ground and heights: median time in seconds, 16 million points, workstation.



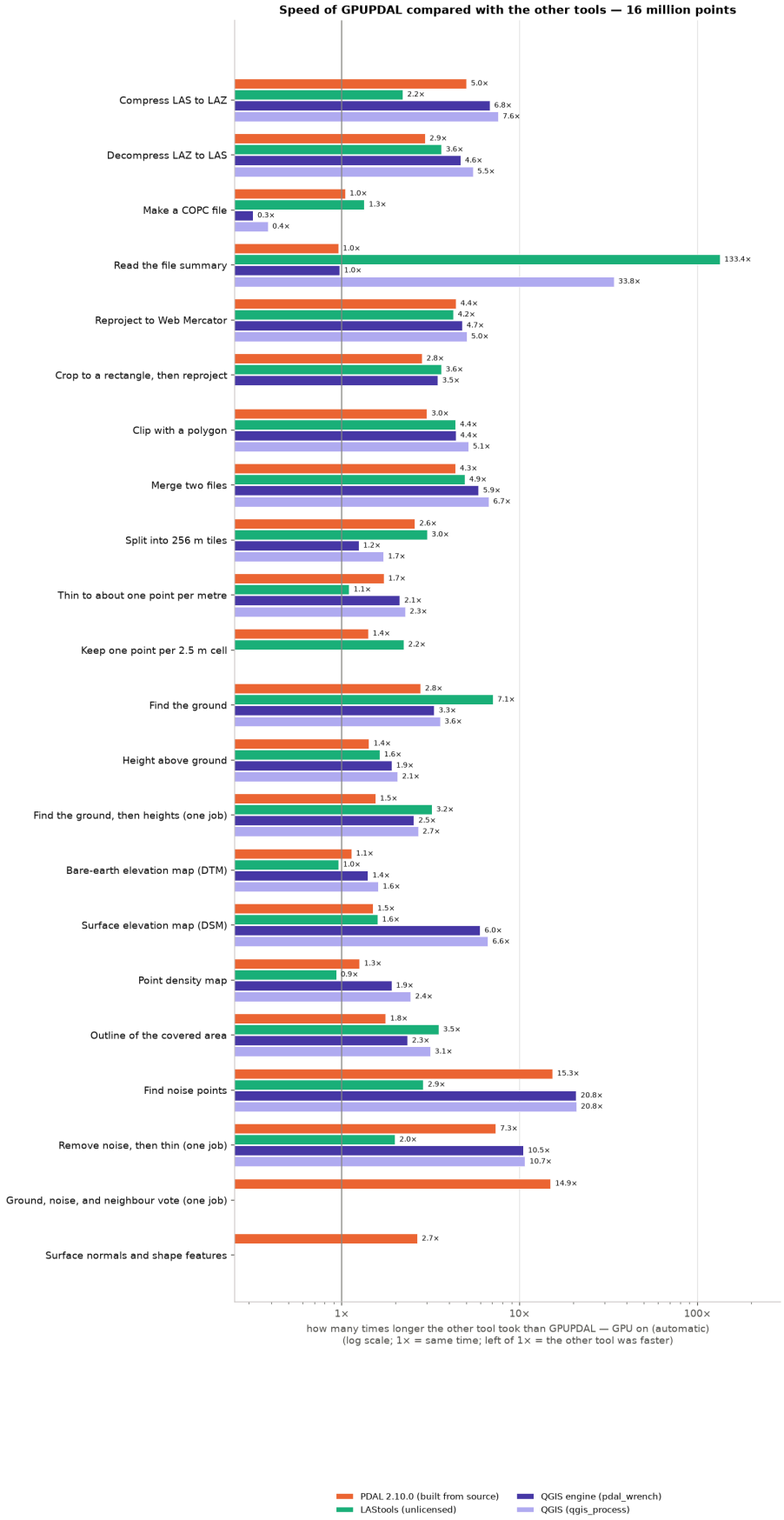
Rasters (maps): median time in seconds, 16 million points, workstation.



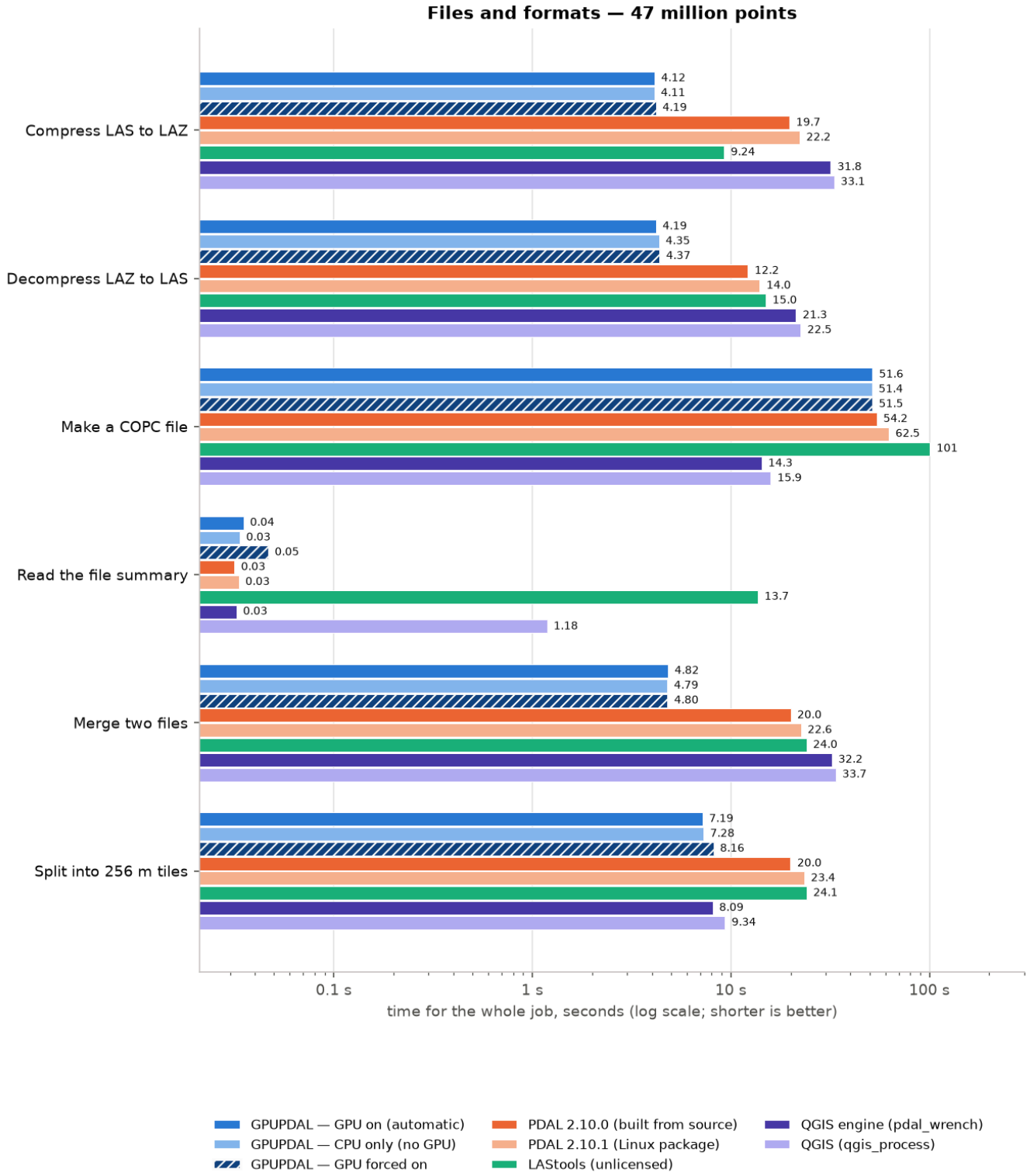
Cleaning: median time in seconds, 16 million points, workstation.



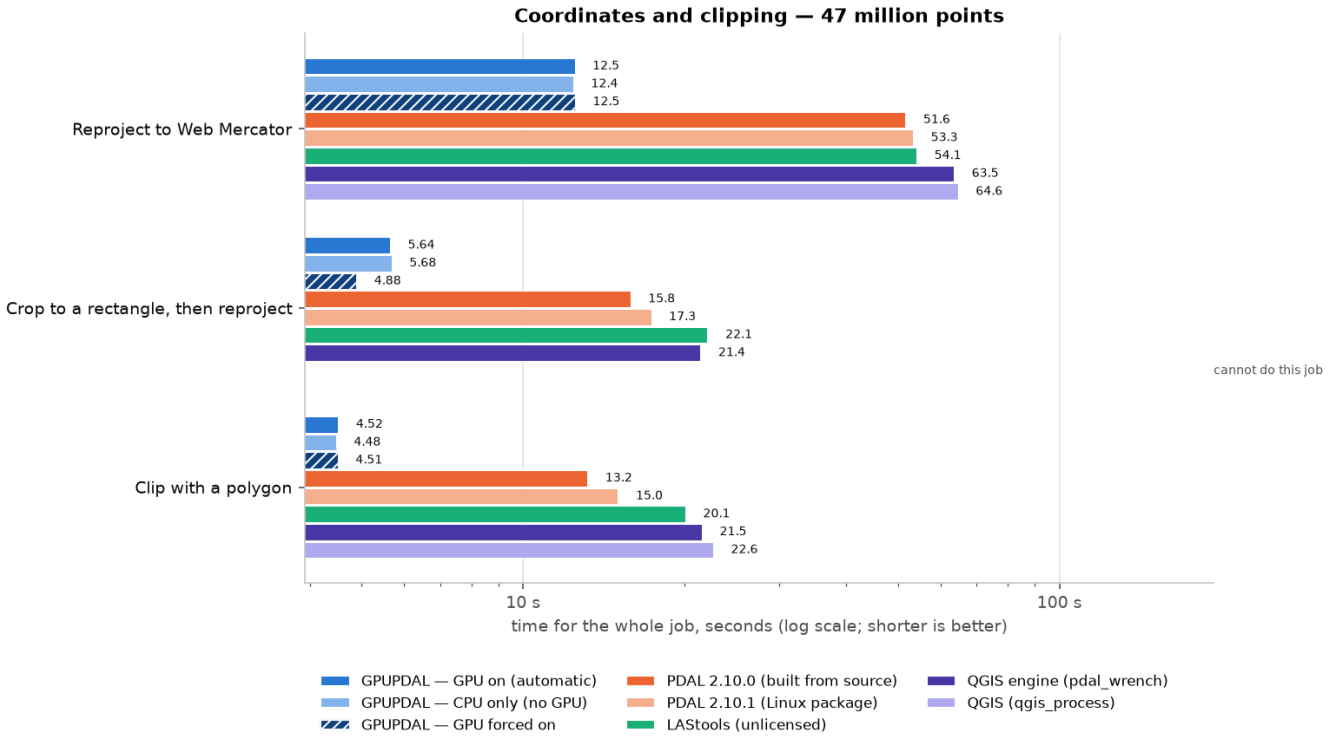
Neighbourhood analysis: median time in seconds, 16 million points, workstation.



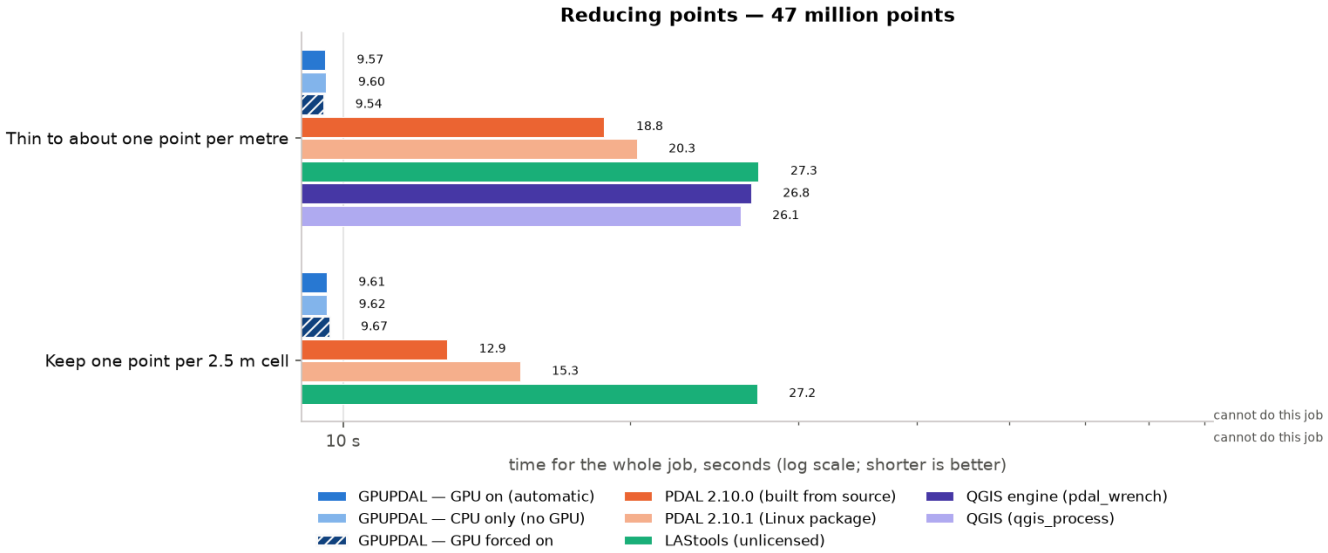
Other tool's time ÷ GPUPDAL's time, 16 million points, workstation.



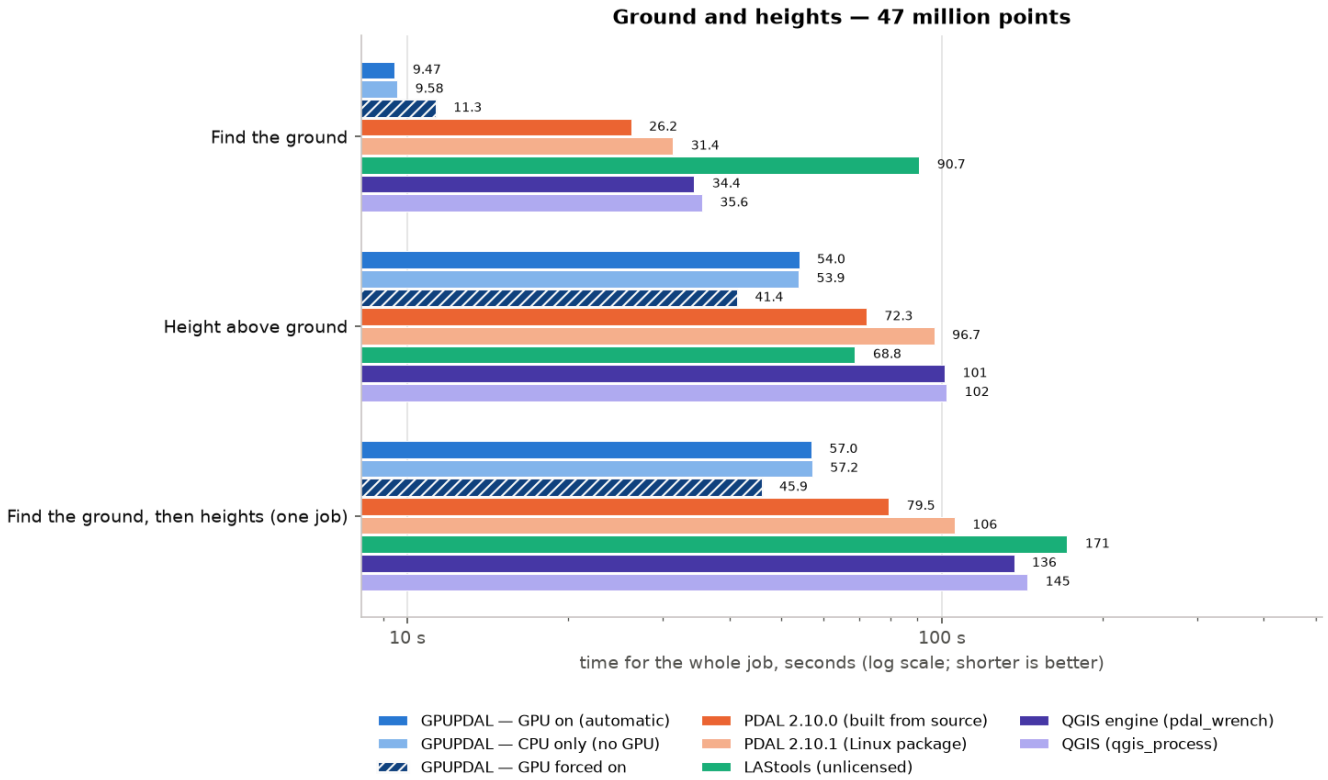
Files and formats: median time in seconds, 47 million points, workstation.



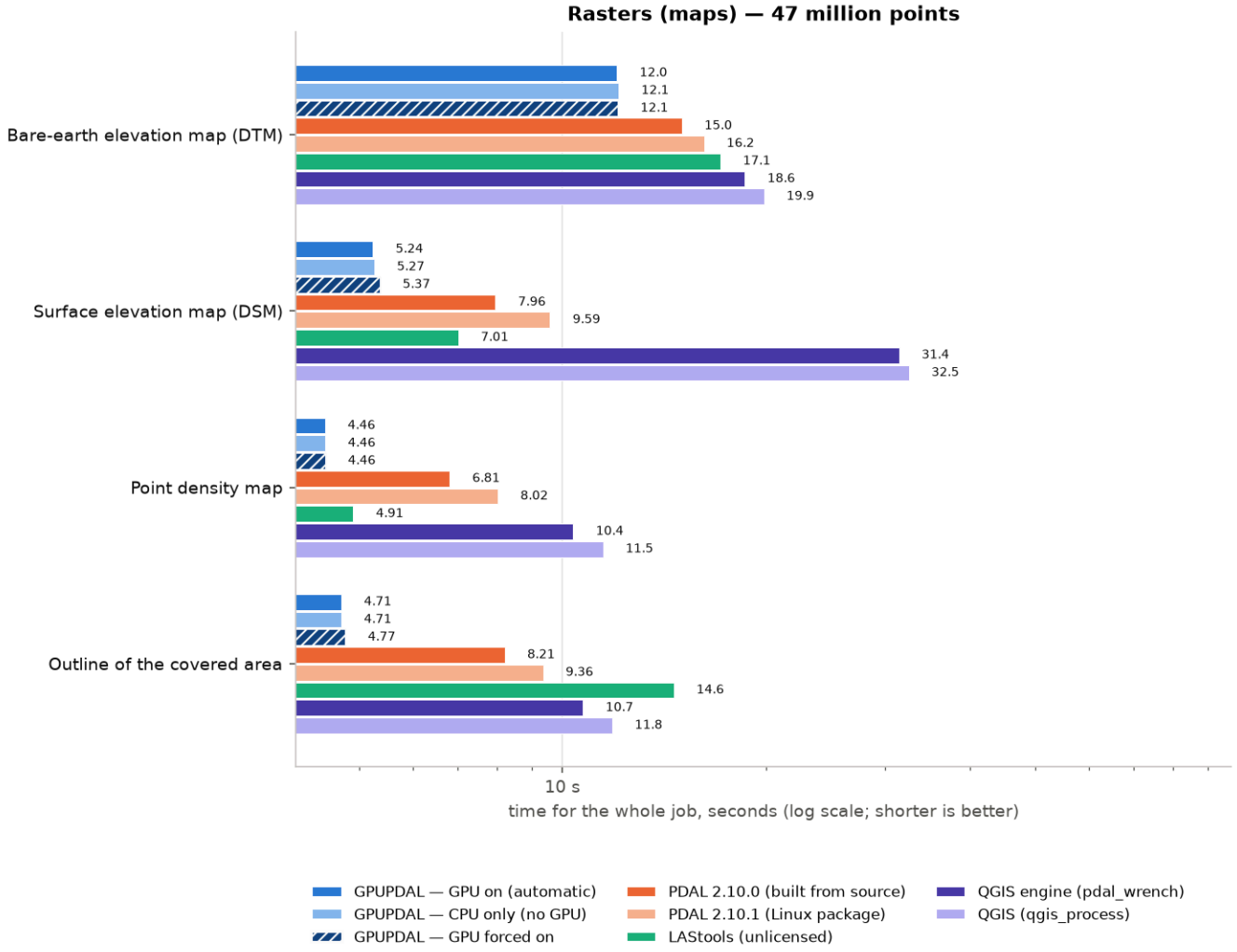
Coordinates and clipping: median time in seconds, 47 million points, workstation.



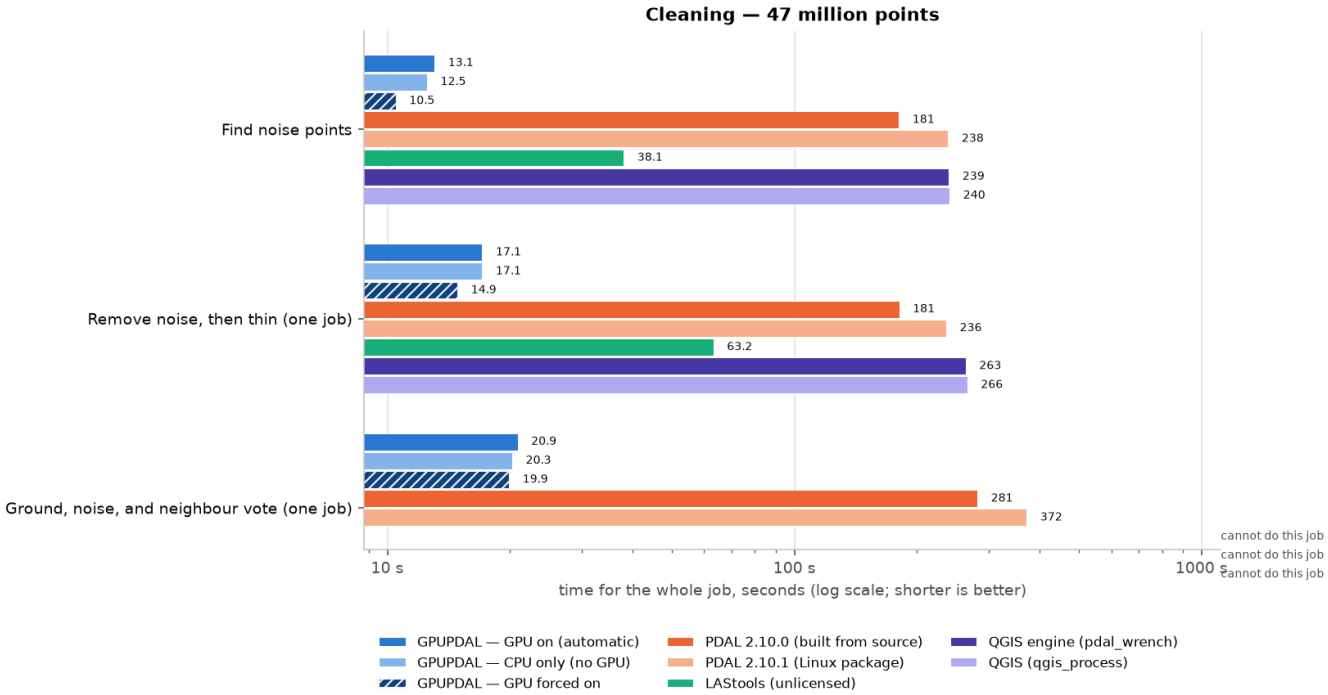
Reducing points: median time in seconds, 47 million points, workstation.



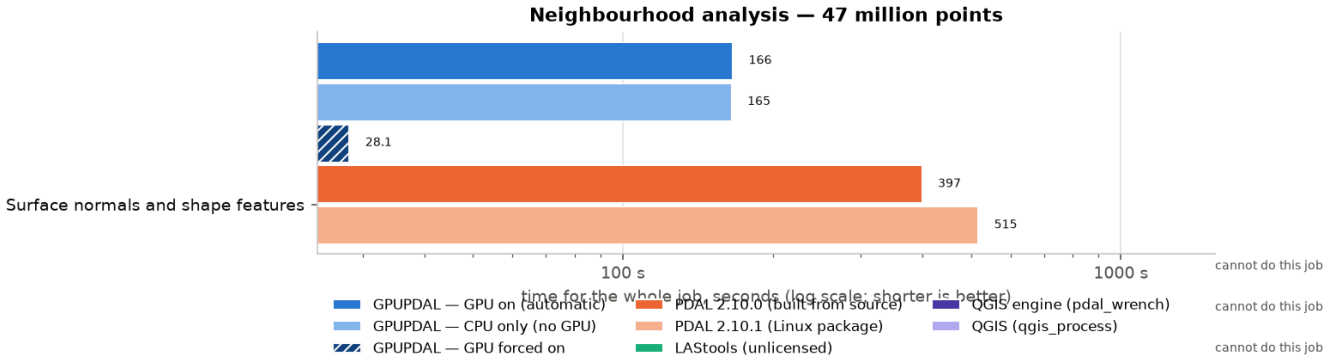
Ground and heights: median time in seconds, 47 million points, workstation.



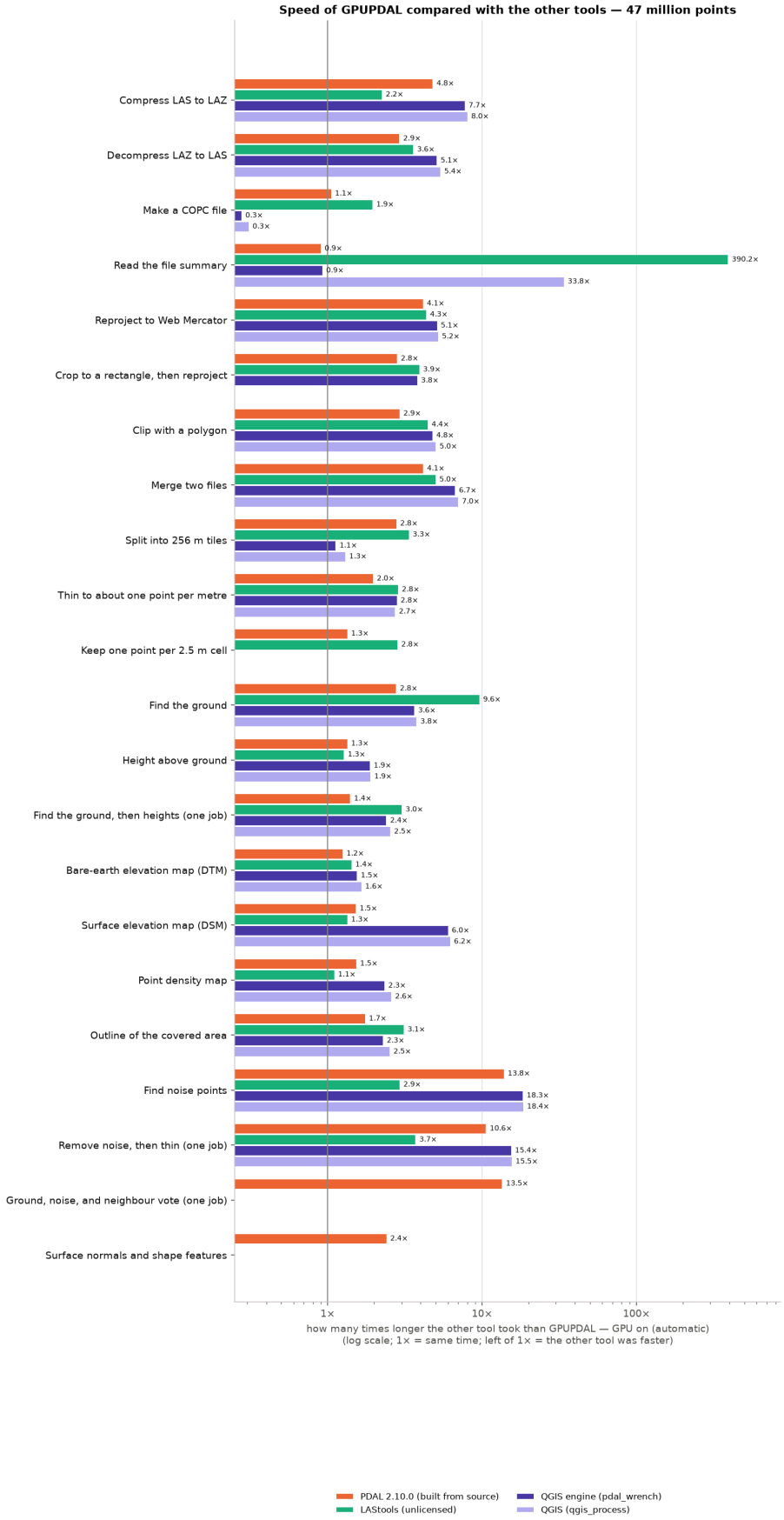
Rasters (maps): median time in seconds, 47 million points, workstation.



Cleaning: median time in seconds, 47 million points, workstation.

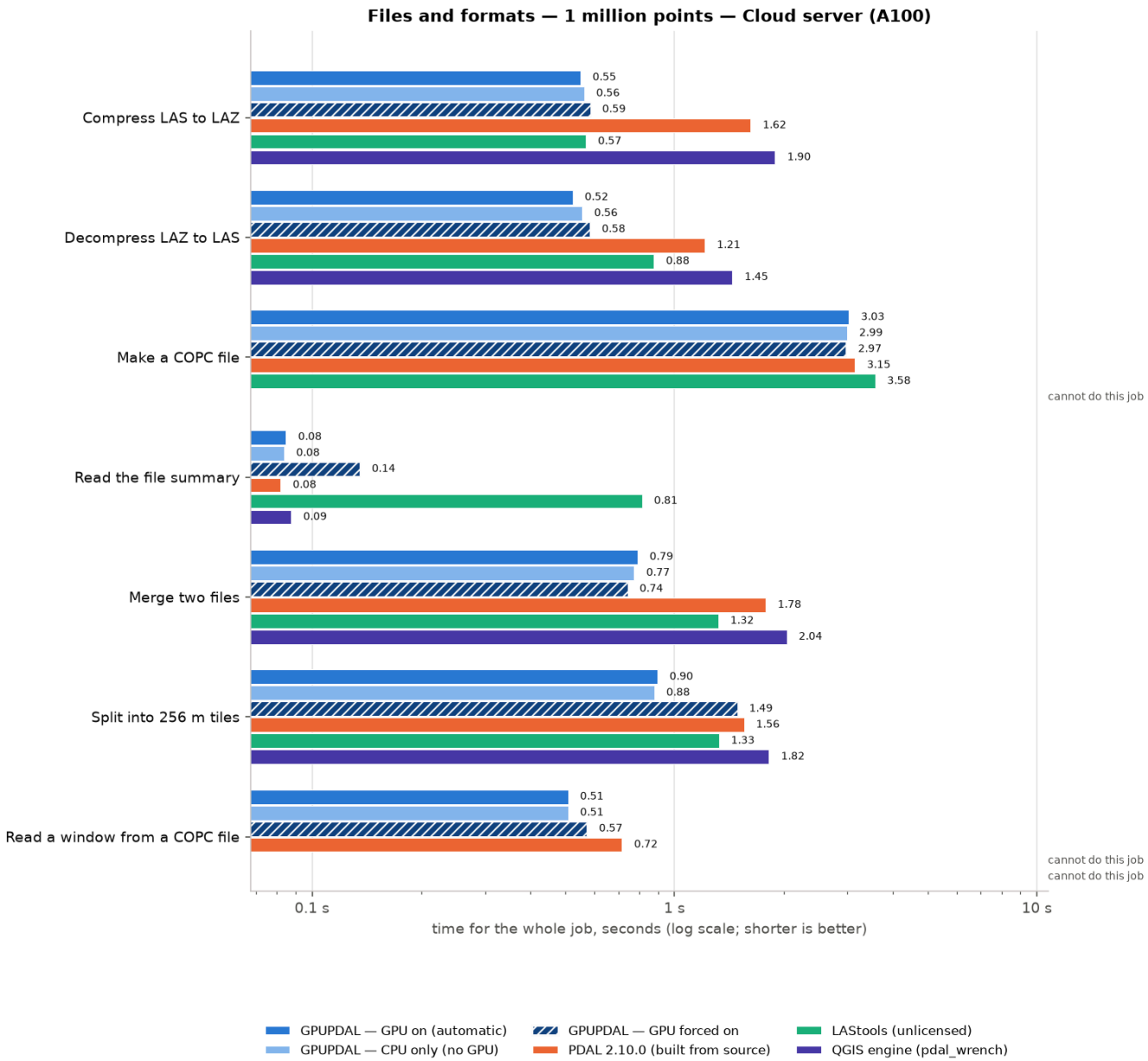


Neighbourhood analysis: median time in seconds, 47 million points, workstation.

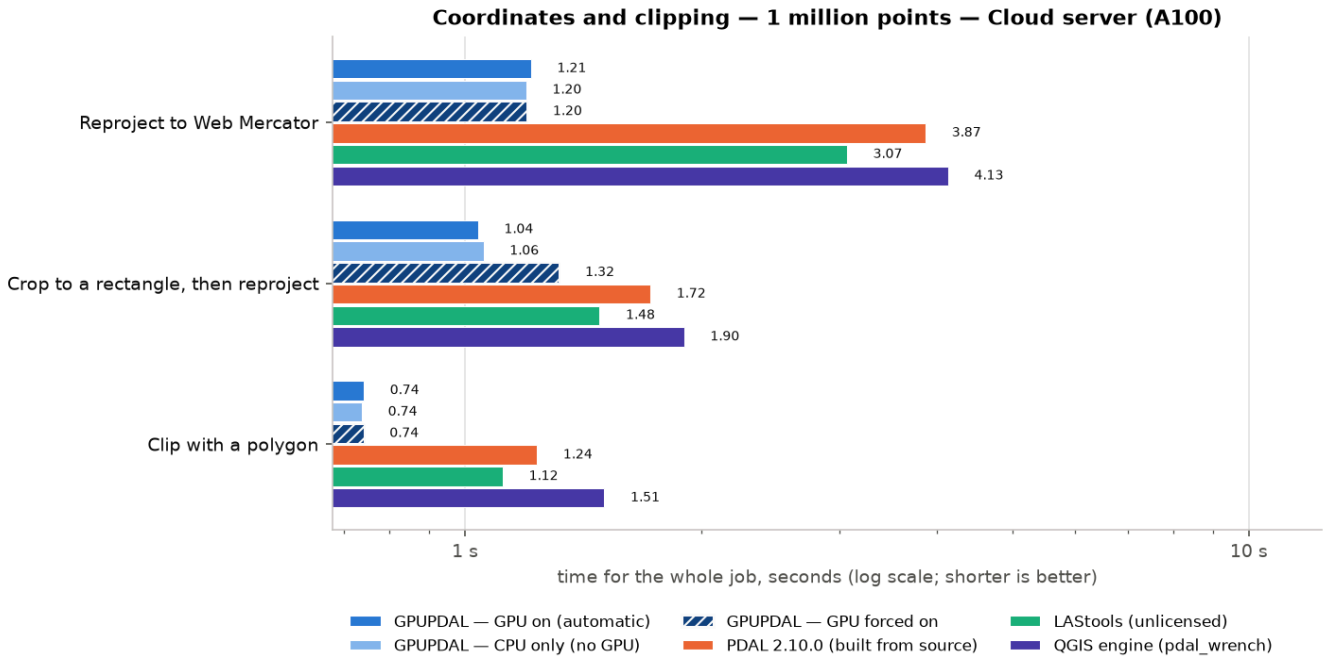


Other tool's time ÷ GPUPDAL's time, 47 million points, workstation.

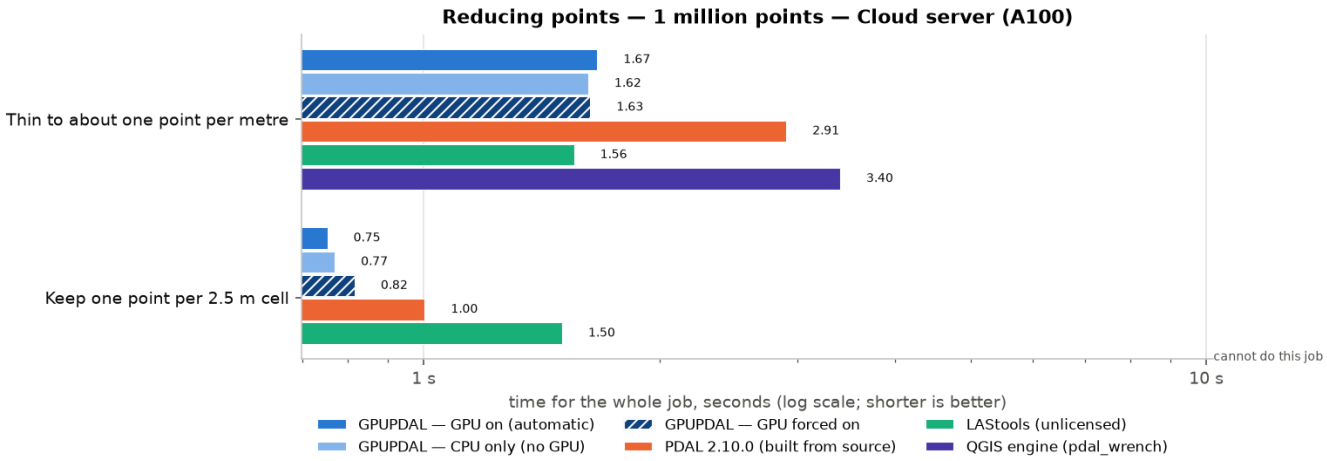
9.2 Every chart for Cloud server (EPYC 7532 + A100 40 GB)



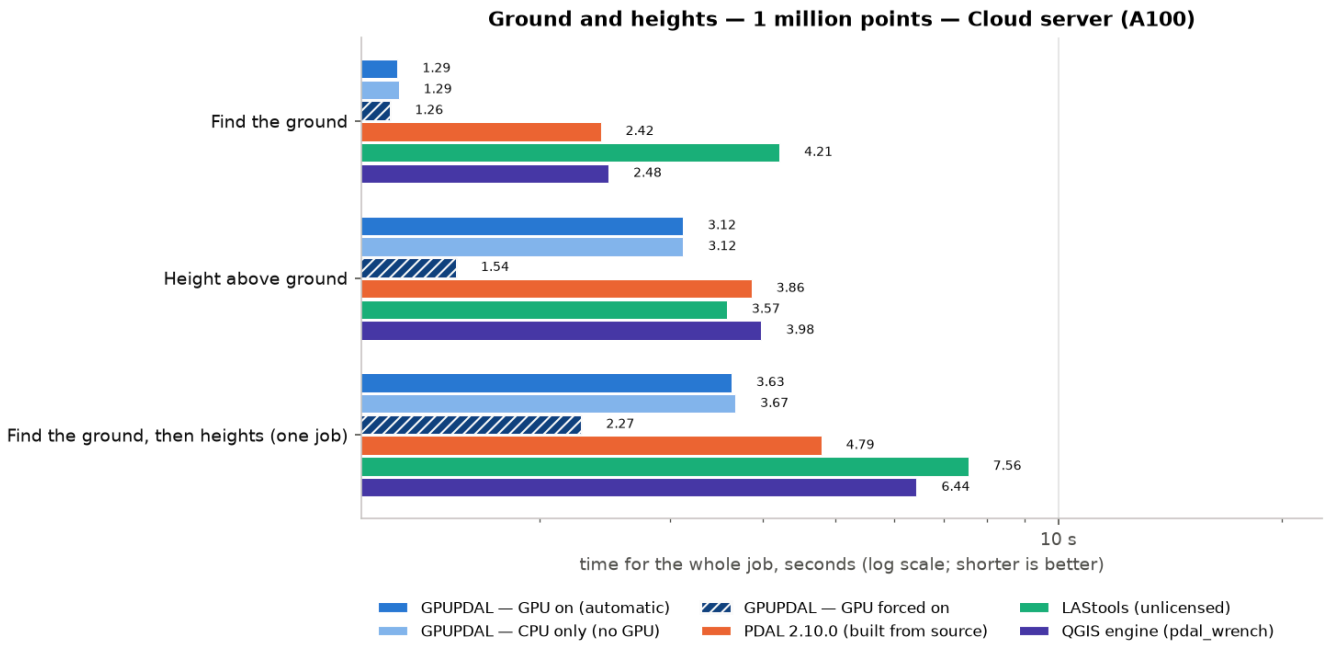
Files and formats: median time in seconds, 1 million points, Cloud server (A100).



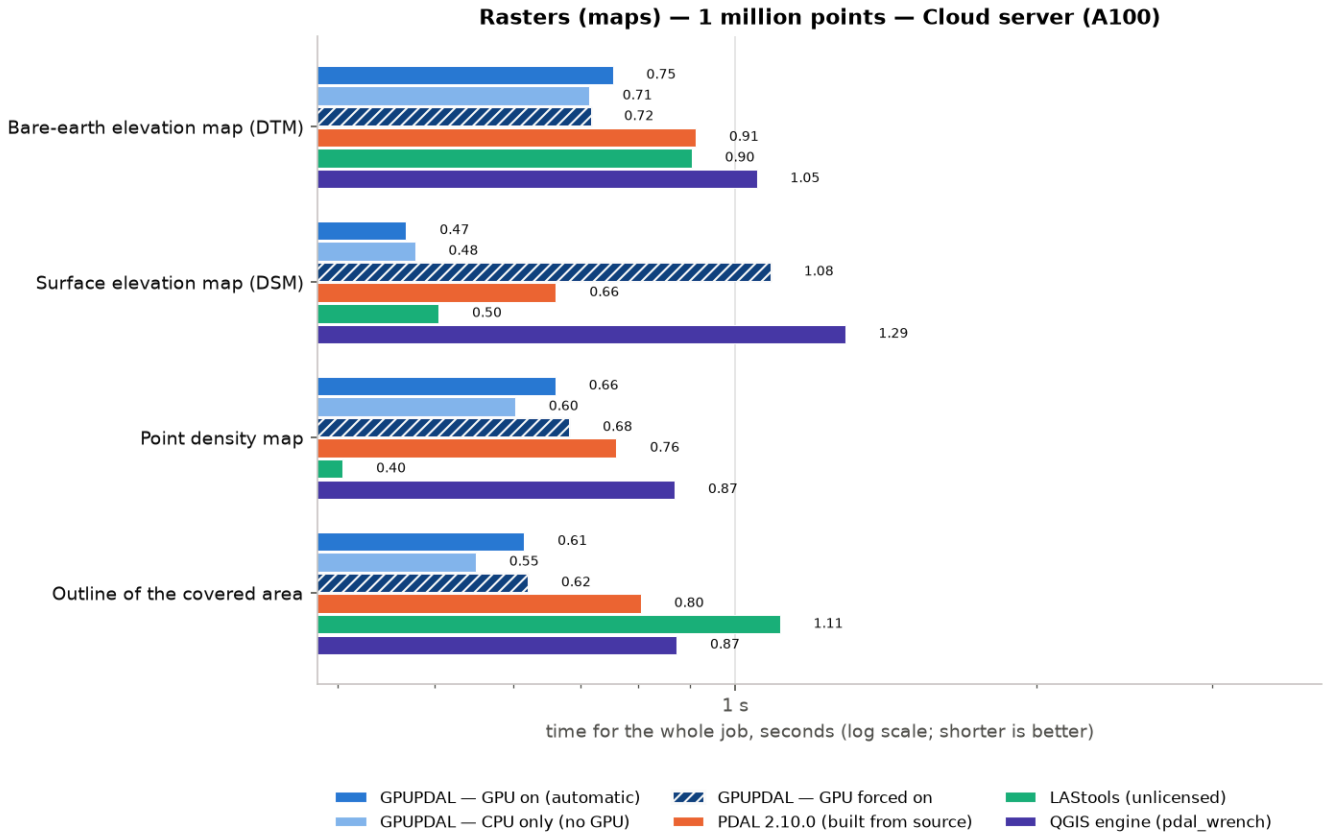
Coordinates and clipping: median time in seconds, 1 million points, Cloud server (A100).



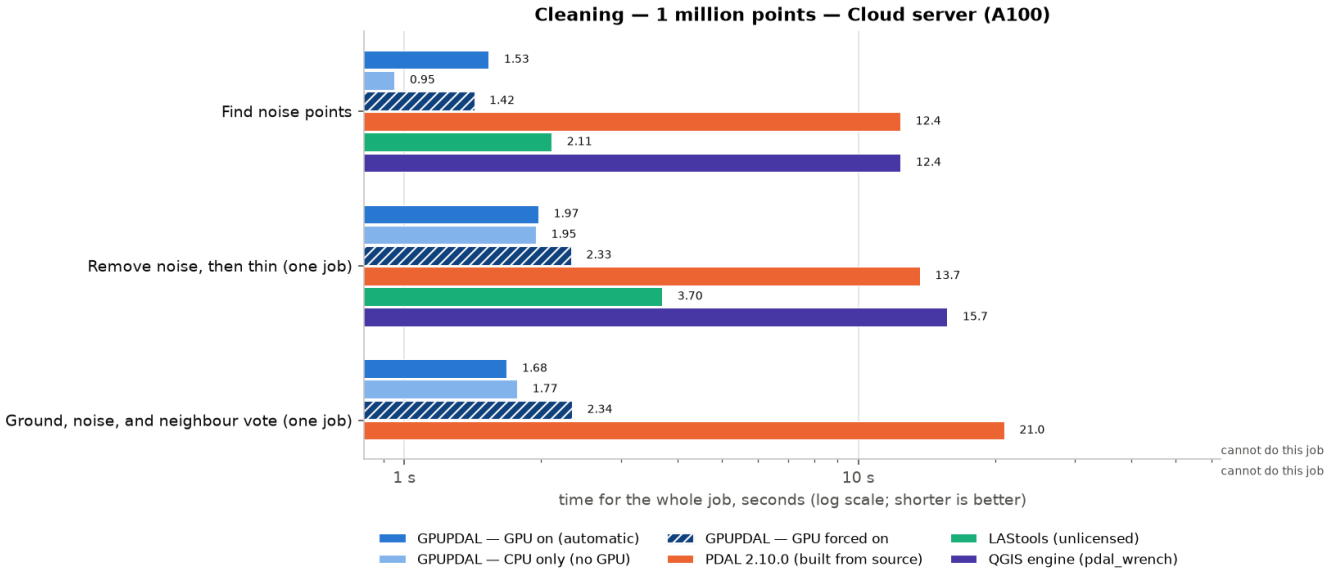
Reducing points: median time in seconds, 1 million points, Cloud server (A100).



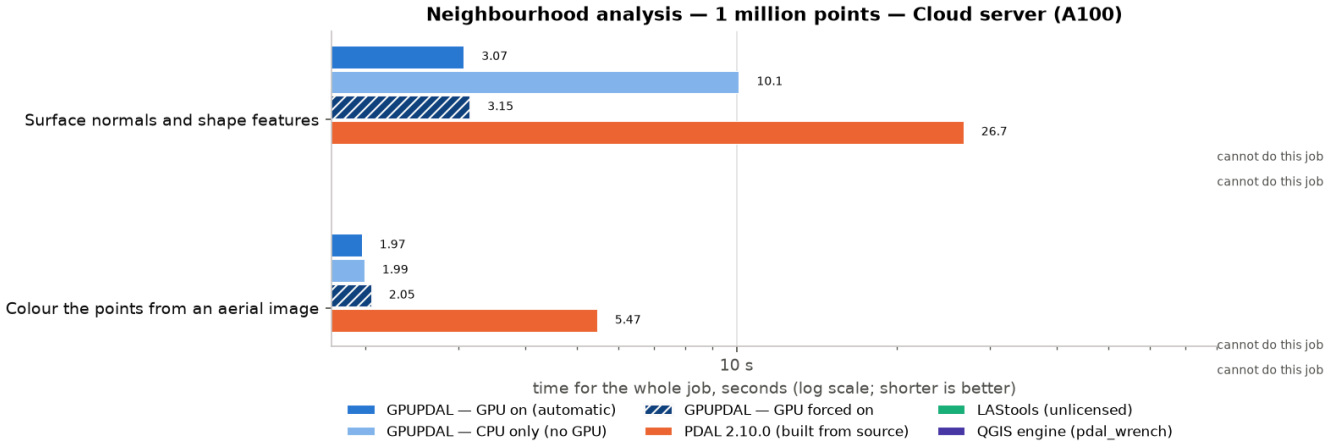
Ground and heights: median time in seconds, 1 million points, Cloud server (A100).



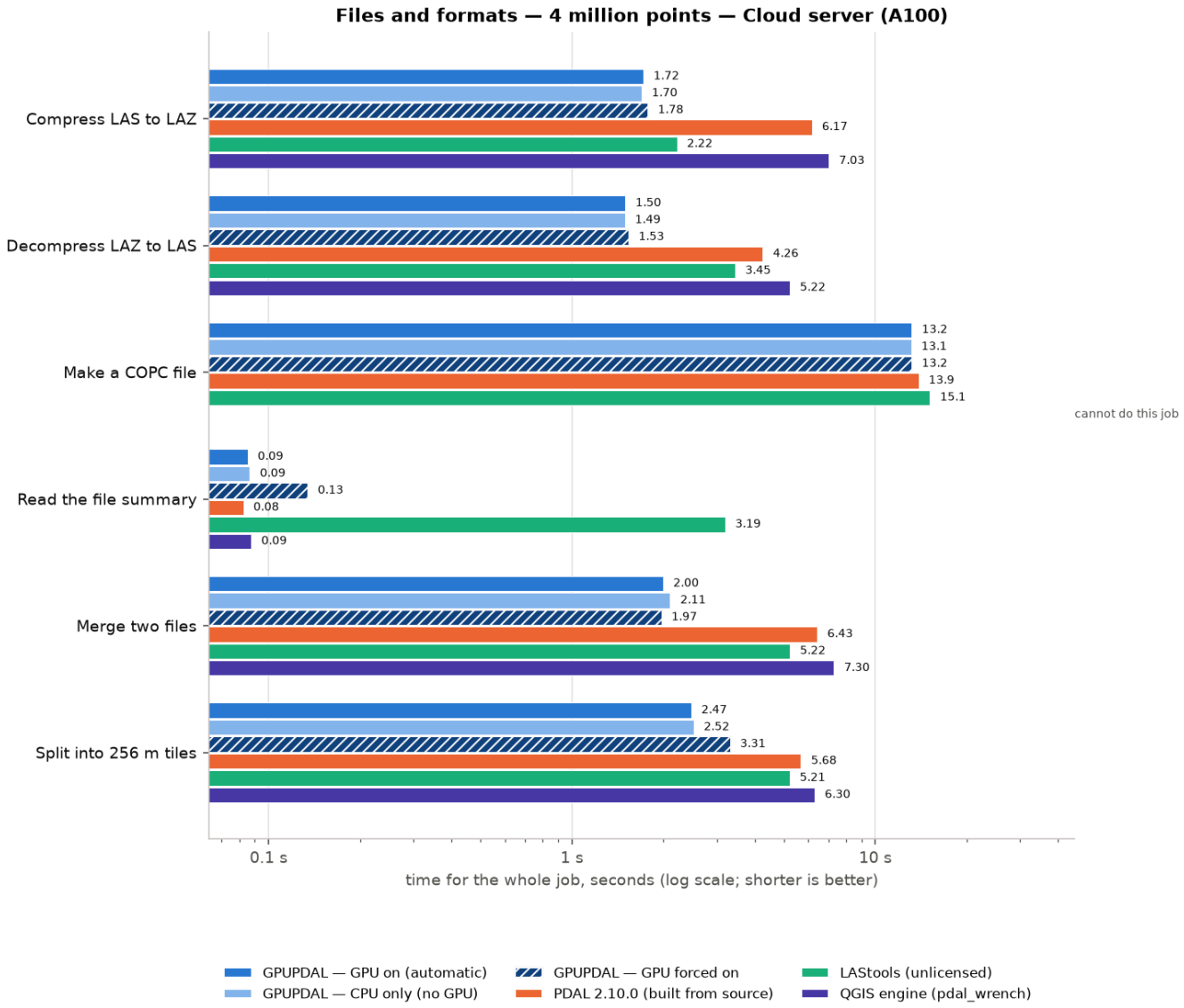
Rasters (maps): median time in seconds, 1 million points, Cloud server (A100).



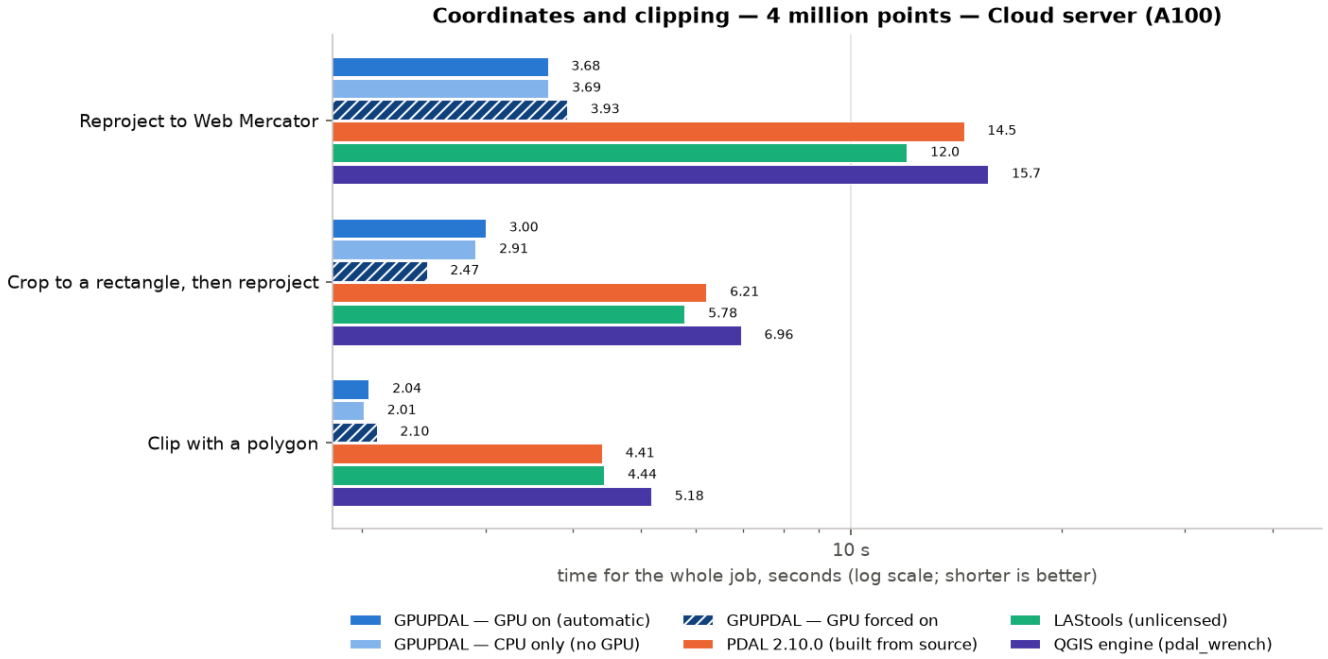
Cleaning: median time in seconds, 1 million points, Cloud server (A100).



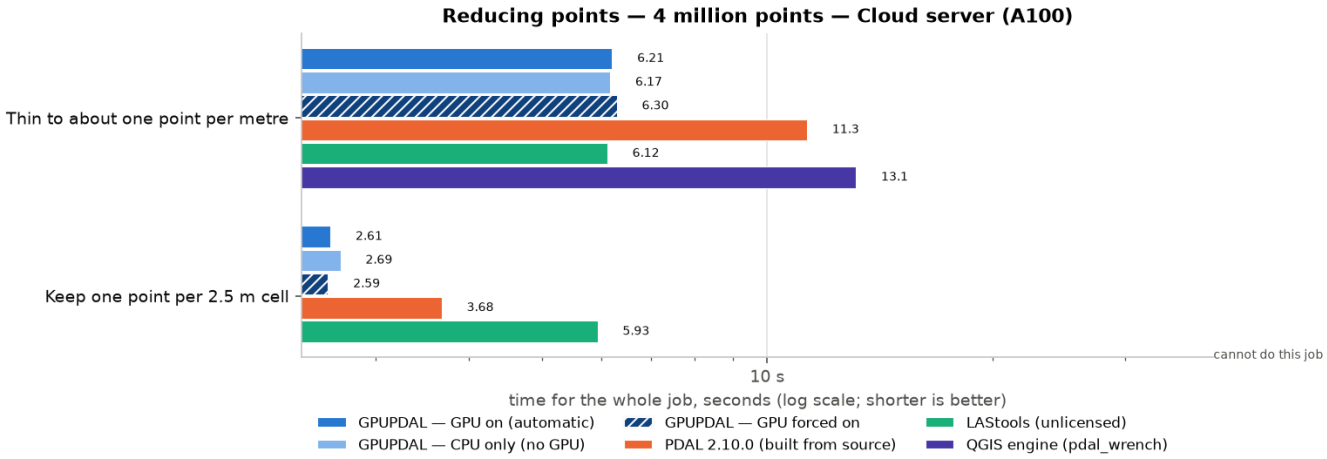
Neighbourhood analysis: median time in seconds, 1 million points, Cloud server (A100).



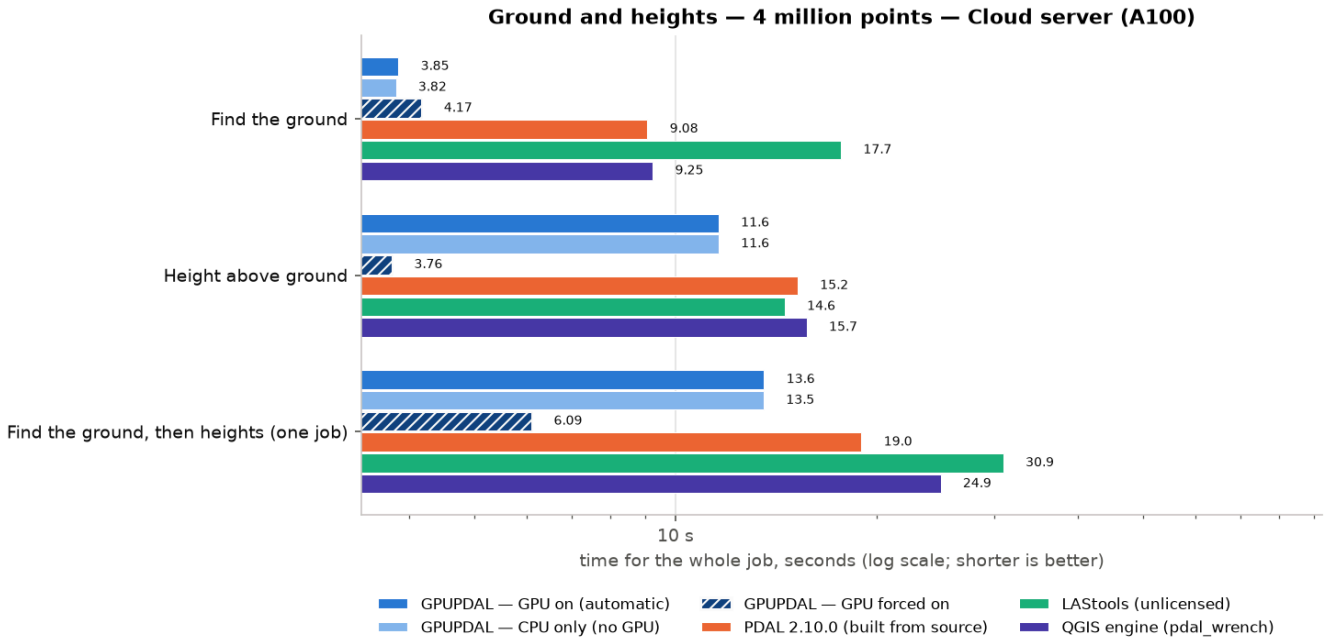
Files and formats: median time in seconds, 4 million points, Cloud server (A100).



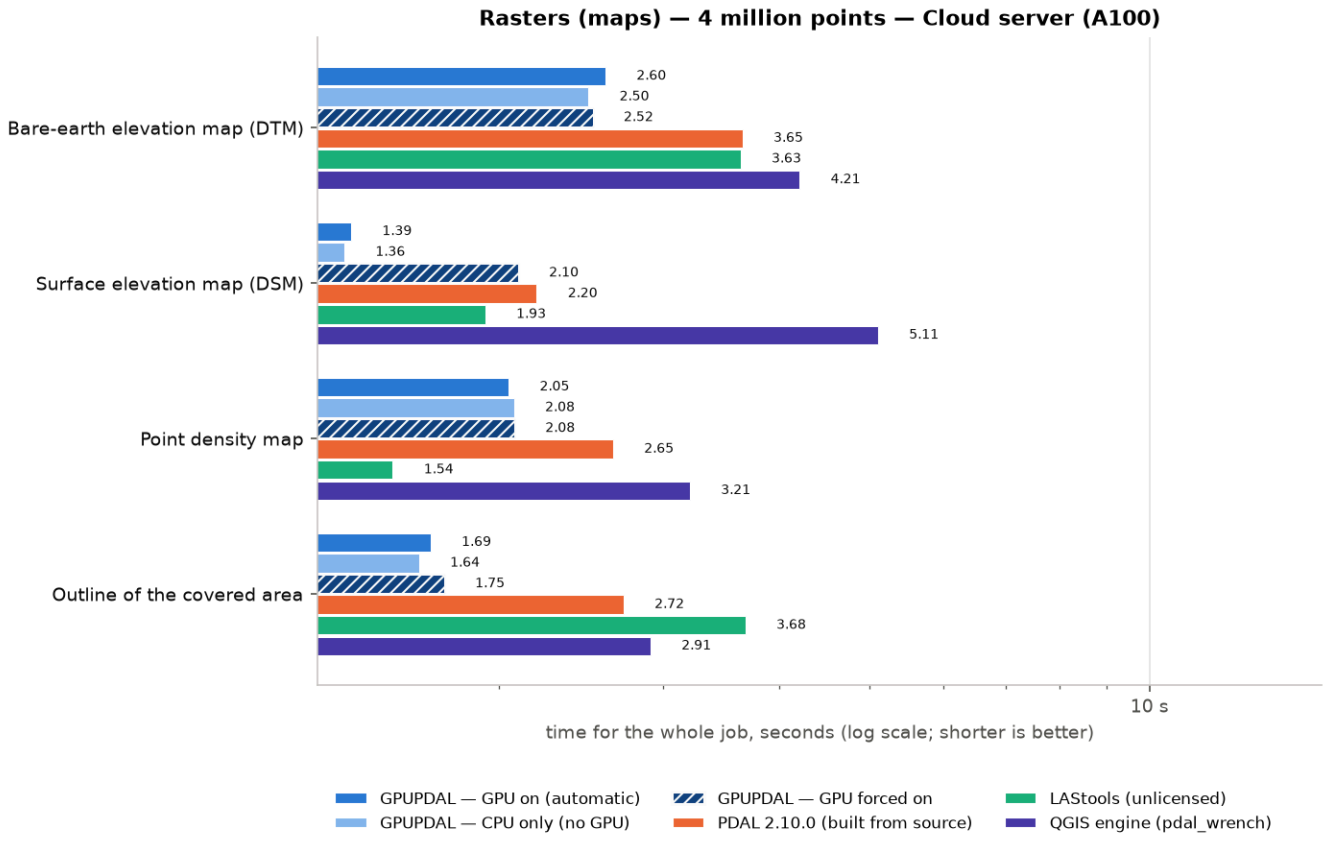
Coordinates and clipping: median time in seconds, 4 million points, Cloud server (A100).



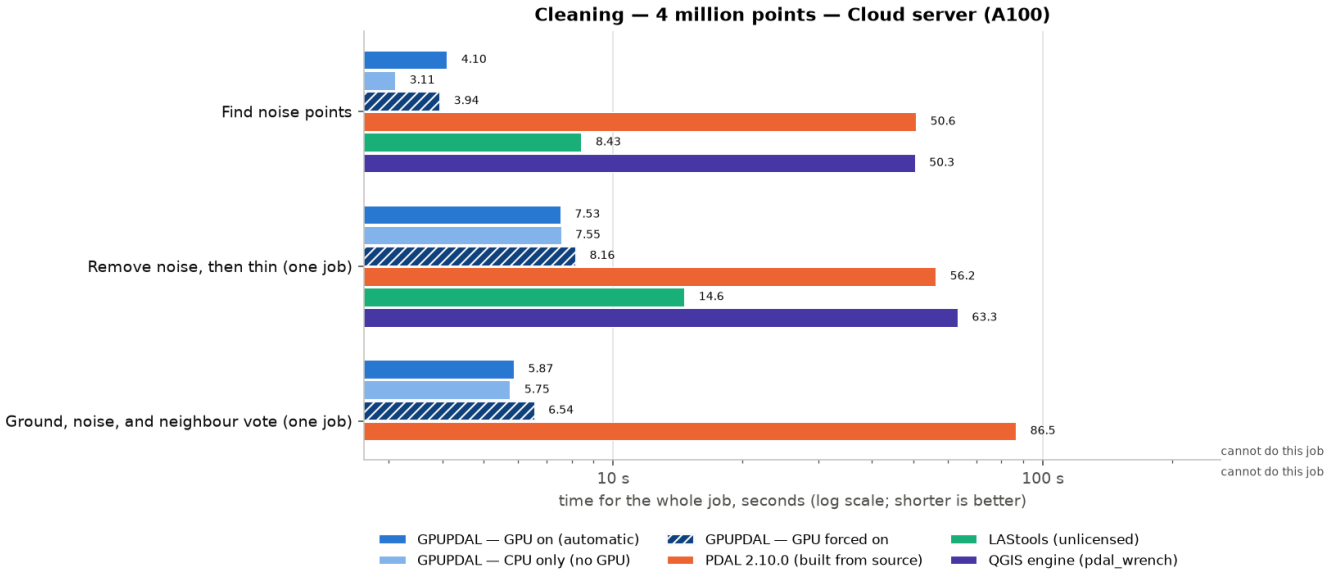
Reducing points: median time in seconds, 4 million points, Cloud server (A100).



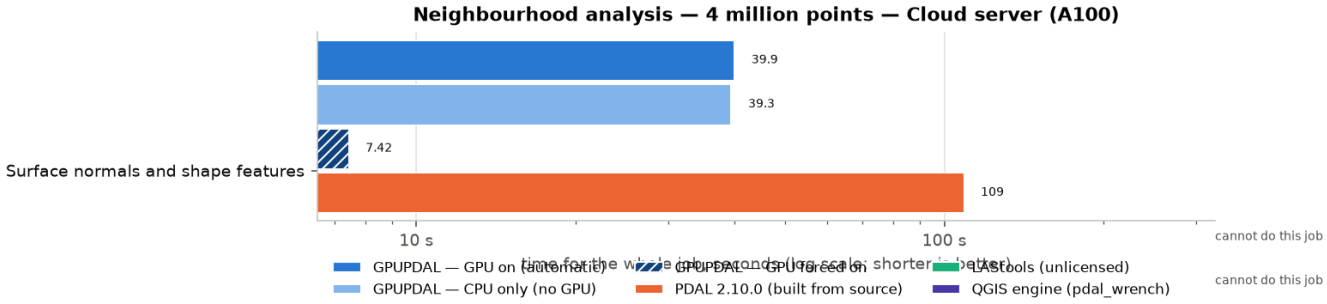
Ground and heights: median time in seconds, 4 million points, Cloud server (A100).



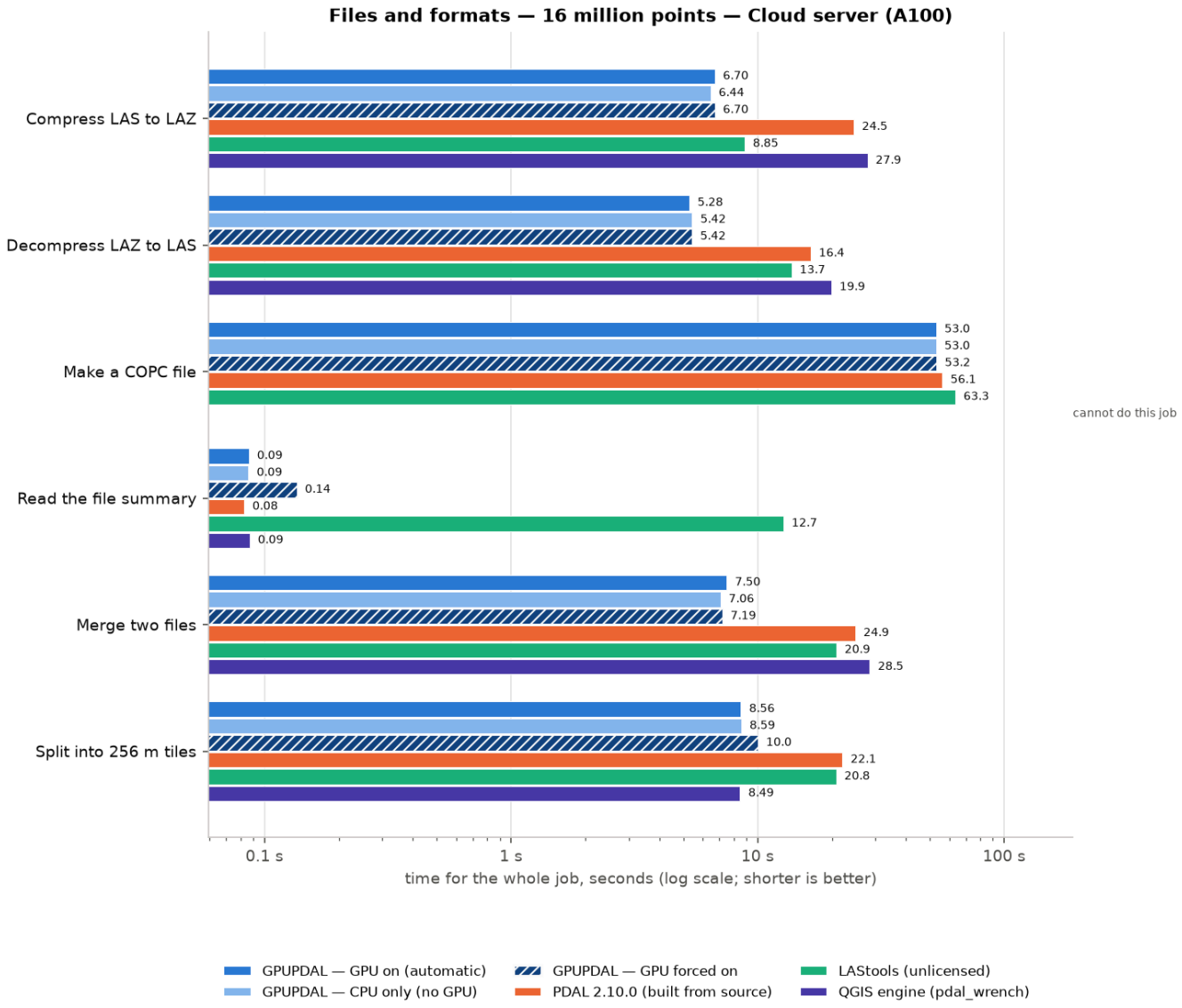
Rasters (maps): median time in seconds, 4 million points, Cloud server (A100).



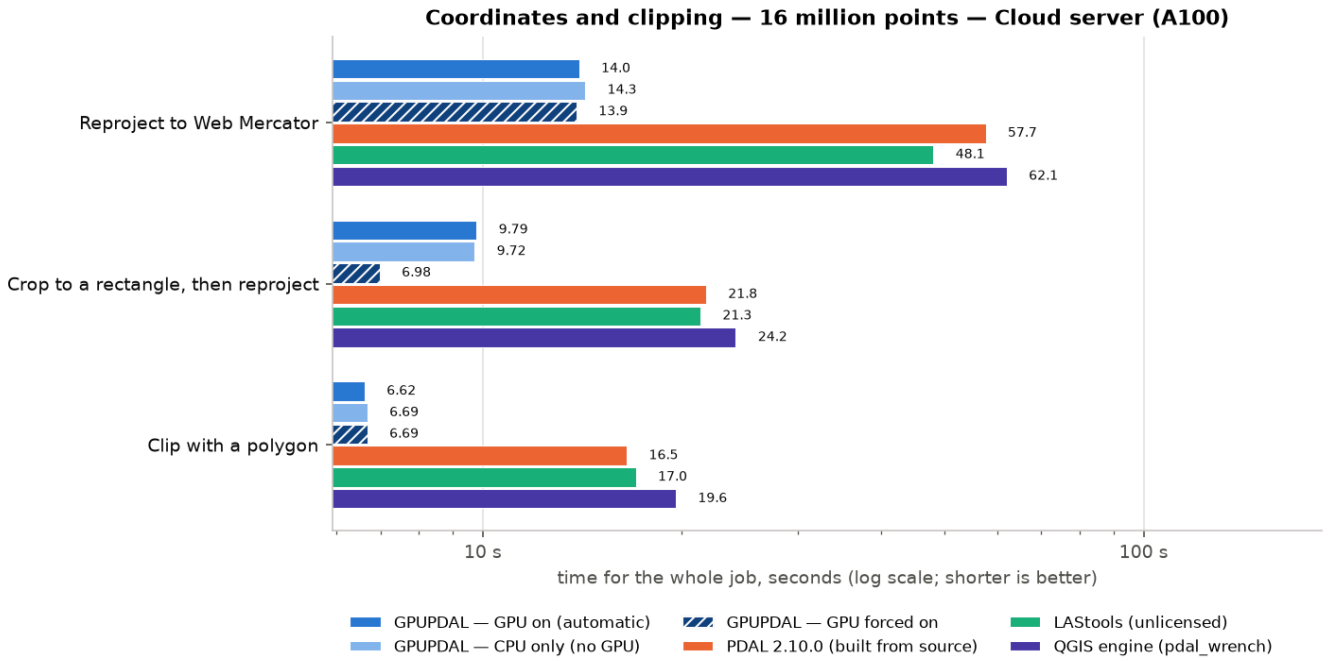
Cleaning: median time in seconds, 4 million points, Cloud server (A100).



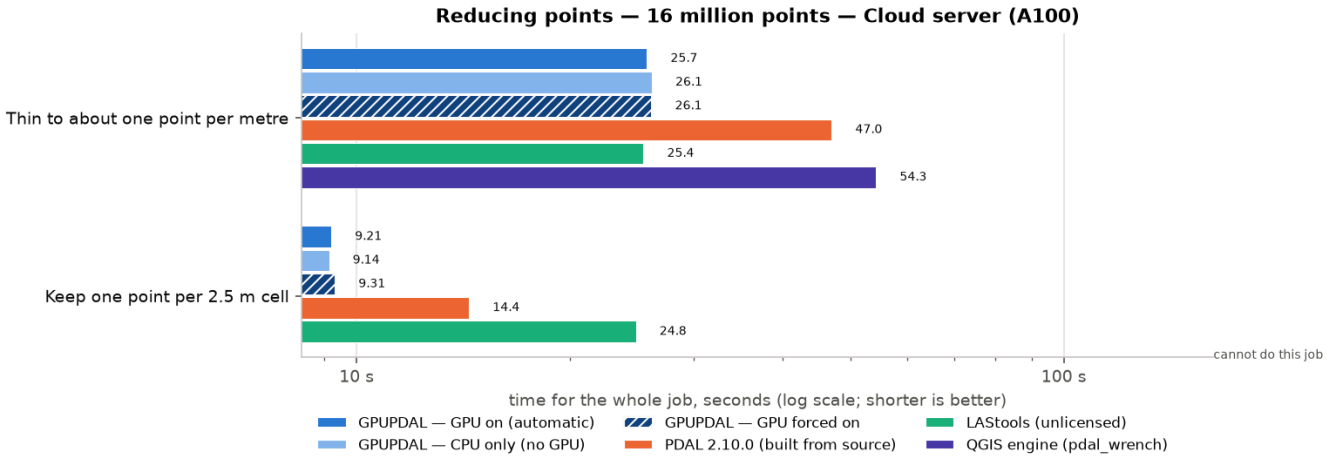
Neighbourhood analysis: median time in seconds, 4 million points, Cloud server (A100).



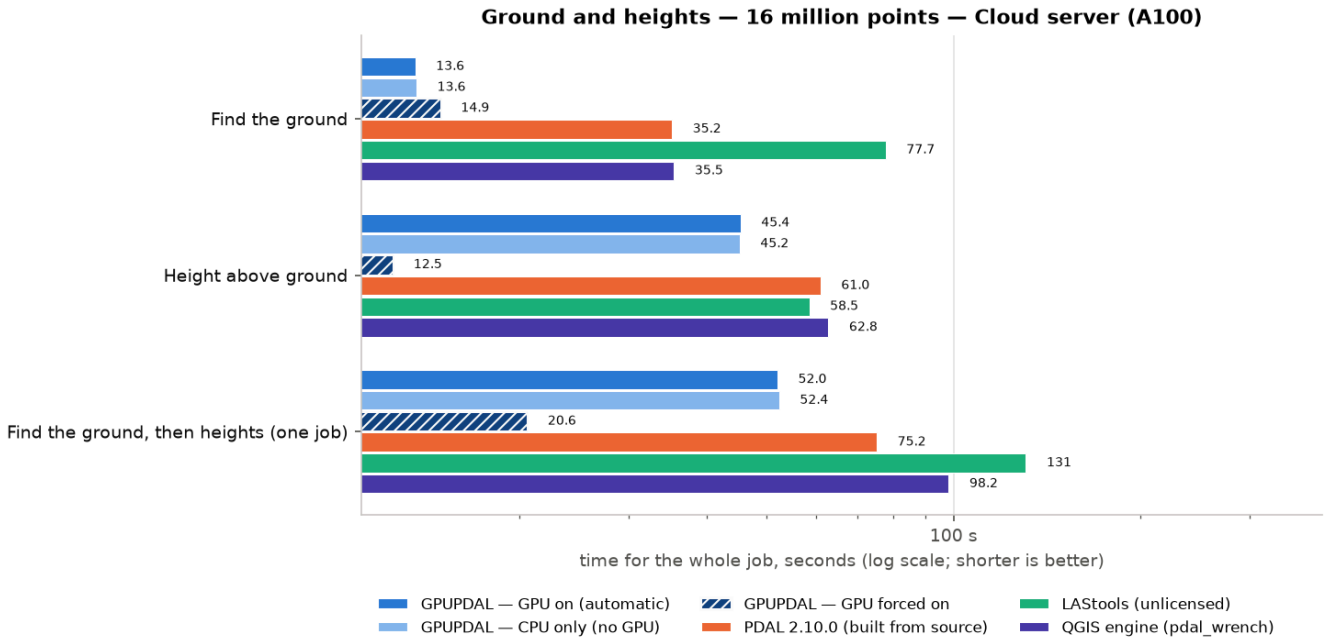
Files and formats: median time in seconds, 16 million points, Cloud server (A100).



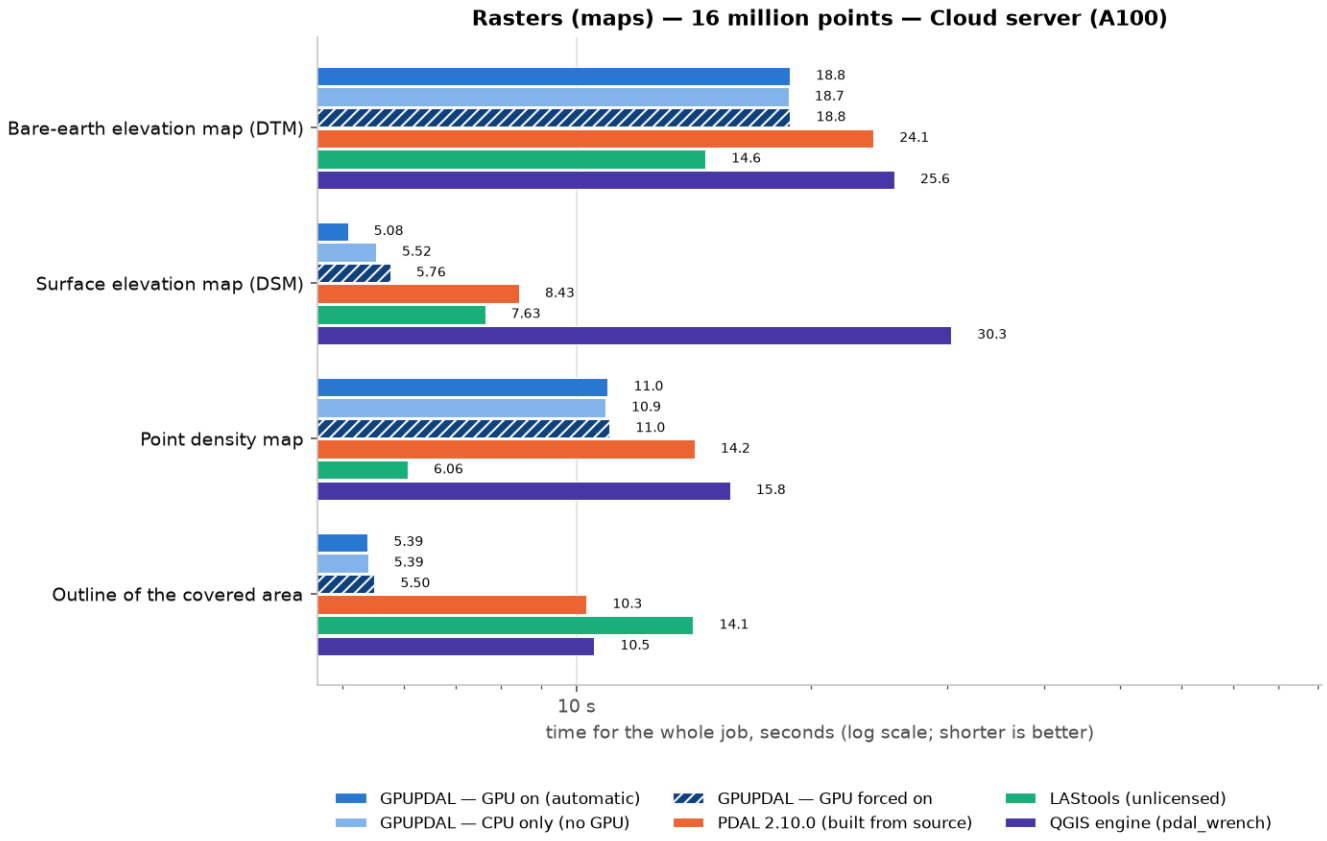
Coordinates and clipping: median time in seconds, 16 million points, Cloud server (A100).



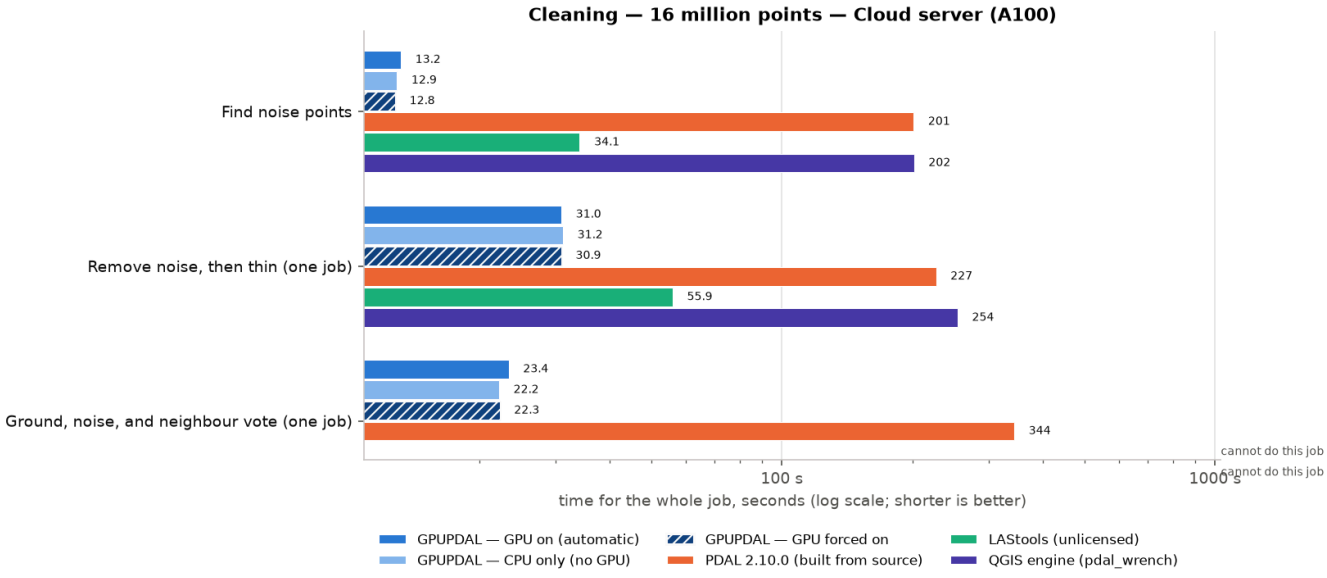
Reducing points: median time in seconds, 16 million points, Cloud server (A100).



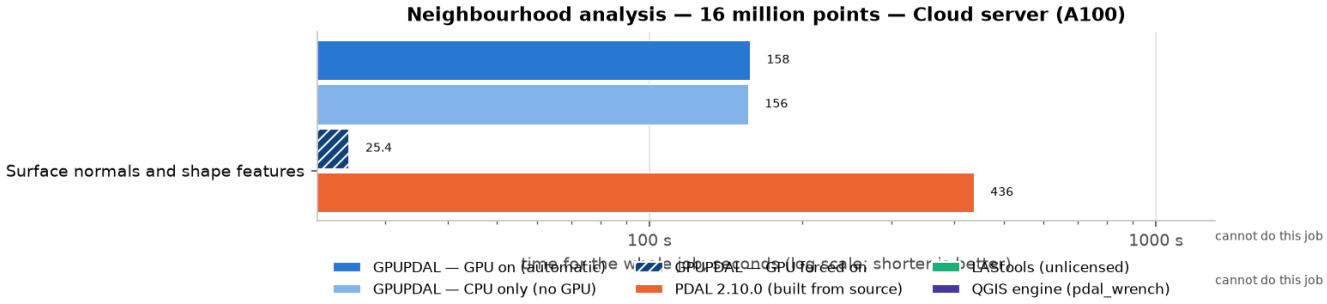
Ground and heights: median time in seconds, 16 million points, Cloud server (A100).



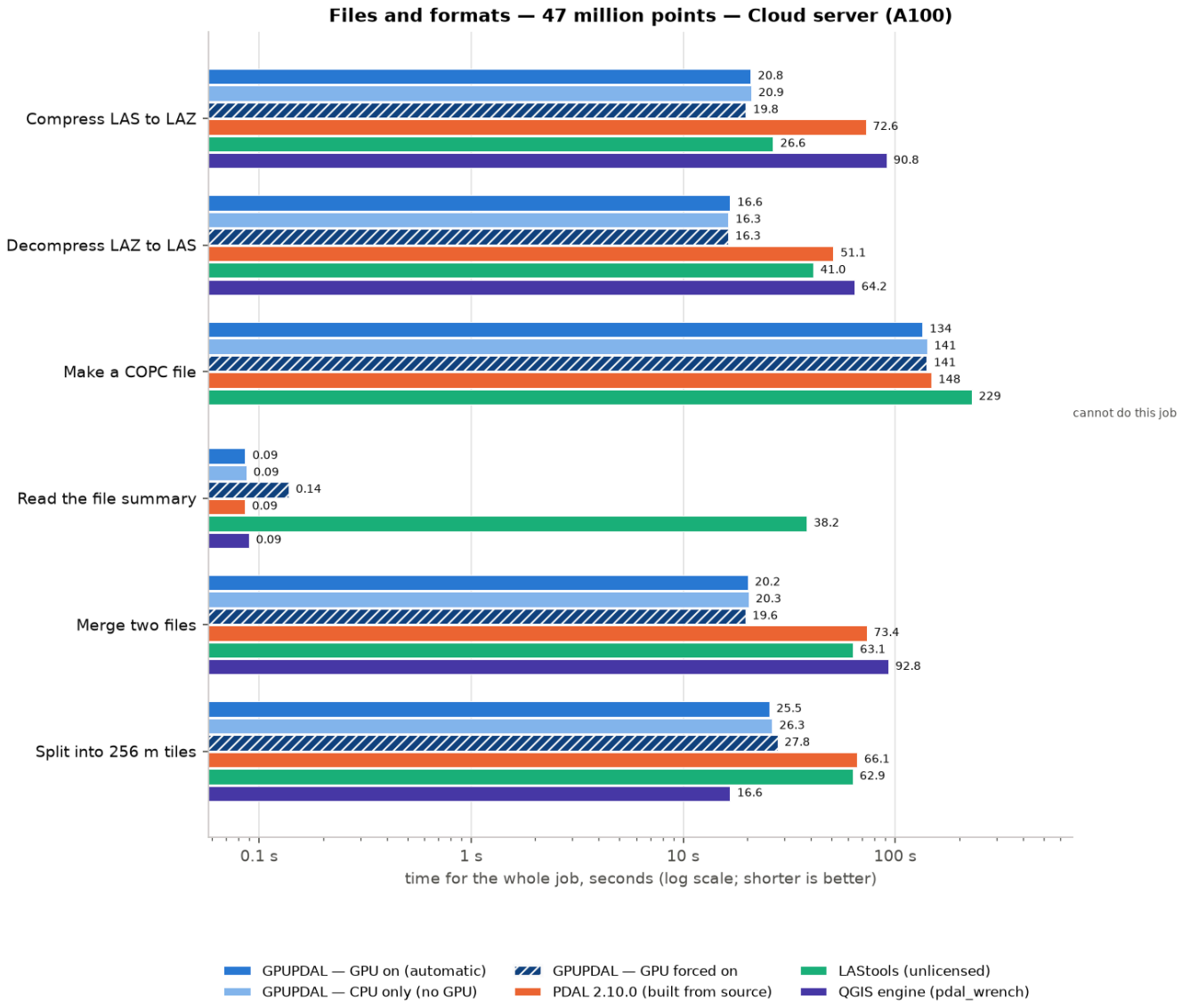
Rasters (maps): median time in seconds, 16 million points, Cloud server (A100).



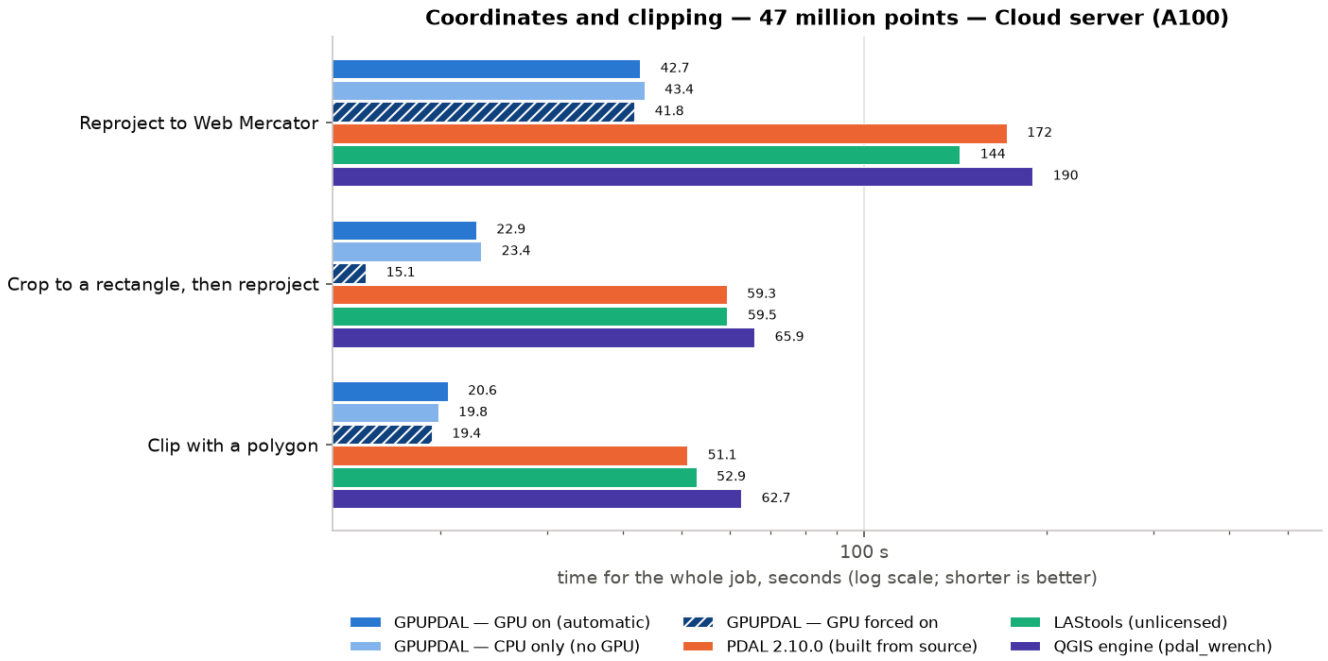
Cleaning: median time in seconds, 16 million points, Cloud server (A100).



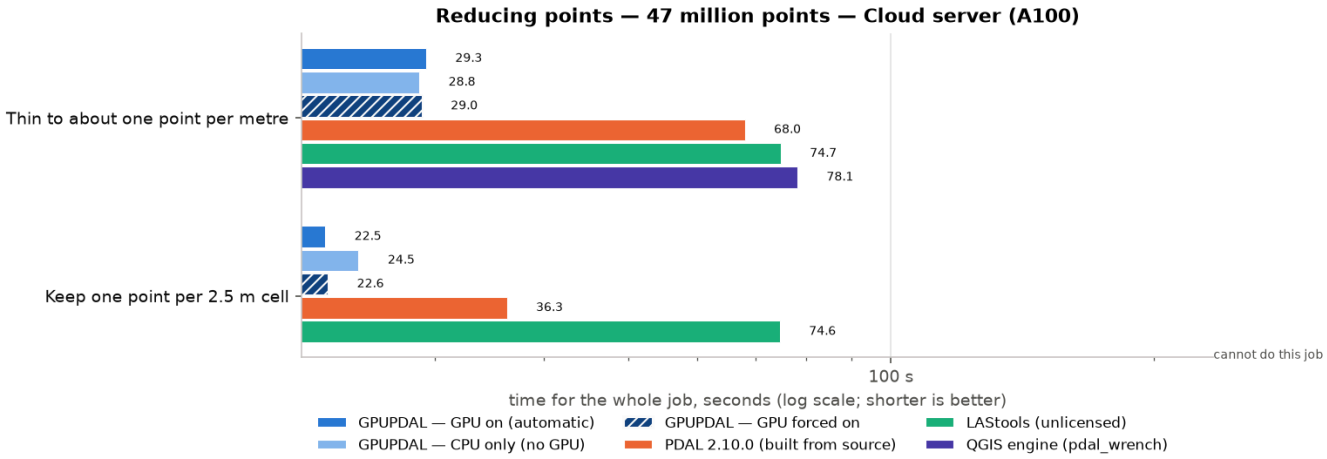
Neighbourhood analysis: median time in seconds, 16 million points, Cloud server (A100).



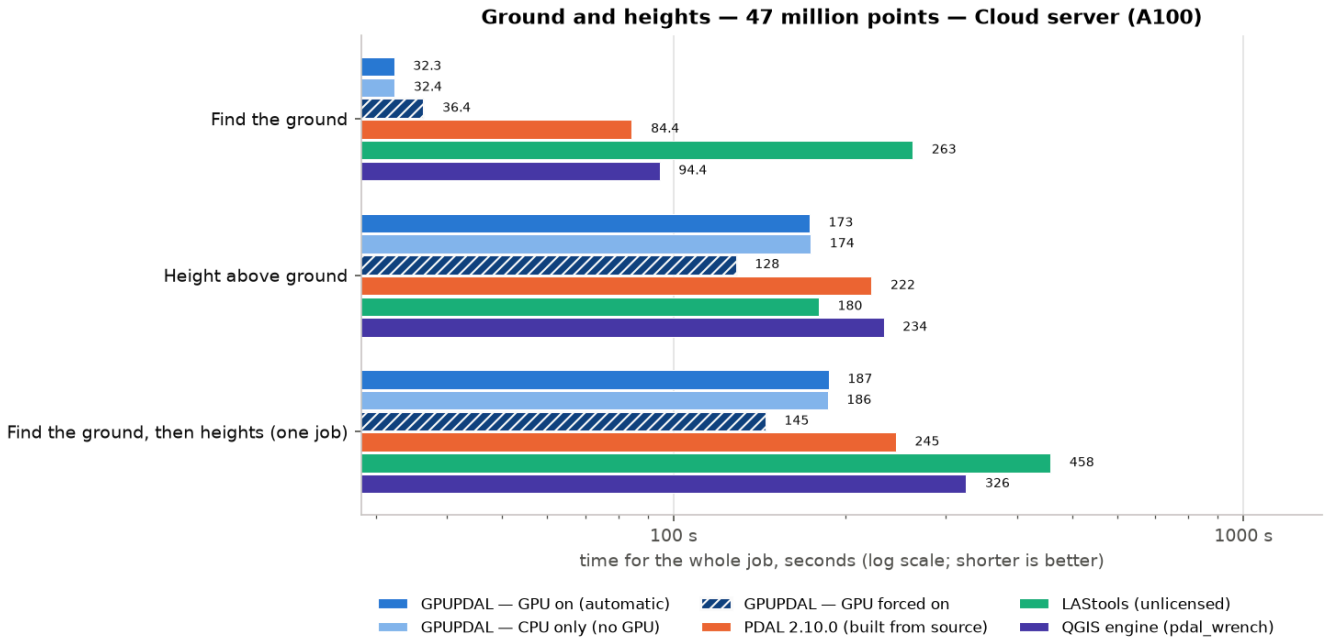
Files and formats: median time in seconds, 47 million points, Cloud server (A100).



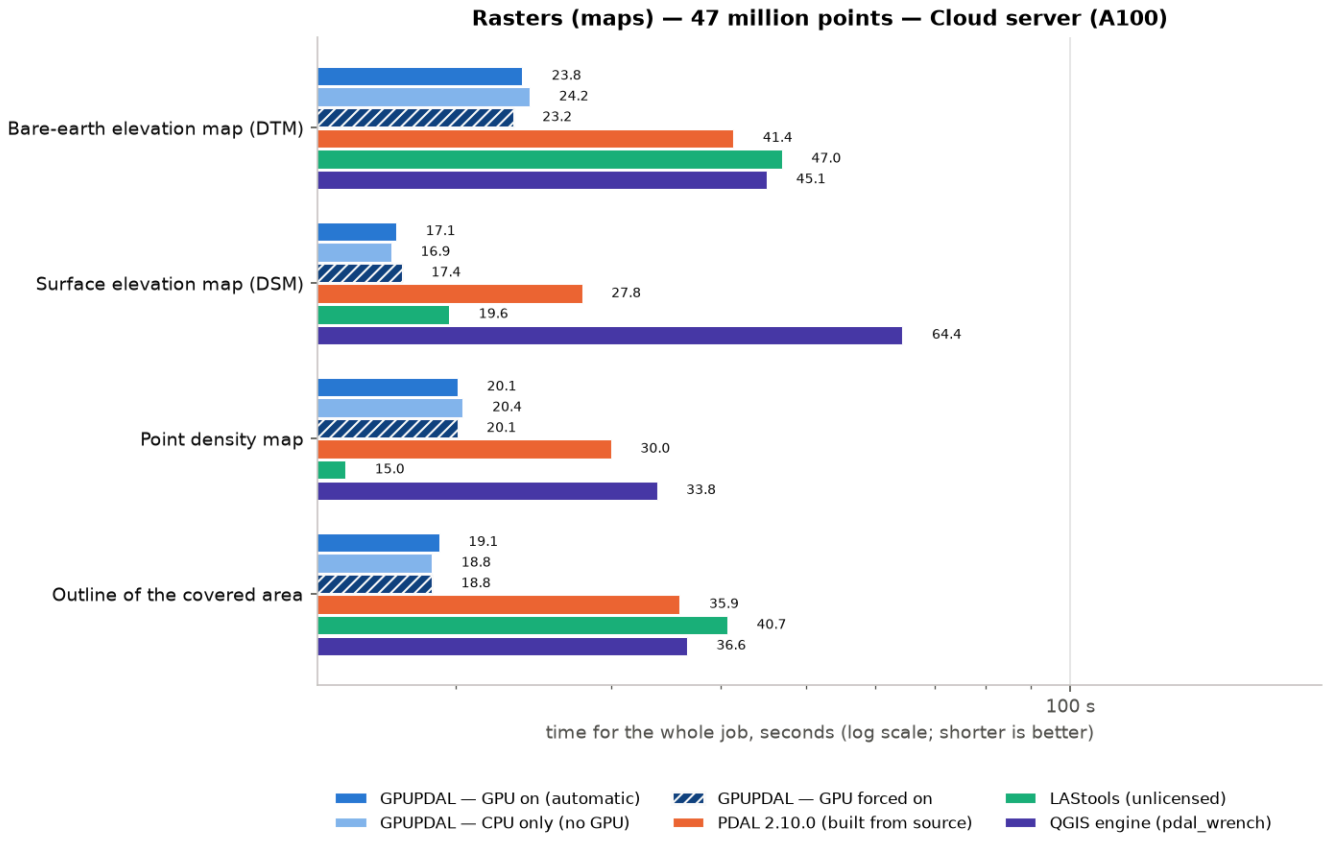
Coordinates and clipping: median time in seconds, 47 million points, Cloud server (A100).



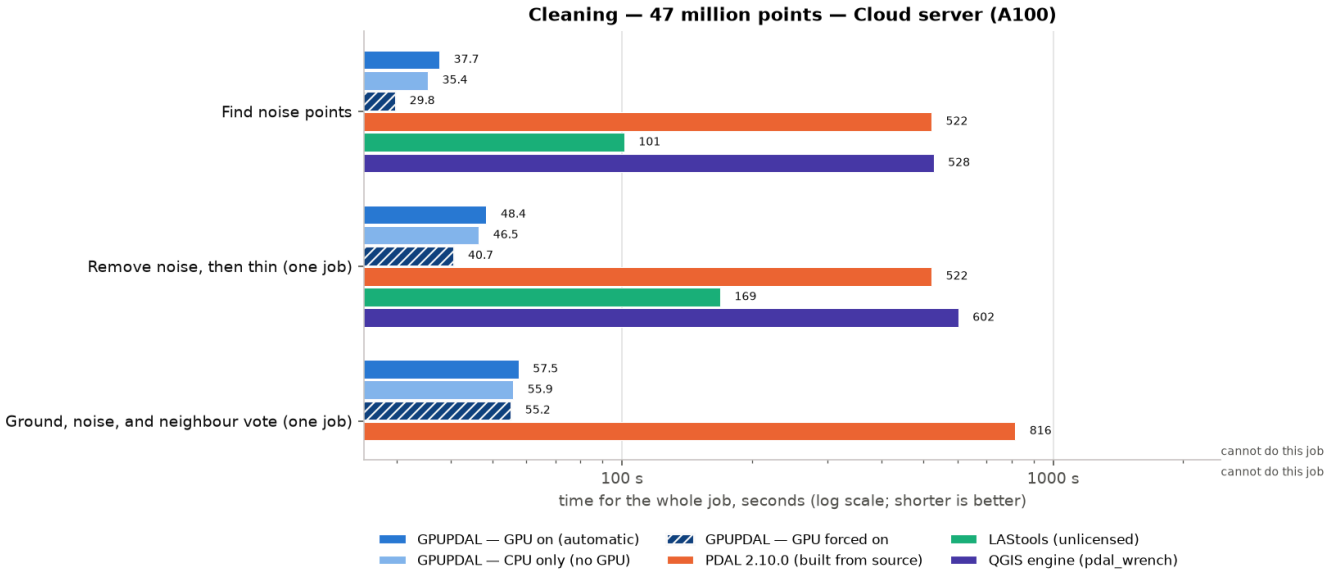
Reducing points: median time in seconds, 47 million points, Cloud server (A100).



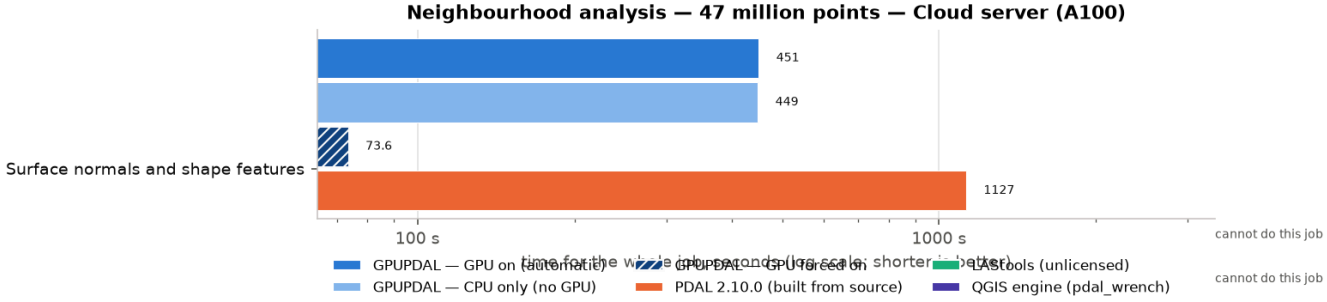
Ground and heights: median time in seconds, 47 million points, Cloud server (A100).



Rasters (maps): median time in seconds, 47 million points, Cloud server (A100).

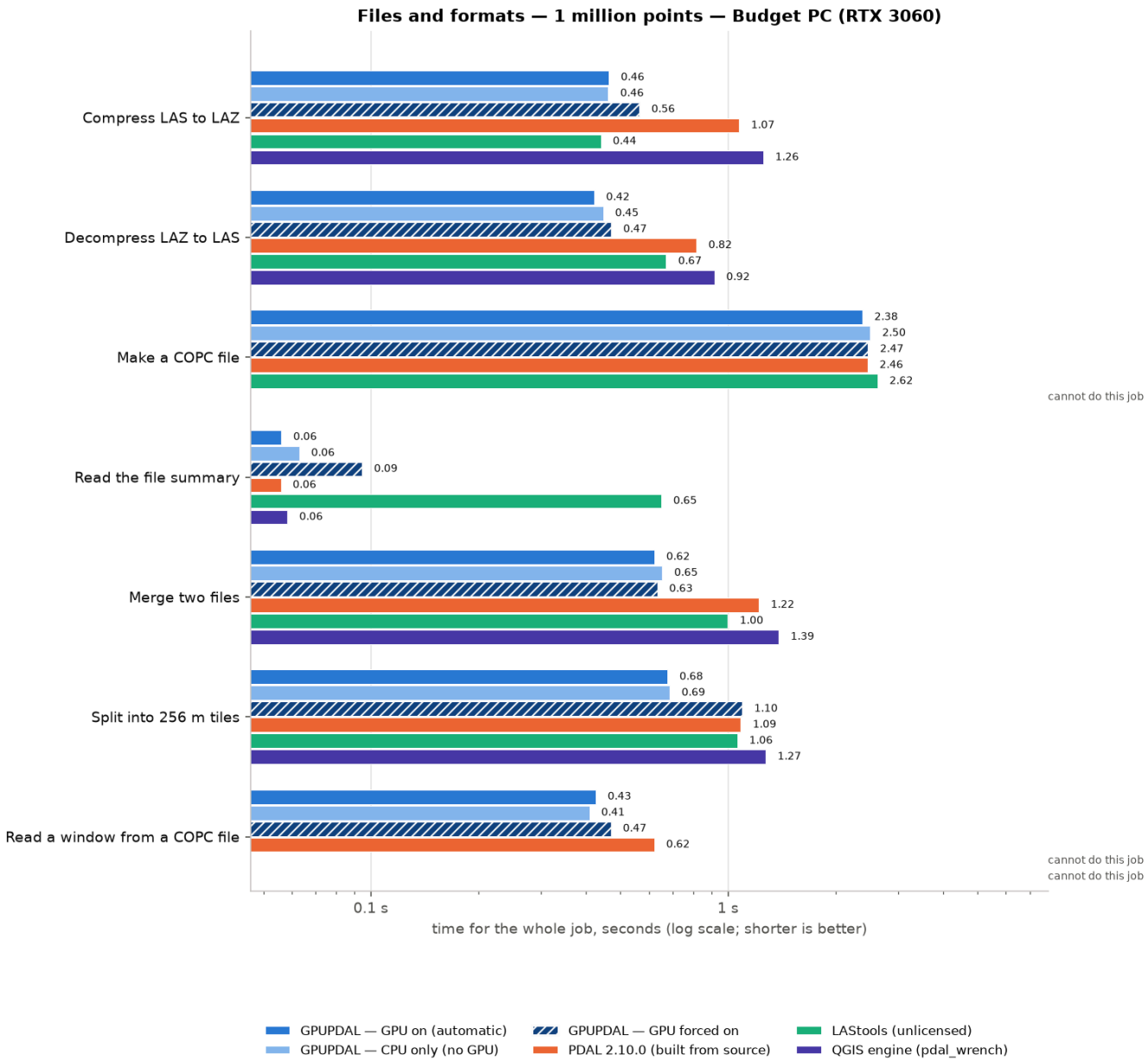


Cleaning: median time in seconds, 47 million points, Cloud server (A100).

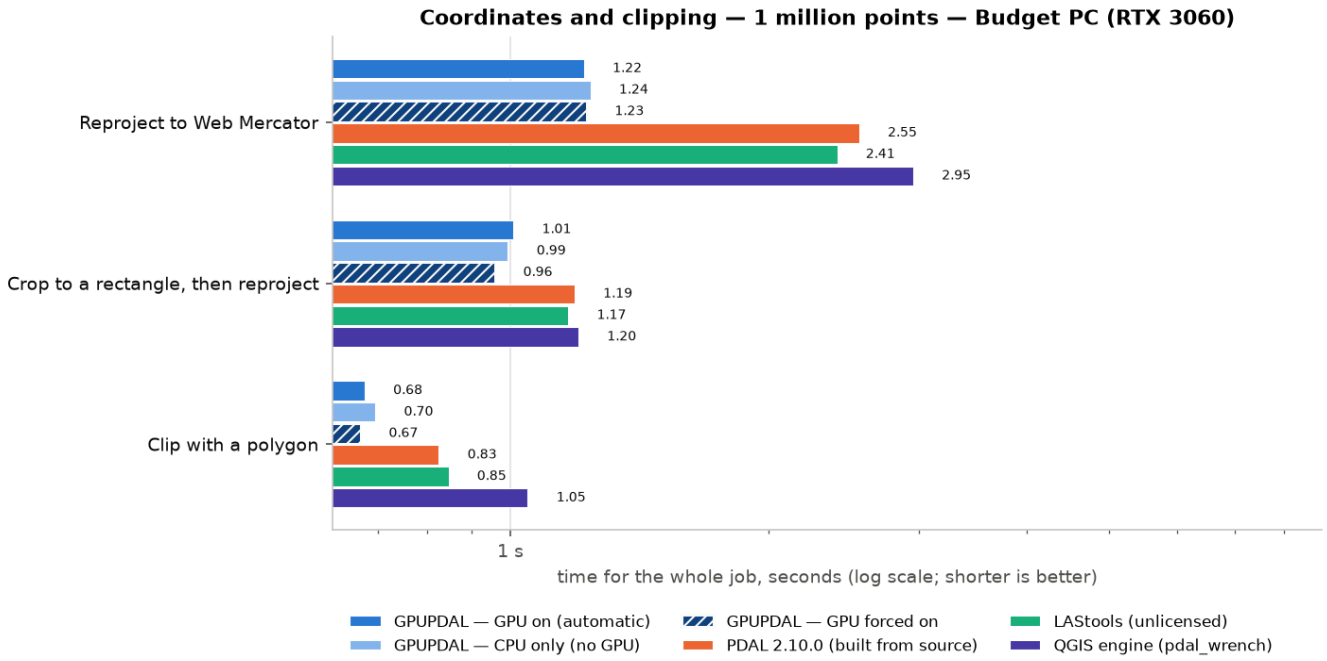


Neighbourhood analysis: median time in seconds, 47 million points, Cloud server (A100).

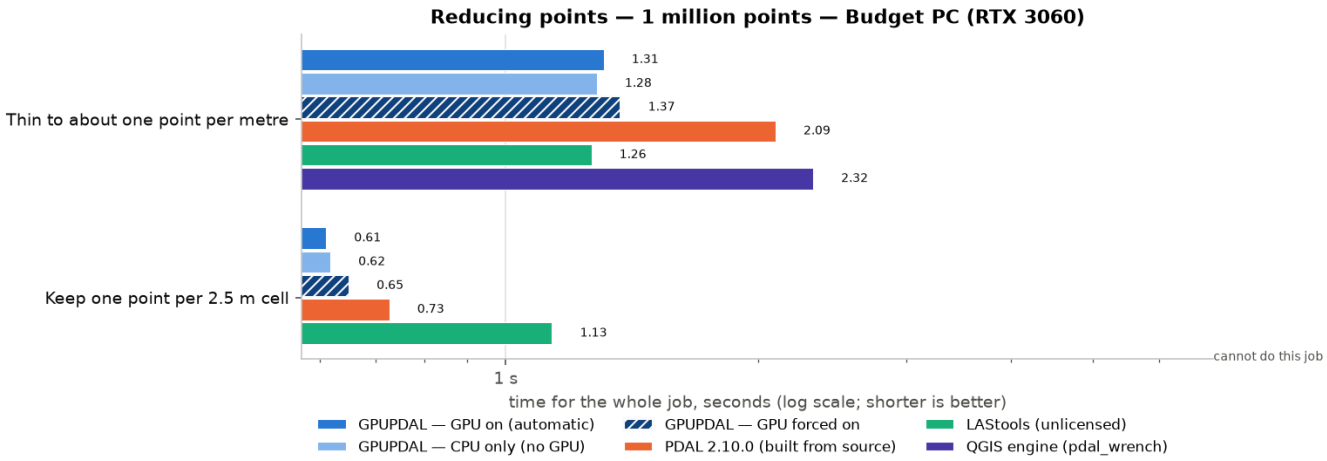
9.2 Every chart for Budget cloud PC (Xeon E5-2697A v4 + RTX 3060 12 GB)



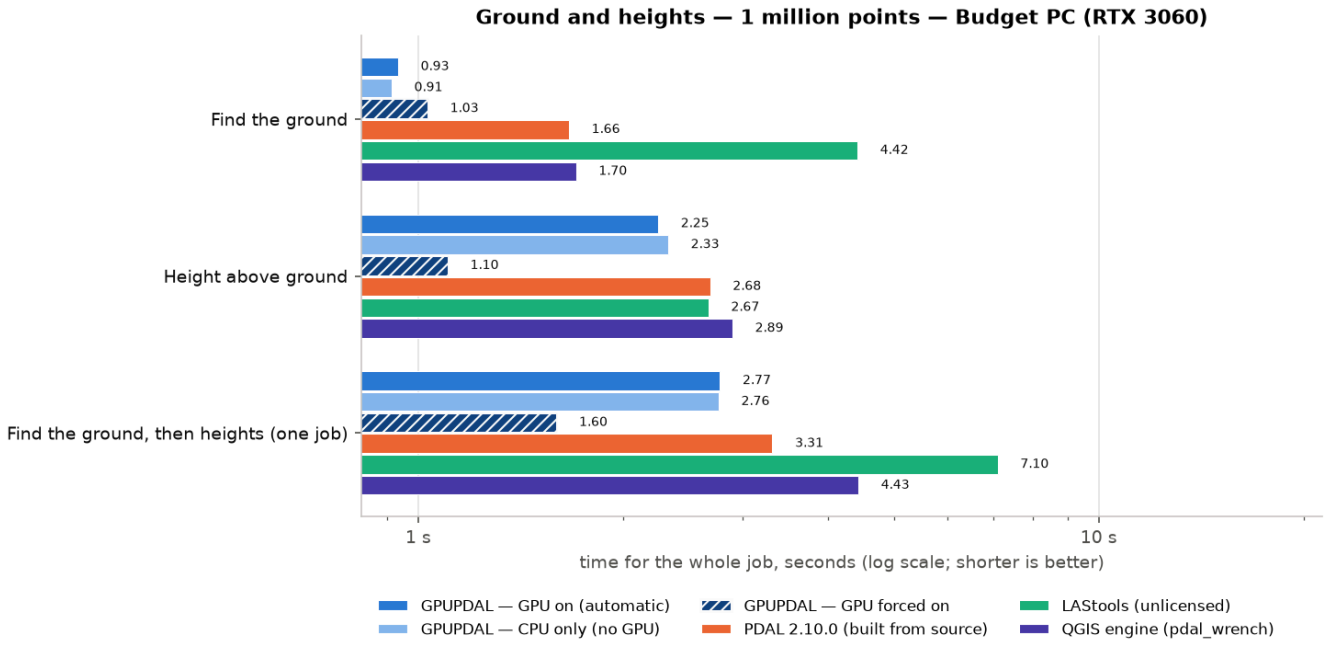
Files and formats: median time in seconds, 1 million points, Budget PC (RTX 3060).



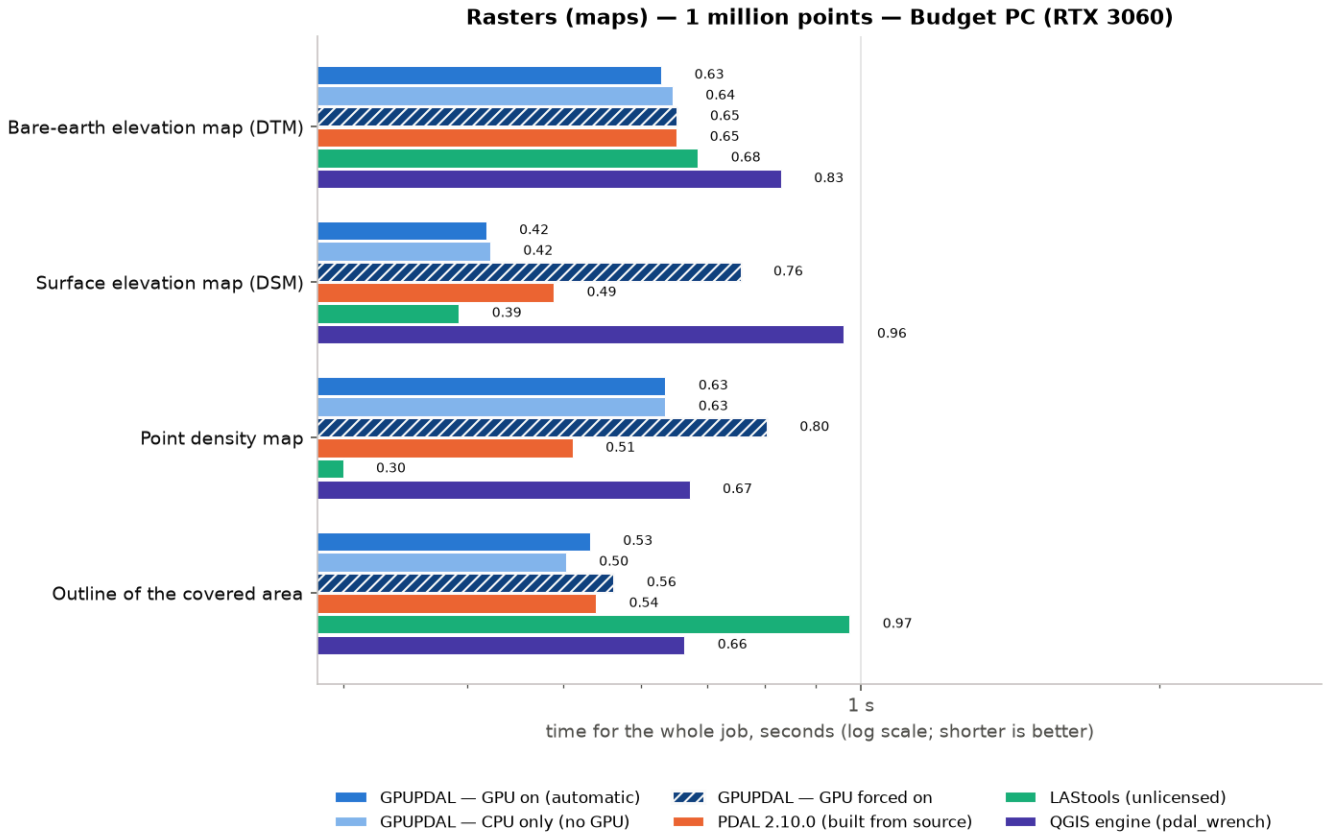
Coordinates and clipping: median time in seconds, 1 million points, Budget PC (RTX 3060).



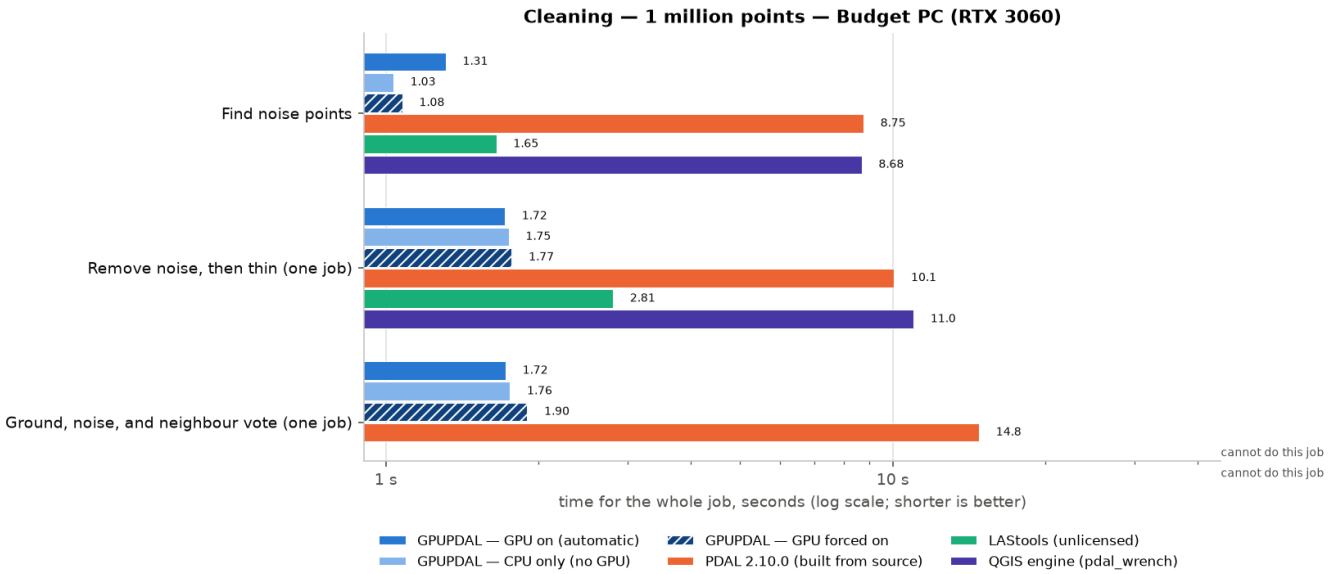
Reducing points: median time in seconds, 1 million points, Budget PC (RTX 3060).



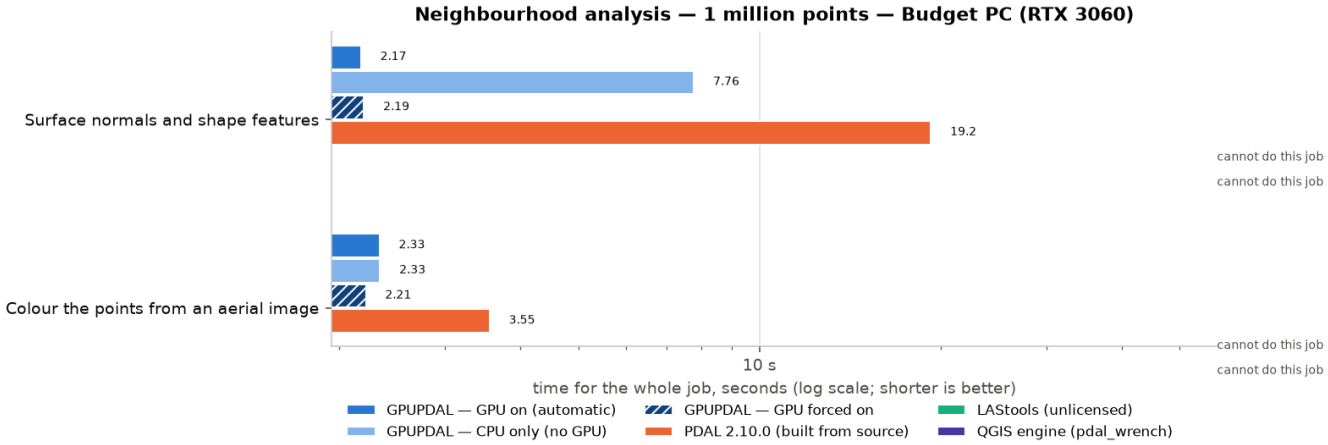
Ground and heights: median time in seconds, 1 million points, Budget PC (RTX 3060).



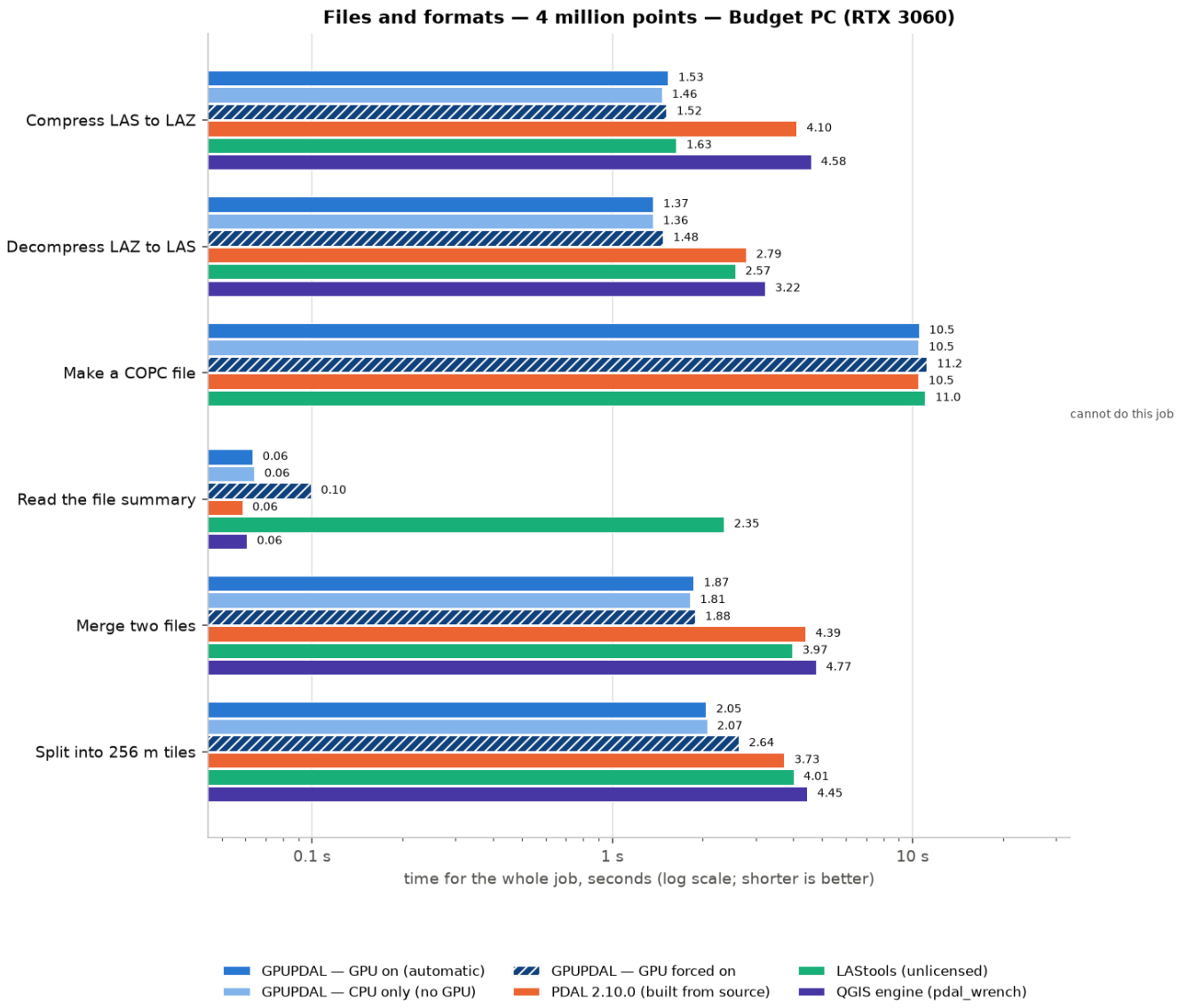
Rasters (maps): median time in seconds, 1 million points, Budget PC (RTX 3060).



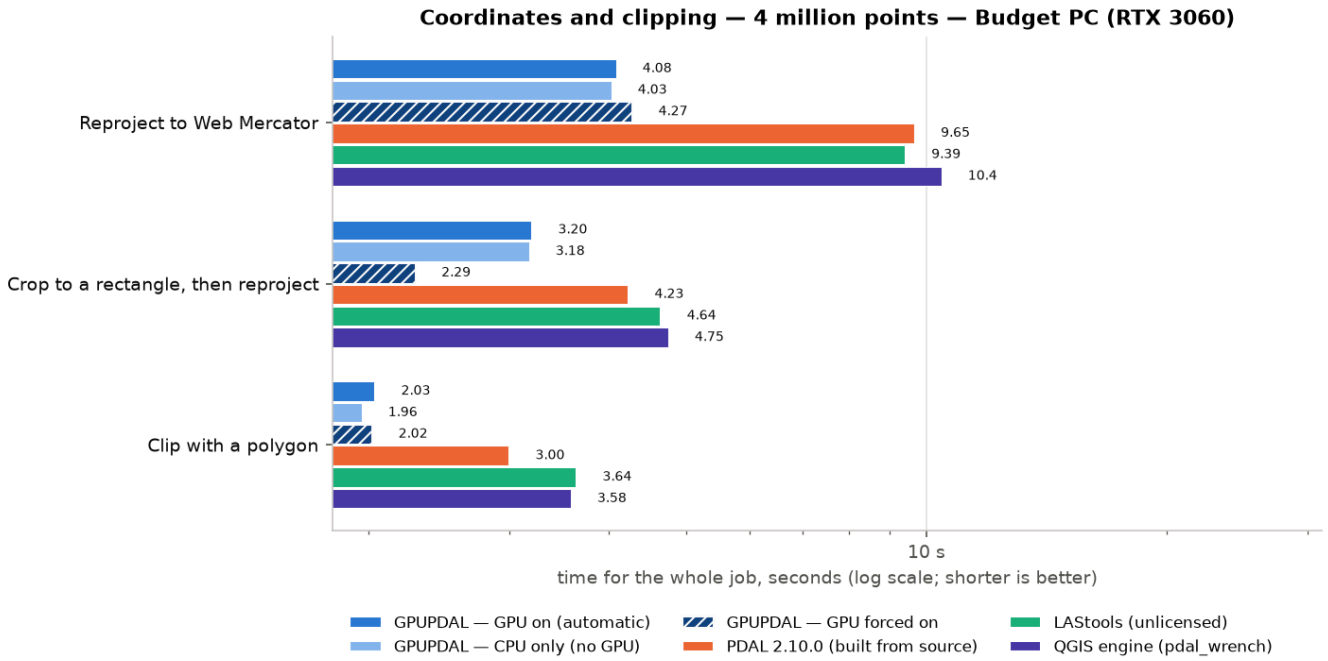
Cleaning: median time in seconds, 1 million points, Budget PC (RTX 3060).



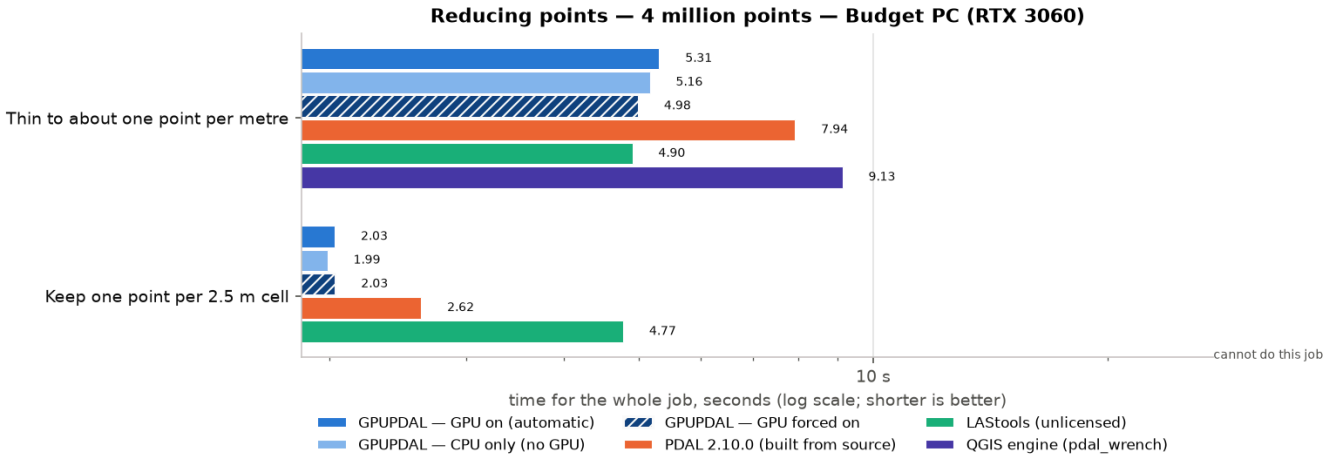
Neighbourhood analysis: median time in seconds, 1 million points, Budget PC (RTX 3060).



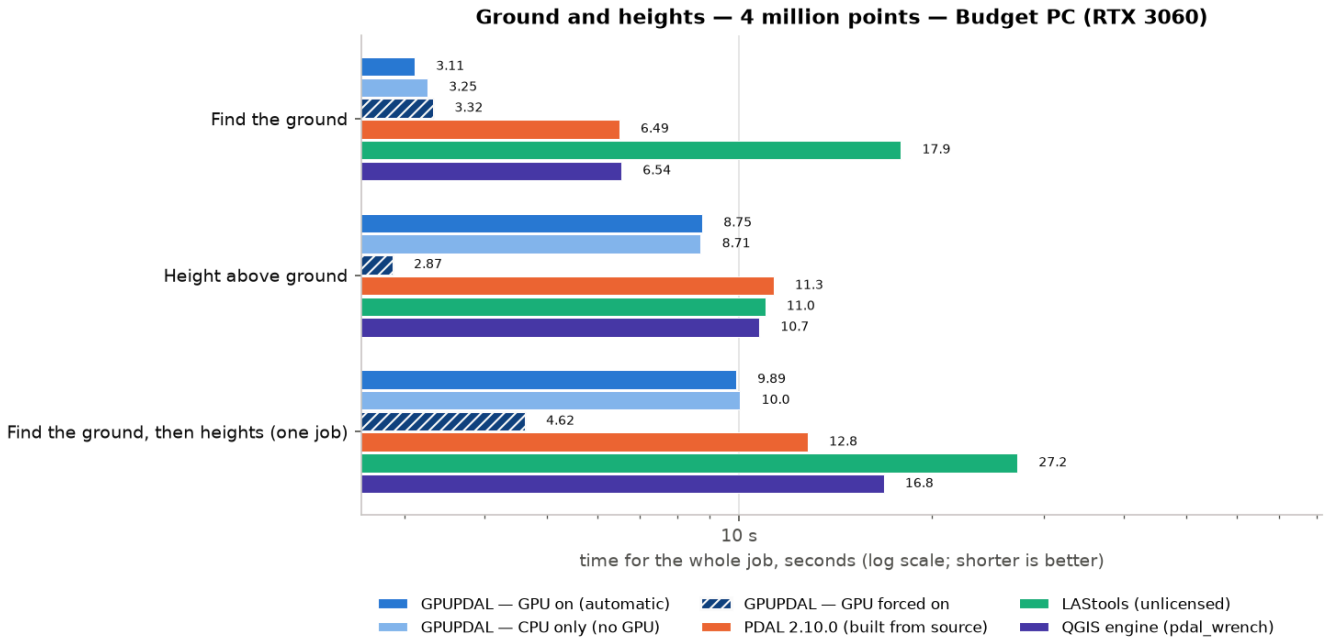
Files and formats: median time in seconds, 4 million points, Budget PC (RTX 3060).



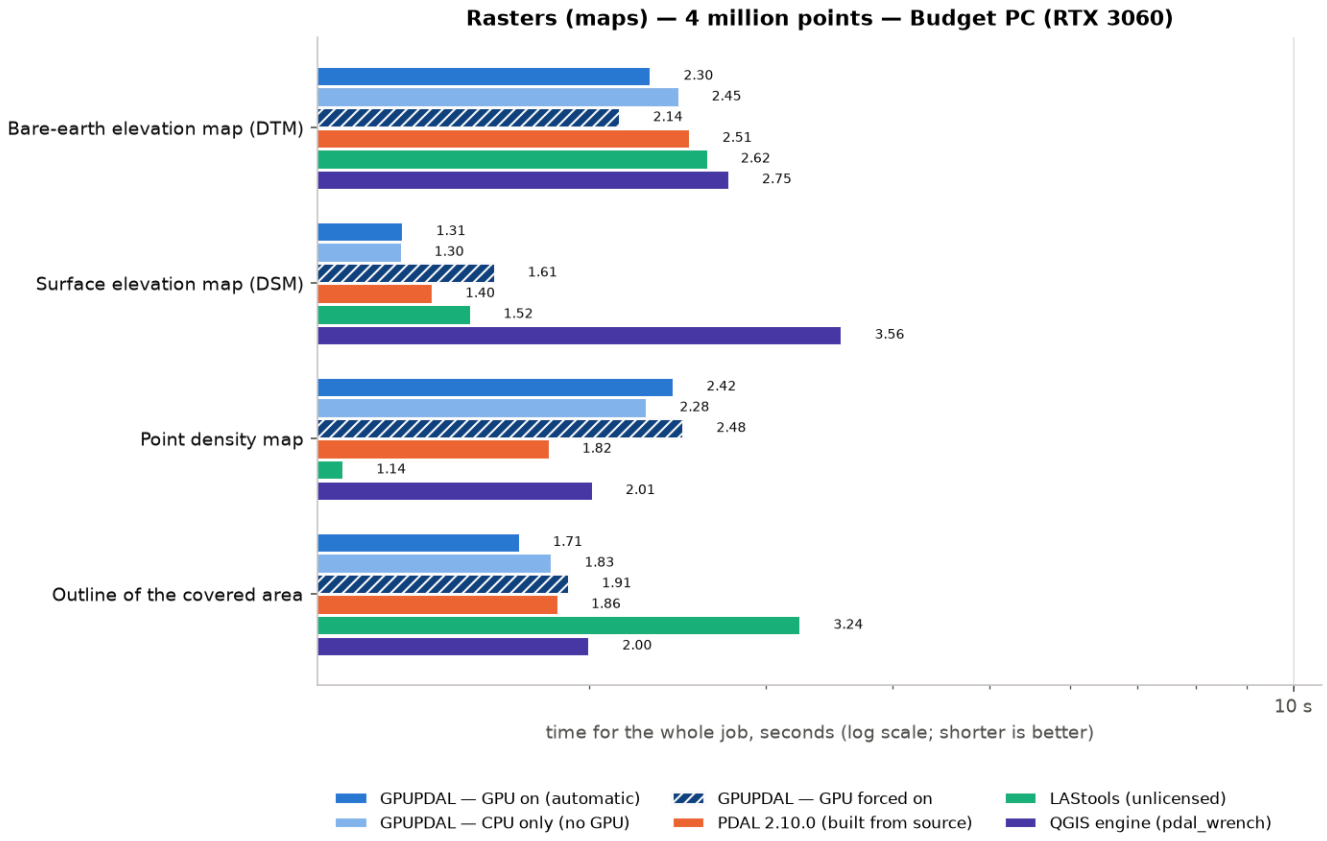
Coordinates and clipping: median time in seconds, 4 million points, Budget PC (RTX 3060).



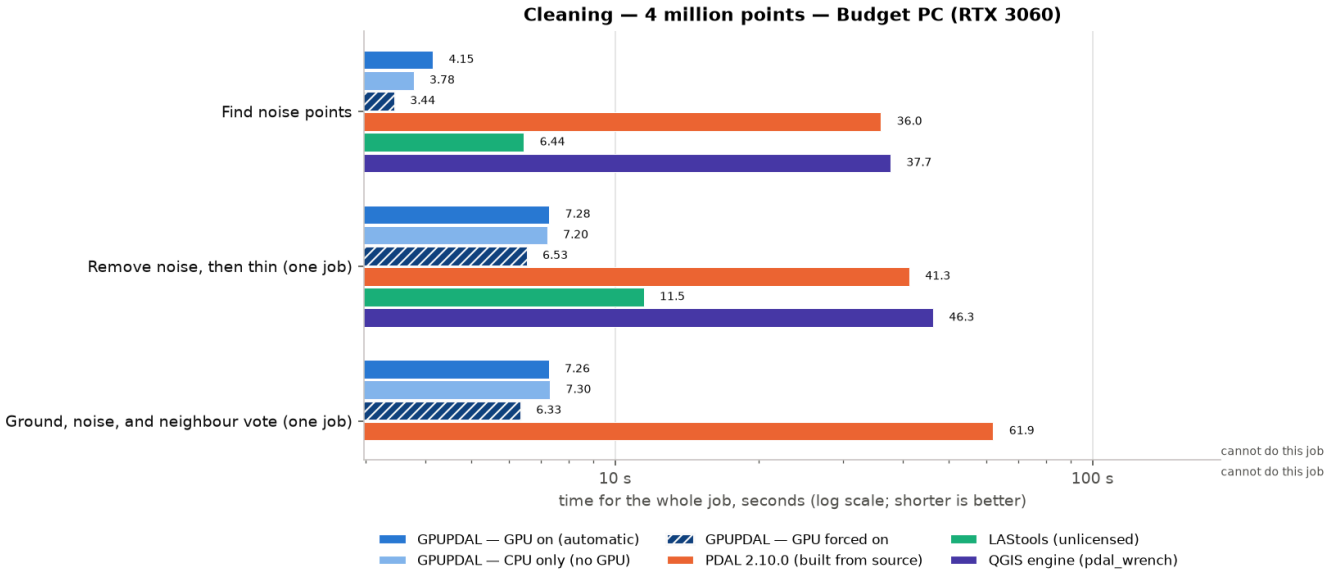
Reducing points: median time in seconds, 4 million points, Budget PC (RTX 3060).



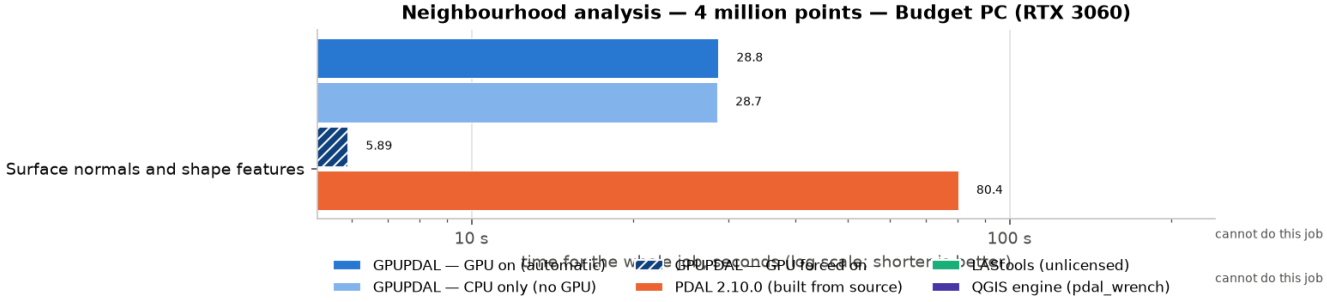
Ground and heights: median time in seconds, 4 million points, Budget PC (RTX 3060).



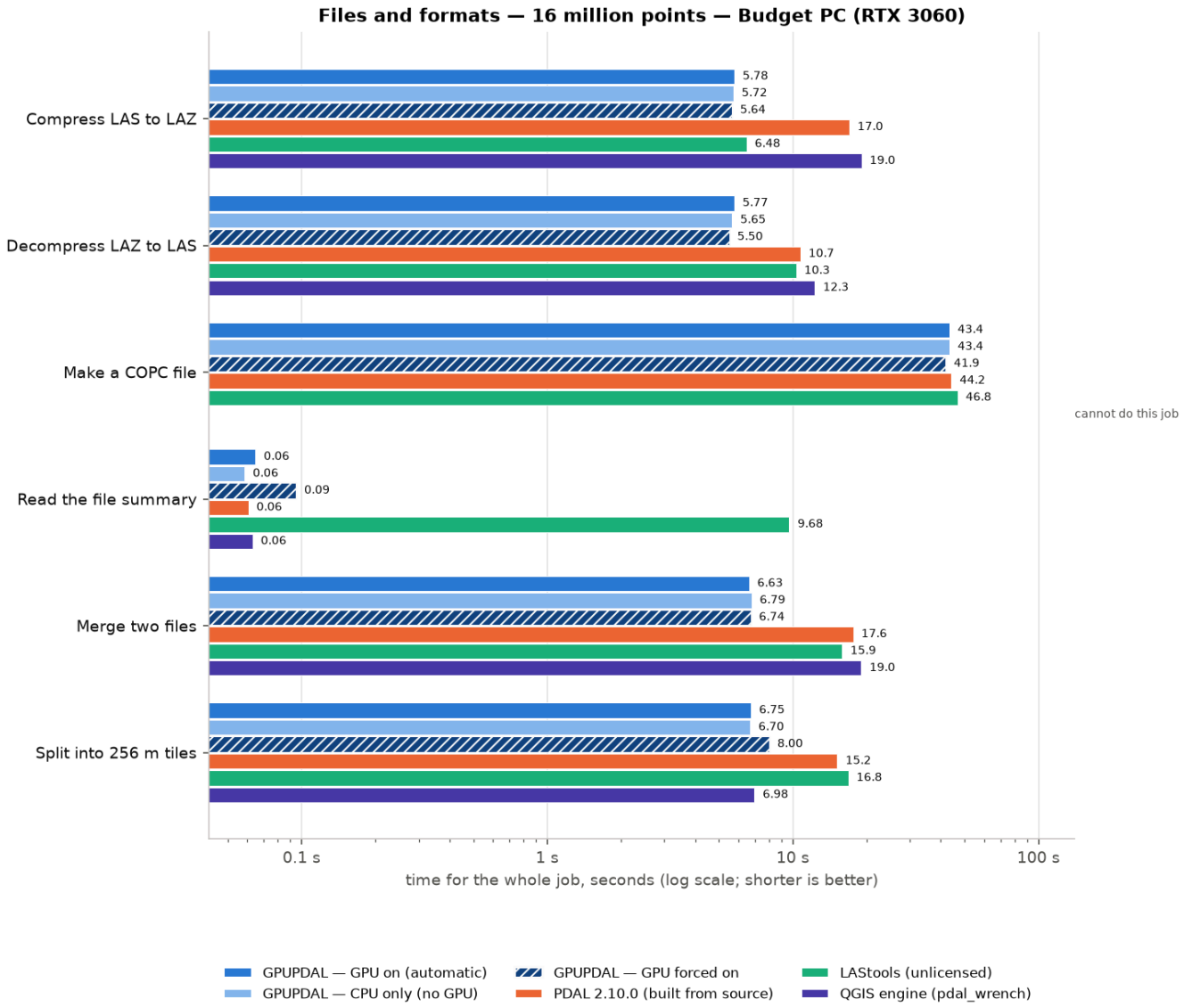
Rasters (maps): median time in seconds, 4 million points, Budget PC (RTX 3060).



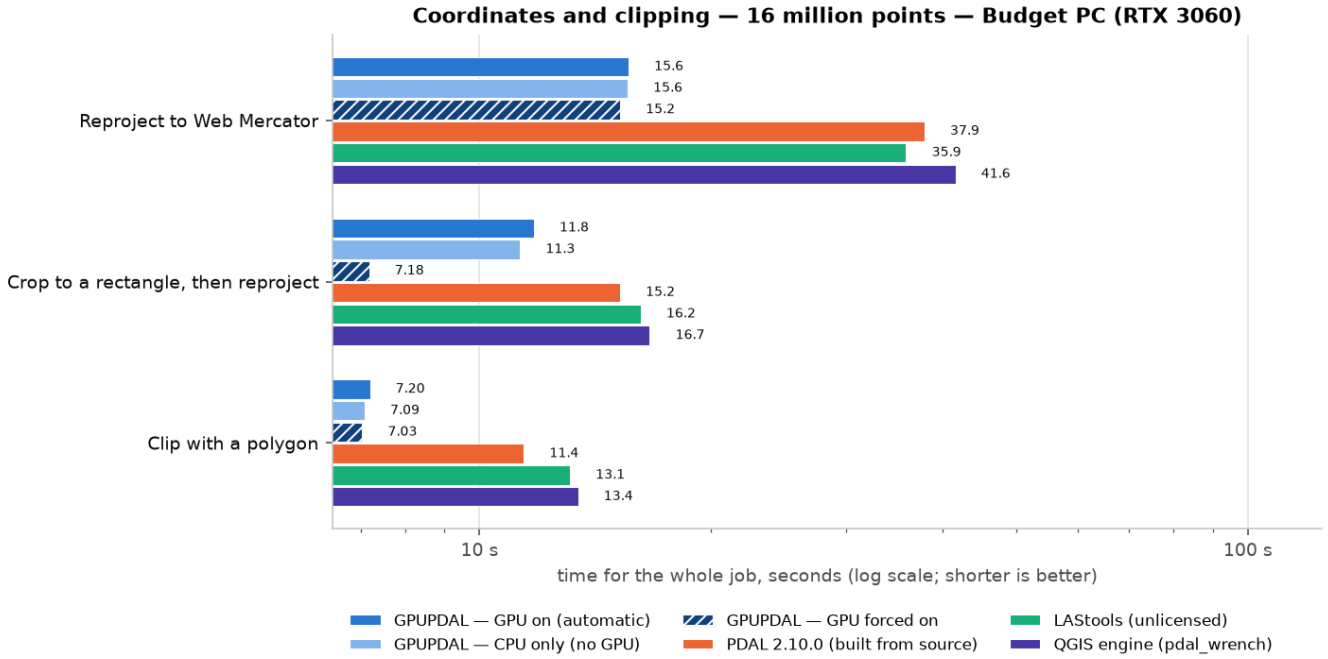
Cleaning: median time in seconds, 4 million points, Budget PC (RTX 3060).



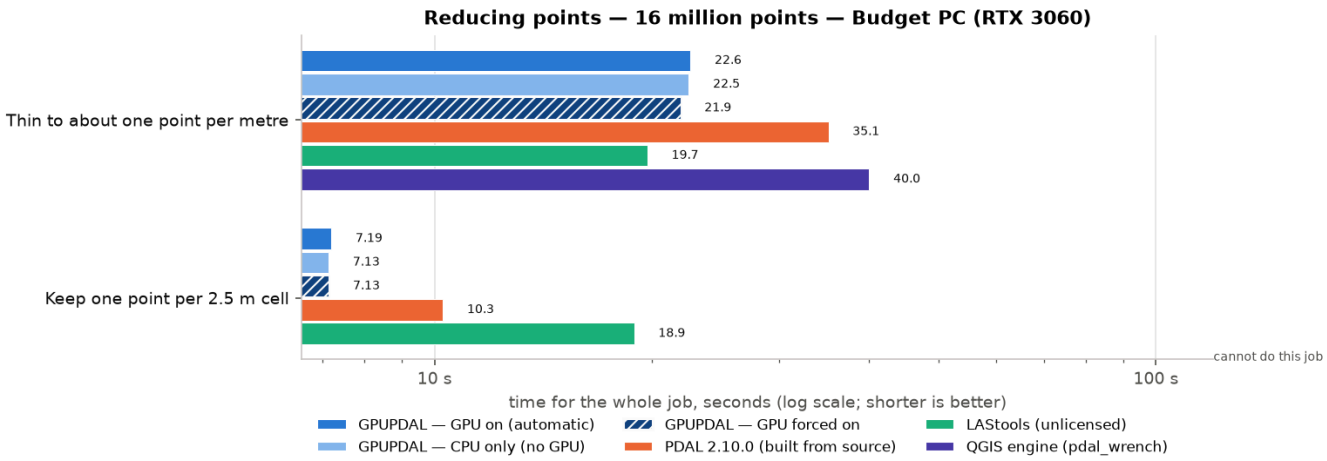
Neighbourhood analysis: median time in seconds, 4 million points, Budget PC (RTX 3060).



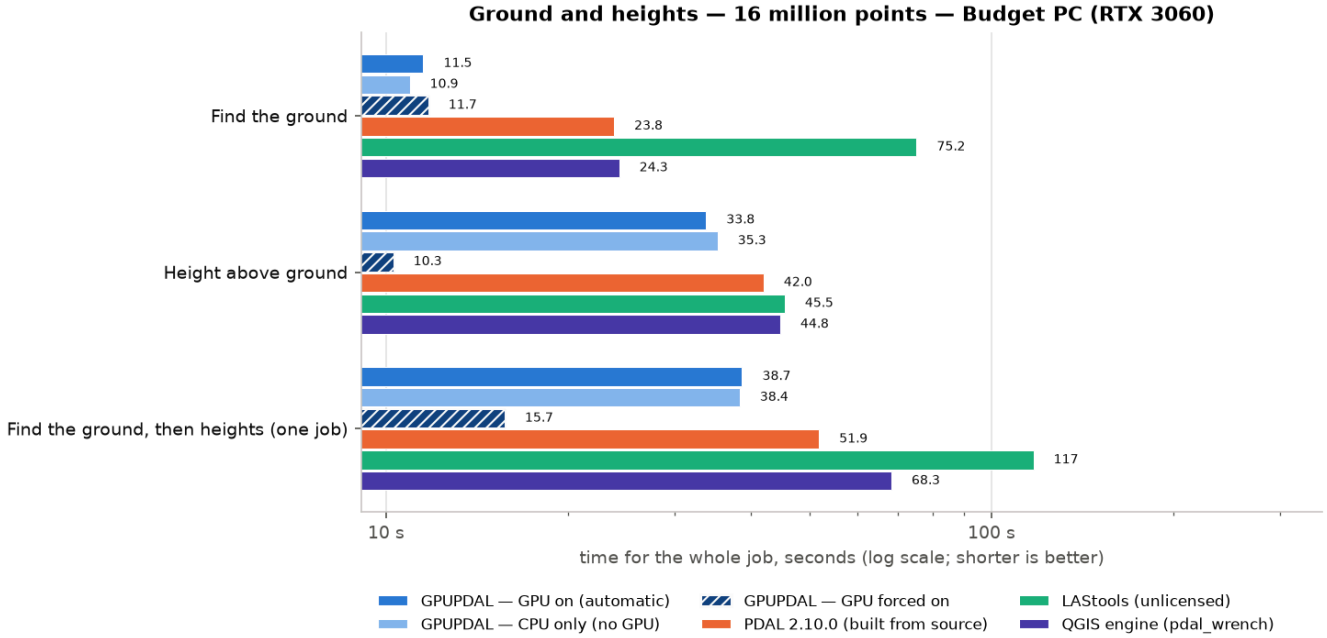
Files and formats: median time in seconds, 16 million points, Budget PC (RTX 3060).



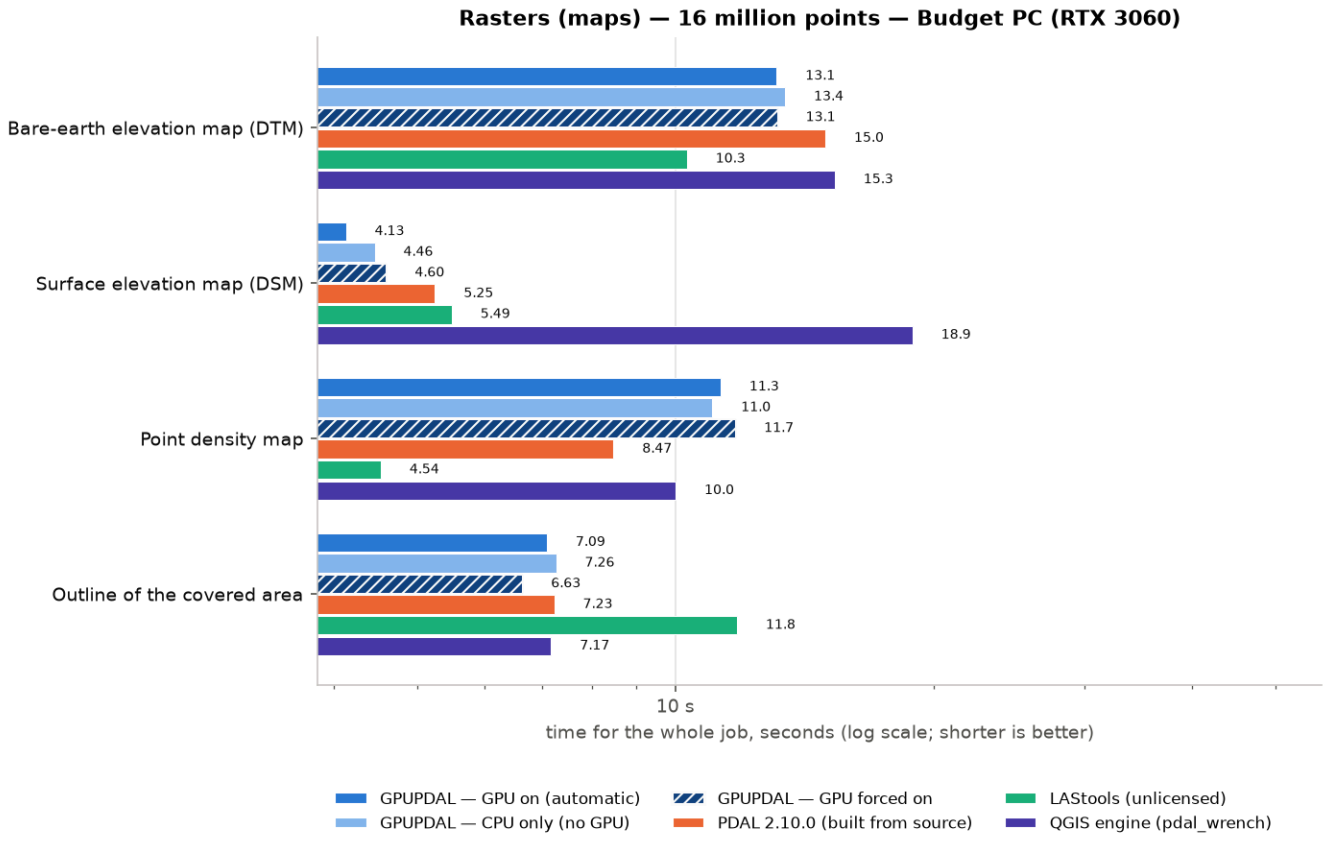
Coordinates and clipping: median time in seconds, 16 million points, Budget PC (RTX 3060).



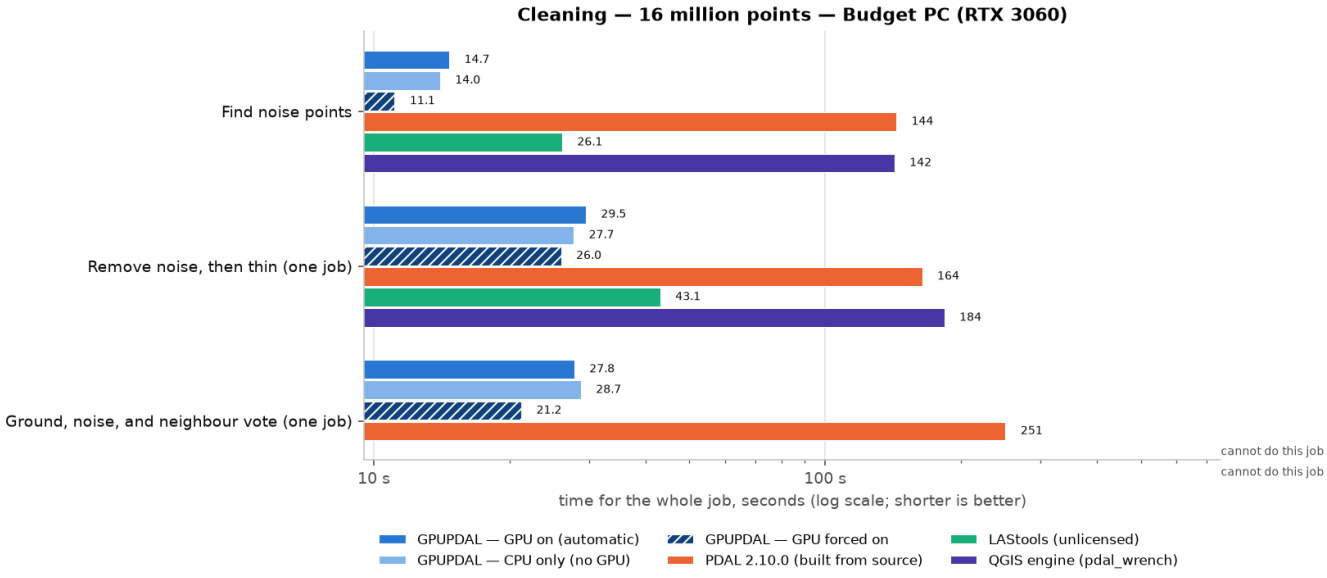
Reducing points: median time in seconds, 16 million points, Budget PC (RTX 3060).



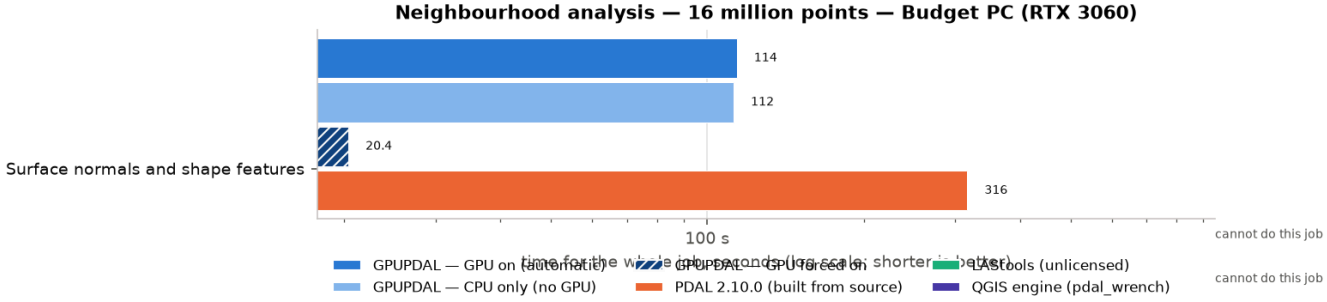
Ground and heights: median time in seconds, 16 million points, Budget PC (RTX 3060).



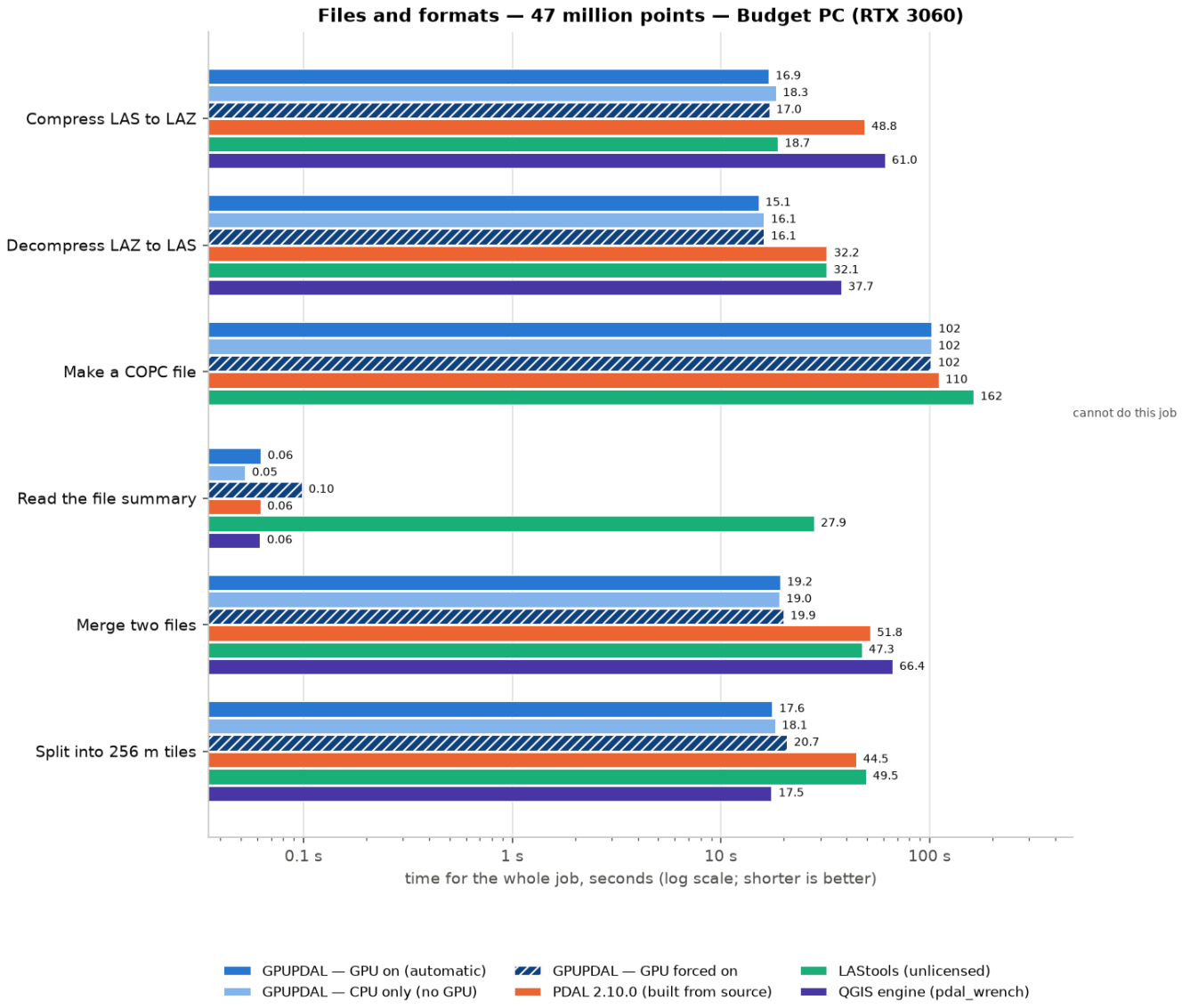
Rasters (maps): median time in seconds, 16 million points, Budget PC (RTX 3060).



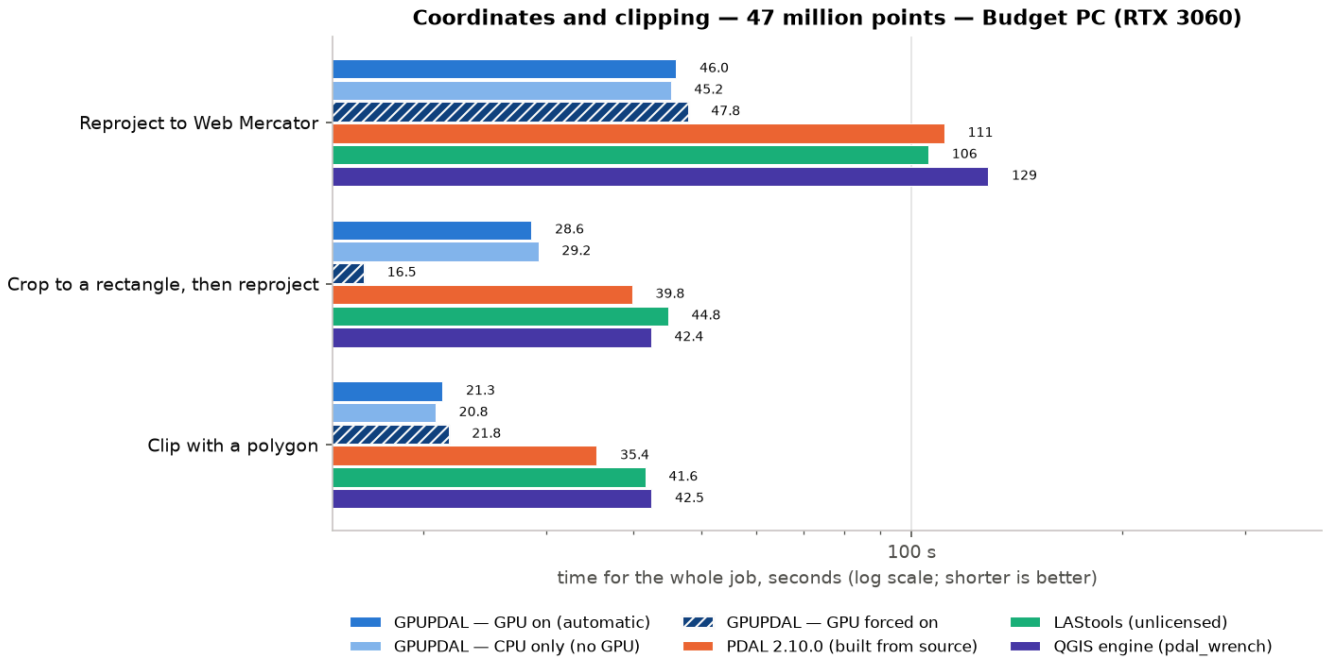
Cleaning: median time in seconds, 16 million points, Budget PC (RTX 3060).



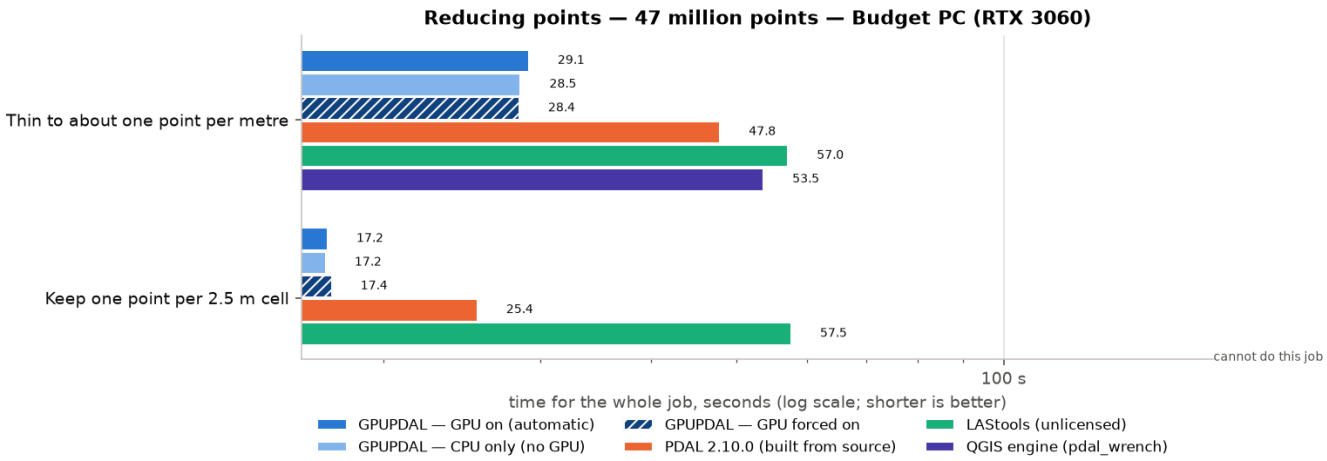
Neighbourhood analysis: median time in seconds, 16 million points, Budget PC (RTX 3060).



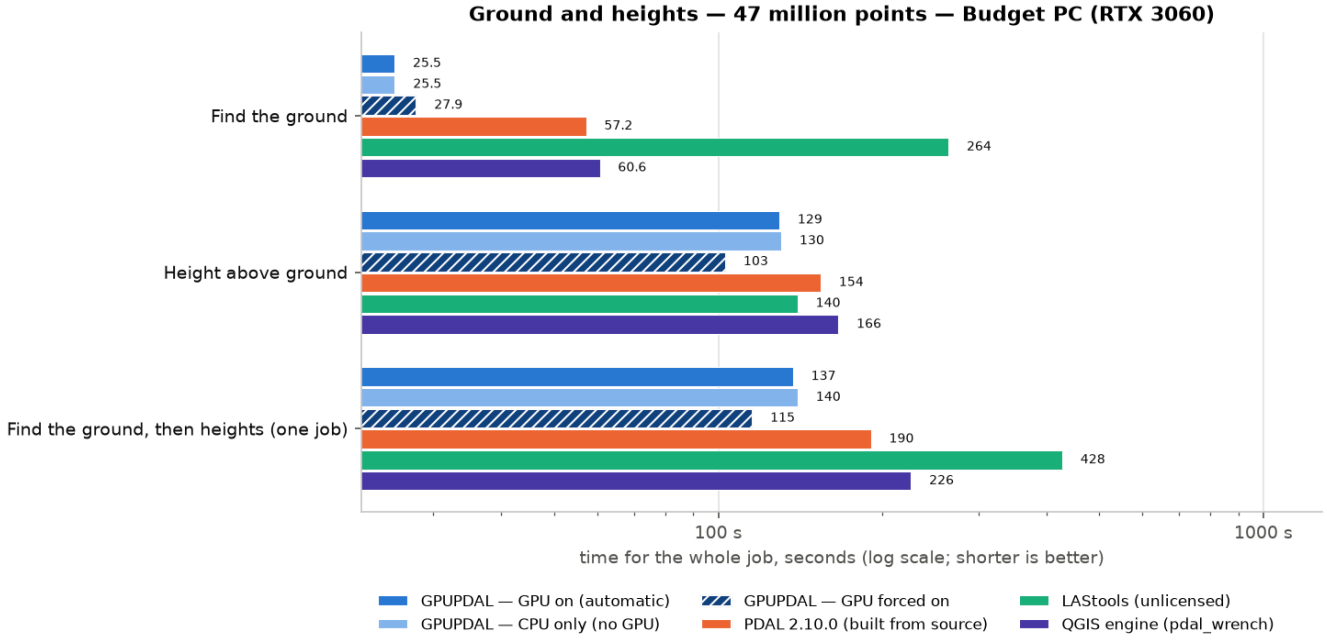
Files and formats: median time in seconds, 47 million points, Budget PC (RTX 3060).



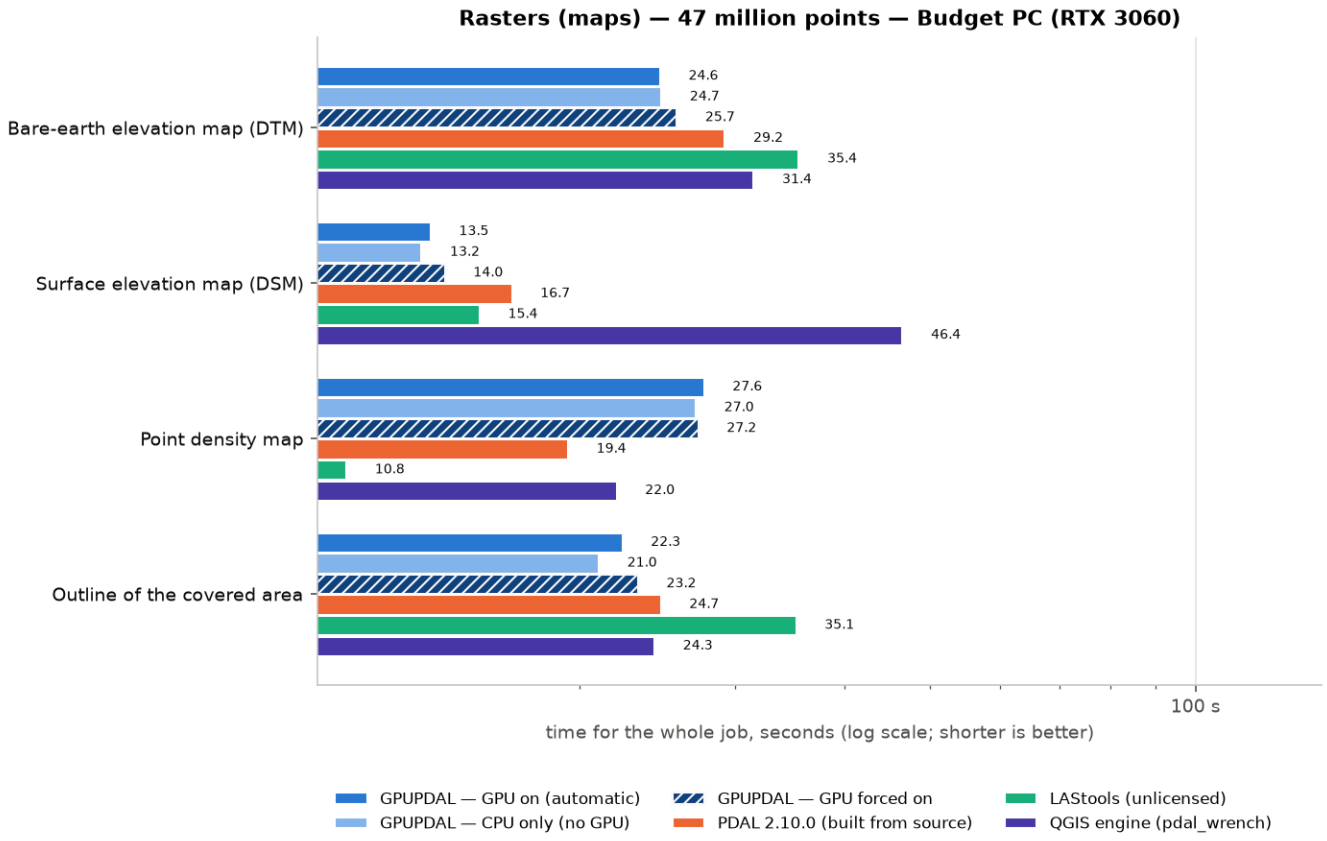
Coordinates and clipping: median time in seconds, 47 million points, Budget PC (RTX 3060).



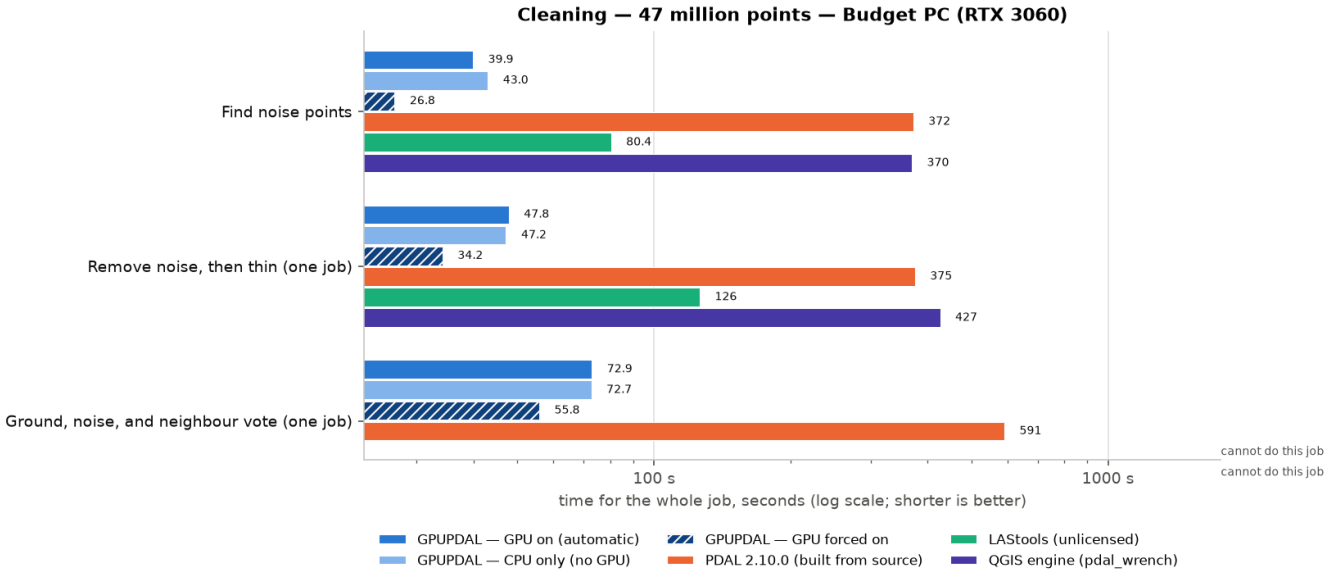
Reducing points: median time in seconds, 47 million points, Budget PC (RTX 3060).



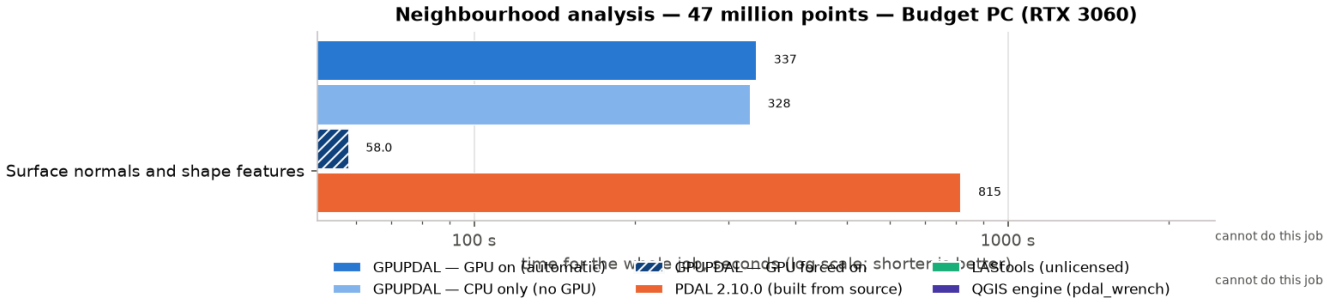
Ground and heights: median time in seconds, 47 million points, Budget PC (RTX 3060).



Rasters (maps): median time in seconds, 47 million points, Budget PC (RTX 3060).



Cleaning: median time in seconds, 47 million points, Budget PC (RTX 3060).



Neighbourhood analysis: median time in seconds, 47 million points, Budget PC (RTX 3060).

9.3 The exact commands (1 million points, workstation)

Job	Tool	Command(s)
Compress LAS to LAZ	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/compress/pipeline.json`
Compress LAS to LAZ	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/compress/pipeline.json`
Compress LAS to LAZ	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/compress/pipeline.json`
Compress LAS to LAZ	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/compress/pipeline.json`
Compress LAS to LAZ	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/compress/pipeline.json`
Compress LAS to LAZ	LAStools	`/home/zy/LASTools/lastools/bin/laszip64 -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.las -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/compress/out.laz`
Compress LAS to LAZ	QGIS engine	`/usr/lib/qgis/pdal_wrench translate --input=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.las --output=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/compress/out.laz --threads=24`
Compress LAS to LAZ	QGIS	`/usr/bin/qgis_process run pdal:convertformat --INPUT=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.las --OUTPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/compress/out.laz --VPC_OUTPUT_FORMAT=0`
Decompress LAZ to LAS	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/decompress/pipeline.json`
Decompress LAZ to LAS	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/decompress/pipeline.json`
Decompress LAZ to LAS	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/decompress/pipeline.json`
Decompress LAZ to LAS	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/decompress/pipeline.json`
Decompress LAZ to LAS	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/decompress/pipeline.json`
Decompress LAZ to LAS	LAStools	`/home/zy/LASTools/lastools/bin/laszip64 -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/decompress/out.las`
Decompress LAZ to LAS	QGIS engine	`/usr/lib/qgis/pdal_wrench translate --input=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --output=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/decompress/out.las --threads=24`

Job	Tool	Command(s)
Decompress LAZ to LAS	QGIS	`/usr/bin/qgis_process run pdal:convertformat --INPUT=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --OUTPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/decompress/out.las --VPC_OUTPUT_FORMAT=0`
Make a COPC file	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/copc/pipeline.json`
Make a COPC file	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/copc/pipeline.json`
Make a COPC file	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/copc/pipeline.json`
Make a COPC file	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/copc/pipeline.json`
Make a COPC file	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/copc/pipeline.json`
Make a COPC file	LAStools	`/home/zy/LASTools/lastools/bin/lascopcindex64 -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/copc/out.copc.laz`
Make a COPC file	QGIS engine	`/usr/lib/qgis/untwine -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz -o /home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/copc/out.copc.laz --single_file`
Make a COPC file	QGIS	`/usr/bin/qgis_process run pdal:createcopc --LAYERS=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --OUTPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/copc/copc`
Read the file summary	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg info --summary /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz`
Read the file summary	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg info --summary /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz`
Read the file summary	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg info --summary /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz`
Read the file summary	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal info --summary /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz`
Read the file summary	PDAL 2.10.1 pkg	`/usr/bin/pdal info --summary /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz`
Read the file summary	LAStools	`/home/zy/LASTools/lastools/bin/lasinfo64 -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/info/info.txt`
Read the file summary	QGIS engine	`/usr/lib/qgis/pdal_wrench info --input=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz`
Read the file summary	QGIS	`/usr/bin/qgis_process run pdal:info --INPUT=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --OUTPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/info/info.html`

Job	Tool	Command(s)
Reproject to Web Mercator	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/reproject/pipeline.json`
Reproject to Web Mercator	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/reproject/pipeline.json`
Reproject to Web Mercator	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/reproject/pipeline.json`
Reproject to Web Mercator	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/reproject/pipeline.json`
Reproject to Web Mercator	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/reproject/pipeline.json`
Reproject to Web Mercator	LAStools	`/home/zy/LASTools/lastools/bin/las2las64 -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz -proj_epsg 28992 3857 -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/reproject/out.laz`
Reproject to Web Mercator	QGIS engine	`/usr/lib/qgis/pdal_wrench translate --input=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --output=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/reproject/out.laz --transform-crs=EPSG:3857 --threads=24`
Reproject to Web Mercator	QGIS	`/usr/bin/qgis_process run pdal:reproject --INPUT=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --CRS=EPSG:3857 --OUTPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/reproject/out.laz --VPC_OUTPUT_FORMAT=0`
Crop to a rectangle, then reproject	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/crop_reproject/pipeline.json`
Crop to a rectangle, then reproject	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/crop_reproject/pipeline.json`
Crop to a rectangle, then reproject	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/crop_reproject/pipeline.json`
Crop to a rectangle, then reproject	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/crop_reproject/pipeline.json`
Crop to a rectangle, then reproject	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/crop_reproject/pipeline.json`
Crop to a rectangle, then reproject	LAStools	`/home/zy/LASTools/lastools/bin/las2las64 -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz -keep_xy 184874.9975 494942.405 185624.9925 494980.795 -proj_epsg 28992 3857 -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/crop_reproject/out.laz`

Job	Tool	Command(s)
Crop to a rectangle, then reproject	QGIS engine	`/usr/lib/qgis/pdal_wrench translate --input=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --output=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/crop_reproject/out.laz --bounds=([184874.9975,185624.9925],[494942.405,494980.795]) --transform-crs=EPSG:3857 --threads=24`
Clip with a polygon	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/clip/pipeline.json`
Clip with a polygon	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/clip/pipeline.json`
Clip with a polygon	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/clip/pipeline.json`
Clip with a polygon	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/clip/pipeline.json`
Clip with a polygon	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/clip/pipeline.json`
Clip with a polygon	LAStools	`/home/zy/LASTools/lastools/bin/lasclip64 -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz -poly /home/zy/dev/pdal-gpu/build/independent-bench/in/1m-clip.shp -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/clip/out.laz`
Clip with a polygon	QGIS engine	`/usr/lib/qgis/pdal_wrench clip --input=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --output=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/clip/out.laz --polygon=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m-clip.gpkg --threads=24`
Clip with a polygon	QGIS	`/usr/bin/qgis_process run pdal:clip --INPUT=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --OVERLAY=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m-clip.gpkg --OUTPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/clip/out.laz --VPC_OUTPUT_FORMAT=0`
Merge two files	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/merge/pipeline.json`
Merge two files	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/merge/pipeline.json`
Merge two files	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/merge/pipeline.json`
Merge two files	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/merge/pipeline.json`
Merge two files	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/merge/pipeline.json`

Job	Tool	Command(s)
Merge two files	LAStools	`/home/zy/LASTools/lastools/bin/lasmerge64 -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m-west.laz /home/zy/dev/pdal-gpu/build/independent-bench/in/1m-east.laz -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/merge/out.laz`
Merge two files	QGIS engine	`/usr/lib/qgis/pdal_wrench merge --output=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/merge/out.laz --threads=24 /home/zy/dev/pdal-gpu/build/independent-bench/in/1m-west.laz /home/zy/dev/pdal-gpu/build/independent-bench/in/1m-east.laz`
Merge two files	QGIS	`/usr/bin/qgis_process run pdal:merge --LAYERS=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m-west.laz --LAYERS=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m-east.laz --OUTPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/merge/out.laz --VPC_OUTPUT_FORMAT=0`
Split into 256 m tiles	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/tile/pipeline.json`
Split into 256 m tiles	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/tile/pipeline.json`
Split into 256 m tiles	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/tile/pipeline.json`
Split into 256 m tiles	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/tile/pipeline.json`
Split into 256 m tiles	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/tile/pipeline.json`
Split into 256 m tiles	LAStools	`/home/zy/LASTools/lastools/bin/lastile64 -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz -tile_size 256 -olaz -odir /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/tile/tiles -o tile.laz`
Split into 256 m tiles	QGIS engine	`/usr/lib/qgis/pdal_wrench tile --input-file-list=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/tile/inputs.txt --output=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/tile/tiles --length=256 --threads=24`
Split into 256 m tiles	QGIS	`/usr/bin/qgis_process run pdal:tile --LAYERS=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --LENGTH=256 --OUTPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/tile/tiles --VPC_OUTPUT_FORMAT=0`
Thin to about one point per metre	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/thin/pipeline.json`
Thin to about one point per metre	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/thin/pipeline.json`
Thin to about one point per metre	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/thin/pipeline.json`

Job	Tool	Command(s)
Thin to about one point per metre	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/thin/pipeline.json`
Thin to about one point per metre	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/thin/pipeline.json`
Thin to about one point per metre	LAStools	`/home/zy/LASTools/lastools/bin/lasthin64 -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz -step 1 -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/thin/out.laz`
Thin to about one point per metre	QGIS engine	`/usr/lib/qgis/pdal_wrench thin --input=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --output=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/thin/out.laz --mode=sample --step-sample=1 --threads=24`
Thin to about one point per metre	QGIS	`/usr/bin/qgis_process run pdal:thinbyradius --INPUT=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --SAMPLING_RADIUS=1 --OUTPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/thin/out.laz --VPC_OUTPUT_FORMAT=0`
Keep one point per 2.5 m cell	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/decimate_grid/pipeline.json`
Keep one point per 2.5 m cell	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/decimate_grid/pipeline.json`
Keep one point per 2.5 m cell	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/decimate_grid/pipeline.json`
Keep one point per 2.5 m cell	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/decimate_grid/pipeline.json`
Keep one point per 2.5 m cell	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/decimate_grid/pipeline.json`
Keep one point per 2.5 m cell	LAStools	`/home/zy/LASTools/lastools/bin/lasthin64 -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz -step 2.5 -central -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/decimate_grid/out.laz`
Find the ground	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/ground/pipeline.json`
Find the ground	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/ground/pipeline.json`
Find the ground	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/ground/pipeline.json`
Find the ground	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/ground/pipeline.json`
Find the ground	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/ground/pipeline.json`

Job	Tool	Command(s)
Find the ground	LAStools	`/home/zy/LASTools/lastools/bin/lasground_new64 -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/ground/out.laz`
Find the ground	QGIS engine	`/usr/lib/qgis/pdal_wrench classify_ground --input=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --output=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/ground/out.laz --cell-size=1 --scalar=1.25 --slope=0.15 --threshold=0.5 --window-size=18 --threads=24`
Find the ground	QGIS	`/usr/bin/qgis_process run pdal:classifyground --INPUT=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --OUTPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/ground/out.laz --VPC_OUTPUT_FORMAT=0`
Height above ground	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/hag/pipeline.json`
Height above ground	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/hag/pipeline.json`
Height above ground	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/hag/pipeline.json`
Height above ground	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/hag/pipeline.json`
Height above ground	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/hag/pipeline.json`
Height above ground	LAStools	`/home/zy/LASTools/lastools/bin/lasheight64 -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m-ground.laz -store_as_extra_bytes -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/hag/out.laz`
Height above ground	QGIS engine	`/usr/lib/qgis/pdal_wrench height_above_ground --input=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m-ground.laz --output=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/hag/out.laz --algorithm=nn --replace-z=false --nn-count=1 --nn-max-distance=0 --threads=24`
Height above ground	QGIS	`/usr/bin/qgis_process run pdal:heightabovegroundbynearestneighbor --INPUT=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m-ground.laz --REPLACE_Z=false --COUNT=1 --MAX_DISTANCE=0 --OUTPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/hag/out.laz --VPC_OUTPUT_FORMAT=0`
Find the ground, then heights (one job)	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/ground_hag/pipeline.json`
Find the ground, then heights (one job)	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/ground_hag/pipeline.json`
Find the ground, then heights (one job)	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/ground_hag/pipeline.json`
Find the ground, then heights (one job)	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/ground_hag/pipeline.json`

Job	Tool	Command(s)
Find the ground, then heights (one job)	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/ground_hag/pipeline.json`
Find the ground, then heights (one job)	LAStools	`/home/zy/LASTools/lastools/bin/lasground_new64 -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/ground_hag/ground.laz /home/zy/LASTools/lastools/bin/lasheight64 -i /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/ground_hag/ground.laz -store_as_extra_bytes -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/ground_hag/out.laz`
Find the ground, then heights (one job)	QGIS engine	`/usr/lib/qgis/pdal_wrench classify_ground --input=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --output=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/ground_hag/ground.laz --threads=24 /usr/lib/qgis/pdal_wrench height_above_ground --input=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/ground_hag/ground.laz --output=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/ground_hag/out.laz --algorithm=nn --replace-z=false --nn-count=1 --nn-max-distance=0 --threads=24`
Find the ground, then heights (one job)	QGIS	`/usr/bin/qgis_process run pdal:classifyground --INPUT=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --OUTPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/ground_hag/ground.laz --VPC_OUTPUT_FORMAT=0 /usr/bin/qgis_process run pdal:heightabovegroundbynearestneighbor --INPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/ground_hag/ground.laz --REPLACE_Z=false --COUNT=1 --MAX_DISTANCE=0 --OUTPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/ground_hag/out.laz --VPC_OUTPUT_FORMAT=0`
Bare-earth elevation map (DTM)	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/dtm/pipeline.json`
Bare-earth elevation map (DTM)	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/dtm/pipeline.json`
Bare-earth elevation map (DTM)	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/dtm/pipeline.json`
Bare-earth elevation map (DTM)	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/dtm/pipeline.json`
Bare-earth elevation map (DTM)	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/dtm/pipeline.json`
Bare-earth elevation map (DTM)	LAStools	`/home/zy/LASTools/lastools/bin/las2dem64 -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m-ground.laz -keep_class 2 -step 1 -elevation -otif -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/dtm/dtm.tif`
Bare-earth elevation map (DTM)	QGIS engine	`/usr/lib/qgis/pdal_wrench to_raster --input=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m-ground.laz --output=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/dtm/dtm.tif --attribute=Z --resolution=1 --tile-size=1000 --filter=Classification == 2 --threads=24`

Job	Tool	Command(s)
Bare-earth elevation map (DTM)	QGIS	`/usr/bin/qgis_process run pdal:exportraster --INPUT=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m-ground.laz --OUTPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/dtm/dtm.tif --ATTRIBUTE=Z --RESOLUTION=1 --TILE_SIZE=1000 --FILTER_EXPRESSION=Classification = 2`
Surface elevation map (DSM)	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/dsm/pipeline.json`
Surface elevation map (DSM)	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/dsm/pipeline.json`
Surface elevation map (DSM)	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/dsm/pipeline.json`
Surface elevation map (DSM)	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/dsm/pipeline.json`
Surface elevation map (DSM)	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/dsm/pipeline.json`
Surface elevation map (DSM)	LAStools	`/home/zy/LASTools/lastools/bin/lasgrid64 -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz -first_only -highest -step 1 -otif -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/dsm/dsm.tif`
Surface elevation map (DSM)	QGIS engine	`/usr/lib/qgis/pdal_wrench to_raster --input=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --output=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/dsm/dsm.tif --attribute=Z --resolution=1 --tile-size=1000 --filter=ReturnNumber == 1 --threads=24`
Surface elevation map (DSM)	QGIS	`/usr/bin/qgis_process run pdal:exportraster --INPUT=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --OUTPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/dsm/dsm.tif --ATTRIBUTE=Z --RESOLUTION=1 --TILE_SIZE=1000 --FILTER_EXPRESSION=ReturnNumber = 1`
Point density map	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/density/pipeline.json`
Point density map	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/density/pipeline.json`
Point density map	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/density/pipeline.json`
Point density map	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/density/pipeline.json`
Point density map	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/density/pipeline.json`
Point density map	LAStools	`/home/zy/LASTools/lastools/bin/lasgrid64 -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz -counter -step 1 -otif -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/density/density.tif`

Job	Tool	Command(s)
Point density map	QGIS engine	`/usr/lib/qgis/pdal_wrench density --input=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --output=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/density/density.tif --resolution=1 --tile-size=1000 --threads=24`
Point density map	QGIS	`/usr/bin/qgis_process run pdal:density --INPUT=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --OUTPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/density/density.tif --RESOLUTION=1 --TILE_SIZE=1000`
Outline of the covered area	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg info --boundary /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz`
Outline of the covered area	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg info --boundary /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz`
Outline of the covered area	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg info --boundary /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz`
Outline of the covered area	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal info --boundary /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz`
Outline of the covered area	PDAL 2.10.1 pkg	`/usr/bin/pdal info --boundary /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz`
Outline of the covered area	LAStools	`/home/zy/LASTools/lastools/bin/lasboundary64 -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/boundary/boundary.shp`
Outline of the covered area	QGIS engine	`/usr/lib/qgis/pdal_wrench boundary --input=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --output=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/boundary/boundary.gpkg --threads=24`
Outline of the covered area	QGIS	`/usr/bin/qgis_process run pdal:boundary --INPUT=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --OUTPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/boundary/boundary.gpkg`
Find noise points	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/outlier/pipeline.json`
Find noise points	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/outlier/pipeline.json`
Find noise points	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/outlier/pipeline.json`
Find noise points	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/outlier/pipeline.json`
Find noise points	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/outlier/pipeline.json`
Find noise points	LAStools	`/home/zy/LASTools/lastools/bin/lasnoise64 -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/outlier/out.laz`

Job	Tool	Command(s)
Find noise points	QGIS engine	`/usr/lib/qgis/pdal_wrench filter_noise --input=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --output=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/outlier/out.laz --algorithm=statistical --remove-noise-points=false --statistical-mean-k=8 --statistical-multiplier=2 --threads=24`
Find noise points	QGIS	`/usr/bin/qgis_process run pdal:filternoisestatistical --INPUT=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --OUTPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/outlier/out.laz --MEAN_K=8 --MULTIPLIER=2 --REMOVE_NOISE_POINTS=false --VPC_OUTPUT_FORMAT=0`
Remove noise, then thin (one job)	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/denoise_thin/pipeline.json`
Remove noise, then thin (one job)	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/denoise_thin/pipeline.json`
Remove noise, then thin (one job)	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/denoise_thin/pipeline.json`
Remove noise, then thin (one job)	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/denoise_thin/pipeline.json`
Remove noise, then thin (one job)	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/denoise_thin/pipeline.json`
Remove noise, then thin (one job)	LAStools	`/home/zy/LASTools/lastools/bin/lasnoise64 -i /home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz -remove_noise -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/denoise_thin/clean.laz /home/zy/LASTools/lastools/bin/lasthin64 -i /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/denoise_thin/clean.laz -step 1 -o /home/zy/dev/pdal-gpu/build/independent-bench/out/lastools/1m/denoise_thin/out.laz`
Remove noise, then thin (one job)	QGIS engine	`/usr/lib/qgis/pdal_wrench filter_noise --input=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --output=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/denoise_thin/clean.laz --algorithm=statistical --remove-noise-points=true --statistical-mean-k=8 --statistical-multiplier=2 --threads=24 /usr/lib/qgis/pdal_wrench thin --input=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/denoise_thin/clean.laz --output=/home/zy/dev/pdal-gpu/build/independent-bench/out/wrench/1m/denoise_thin/out.laz --mode=sample --step-sample=1 --threads=24`
Remove noise, then thin (one job)	QGIS	`/usr/bin/qgis_process run pdal:filternoisestatistical --INPUT=/home/zy/dev/pdal-gpu/build/independent-bench/in/1m.laz --OUTPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/denoise_thin/clean.laz --MEAN_K=8 --MULTIPLIER=2 --REMOVE_NOISE_POINTS=true --VPC_OUTPUT_FORMAT=0 /usr/bin/qgis_process run pdal:thinbyradius --INPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/denoise_thin/clean.laz --SAMPLING_RADIUS=1 --OUTPUT=/home/zy/dev/pdal-gpu/build/independent-bench/out/qgis/1m/denoise_thin/out.laz --VPC_OUTPUT_FORMAT=0`

Job	Tool	Command(s)
Ground, noise, and neighbour vote (one job)	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/classify_refine/pipeline.json`
Ground, noise, and neighbour vote (one job)	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/classify_refine/pipeline.json`
Ground, noise, and neighbour vote (one job)	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/classify_refine/pipeline.json`
Ground, noise, and neighbour vote (one job)	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/classify_refine/pipeline.json`
Ground, noise, and neighbour vote (one job)	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/classify_refine/pipeline.json`
Surface normals and shape features	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/features/pipeline.json`
Surface normals and shape features	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/features/pipeline.json`
Surface normals and shape features	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/features/pipeline.json`
Surface normals and shape features	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/features/pipeline.json`
Surface normals and shape features	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/features/pipeline.json`
Colour the points from an aerial image	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/colorize/pipeline.json`
Colour the points from an aerial image	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/colorize/pipeline.json`
Colour the points from an aerial image	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/colorize/pipeline.json`
Colour the points from an aerial image	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/colorize/pipeline.json`
Colour the points from an aerial image	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/colorize/pipeline.json`
Read a window from a COPC file	GPUPDAL (GPU)	`/home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu/1m/copc_query/pipeline.json`

Job	Tool	Command(s)
Read a window from a COPC file	GPUPDAL (CPU only)	`CUDA_VISIBLE_DEVICES='' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_cpu/1m/copc_query/pipeline.json`
Read a window from a COPC file	GPUPDAL (GPU forced)	`PDG_EXPERIMENTAL_CUDA_HYBRID='1' /home/zy/dev/pdal-gpu/build/pdg-cuda-release/bin/pdg pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdg_gpu_all/1m/copc_query/pipeline.json`
Read a window from a COPC file	PDAL 2.10.0	`/home/zy/dev/pdal-gpu/build/pdal-upstream-tests/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_pinned/1m/copc_query/pipeline.json`
Read a window from a COPC file	PDAL 2.10.1 pkg	`/usr/bin/pdal pipeline /home/zy/dev/pdal-gpu/build/independent-bench/out/pdal_sys/1m/copc_query/pipeline.json`

Pipeline JSON files (for GPUPDAL and PDAL) are written by the harness; their contents are in `bench/independent/tasks.py`. Paths were shortened for readability where they only differ by tool name.

9.4 How to repeat this

```
python3 bench/independent/prepare.py            # stage the inputs (needs the local corpus and pinned PDAL
python3 bench/independent/harness.py --label workstation --sizes 1m --repeats 3 --keep-outputs
python3 bench/independent/harness.py --label workstation --sizes 4m --repeats 3
python3 bench/independent/harness.py --label workstation --sizes 16m --repeats 3
python3 bench/independent/harness.py --label workstation --sizes 47m --repeats 3
python3 bench/independent/render.py               # pictures of inputs and outputs
python3 bench/independent/report.py               # charts, HTML and PDF
# on a rented box, after bench/remote/vast_bootstrap.sh:
bash bench/independent/remote_setup.sh <label> lastools.tar.gz /workspace/fix 1m 4m 16m 47m
```

Raw results (every repeat, every output's size, point count and checksum): `build/independent-bench/results/*.json`. The published bundle's `manifest.json` hashes all three result files and every staged input and records that this was author-run evidence.